

Big Data, méthodes et solutions pratiques pour l'analyse des données



Mehdi TAJMOUATI

Sommaire :

Part 1

1.1 Comprendre les concepts et les enjeux du Big Data

- Pourquoi parle-t-on de Big Data
- Qu'est-ce que le Big Data ?
- Les origines du Big Data
- Architecture Big Data
- Les enjeux éthiques et juridiques du Big Data
- Sécurité des données

Sommaire :

Part 2

2.1 Les technologies du Big Data

- L'architecture Hadoop : le socle du Big Data
- Les modes de stockage (NoSQL, HDFS)
- Architectures de calcul distribuées
- Les Plateformes Big DataLes technologies du Data Scientist
- De l'ingestion à la visualisation : Talend, Qlik, Tableau & co

Sommaire :

Part 3

3.1 Fondamentaux du Data Engineering

- HDFS : Principes et Fonctionnement
- HDFS : Les données vers HDFS
- HIVE : database & SQL
- Apache PIG
- Introduction aux ETL

Sommaire :

Part 4

4.1 Gestion de streaming de données massive (NIFI, Kafka, Spark, Storm...)

- Introduction Spark Streaming
- Structured Streaming
- Streaming DataFrame
- readStream
- Transformation
- window & Kafka
- writeStream
- foreachBatch
- Intégration

1.1

Comprendre les concepts et les enjeux du Big Data



Comprendre les concepts et les enjeux du Big Data



1. Pourquoi parle-t-on de Big Data



2. Qu'est-ce que le Big Data ?

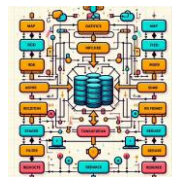


3. Les origines du Big Data ?

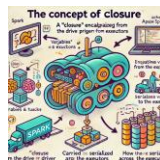
Wide Transformations (shuffles)
1 to N



4. Architecture Big Data ?



5. Les enjeux éthiques et juridiques du Big Data

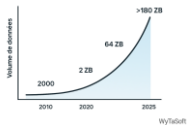


6. Sécurité des données

Pourquoi parle-t-on de Big Data



Pourquoi parle-t-on de Big Data



Sommes-nous submergés par les données ?



Une réalité vertigineuse

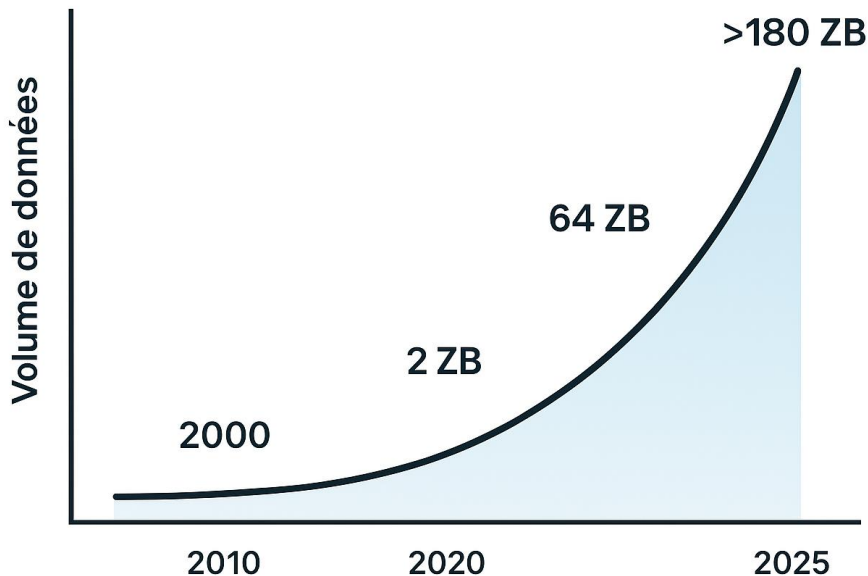


D'où viennent toutes ces données ?



Transformer la donnée en valeur business

Sommes-nous submergés par les données ?

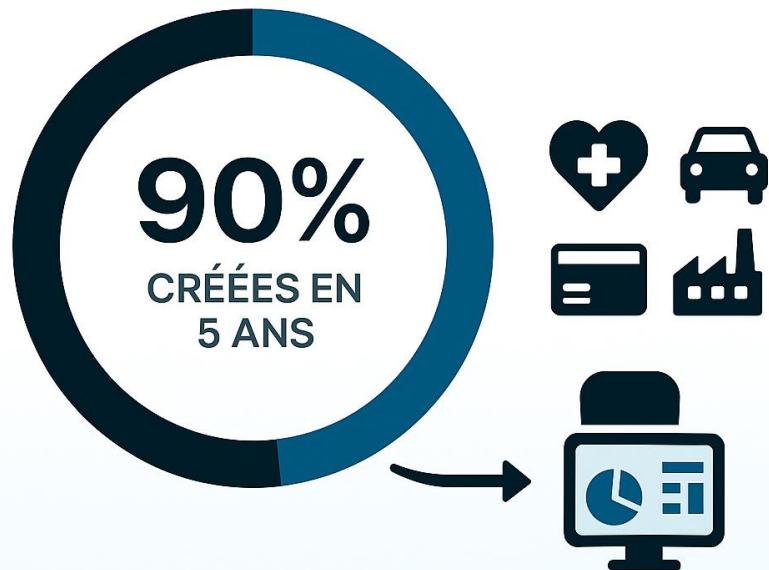


WyTaSoft

D'où viennent toutes ces données ?



Une réalité vertigineuse



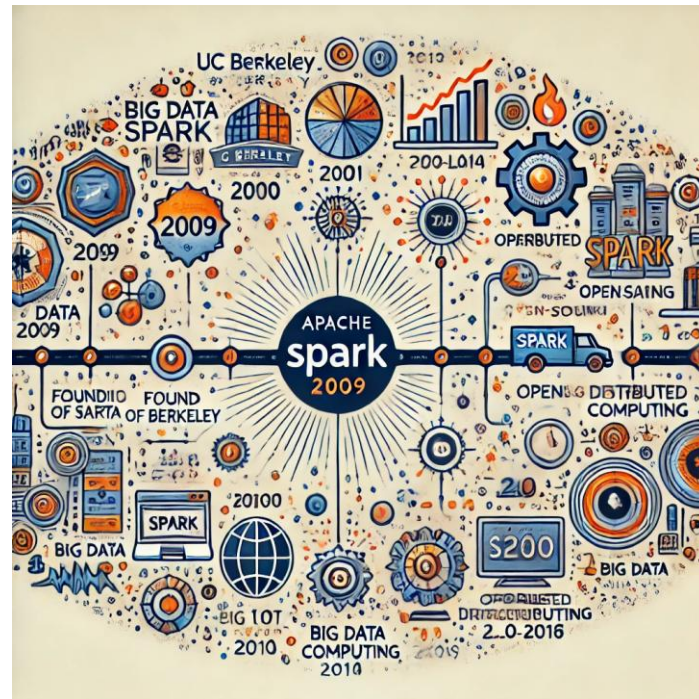
WyTaSoft

Transformer la donnée en valeur **business**

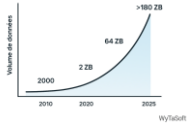


Comprendre les concepts et les enjeux du Big Data

Qu'est-ce que le Big Data ?



Qu'est-ce que le Big Data ?



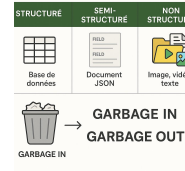
Sommes-nous submergés par les données ?



Le deuxième V :
Vélocité ?



Le premier V : Volume ?



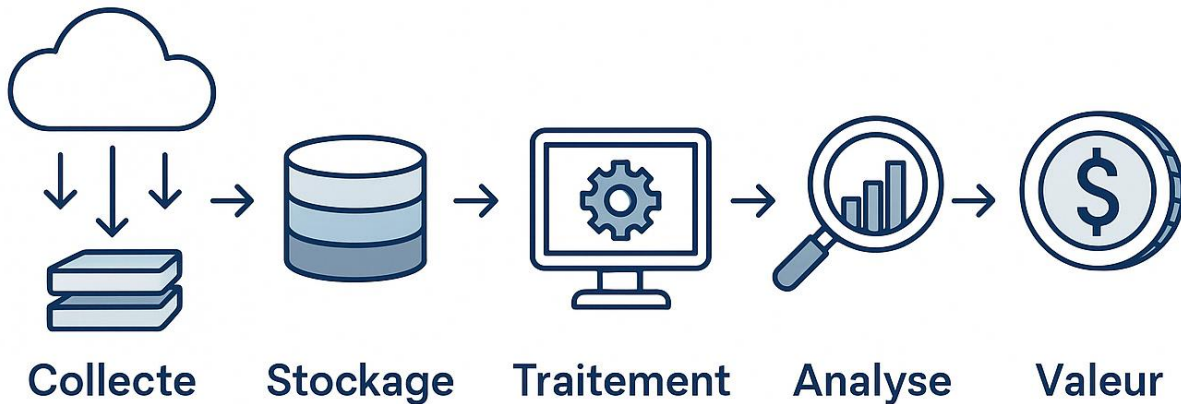
Variété et Véracité ?



Le cinquième V : Valeur ?

Qu'est-ce que le **Big Data** ?

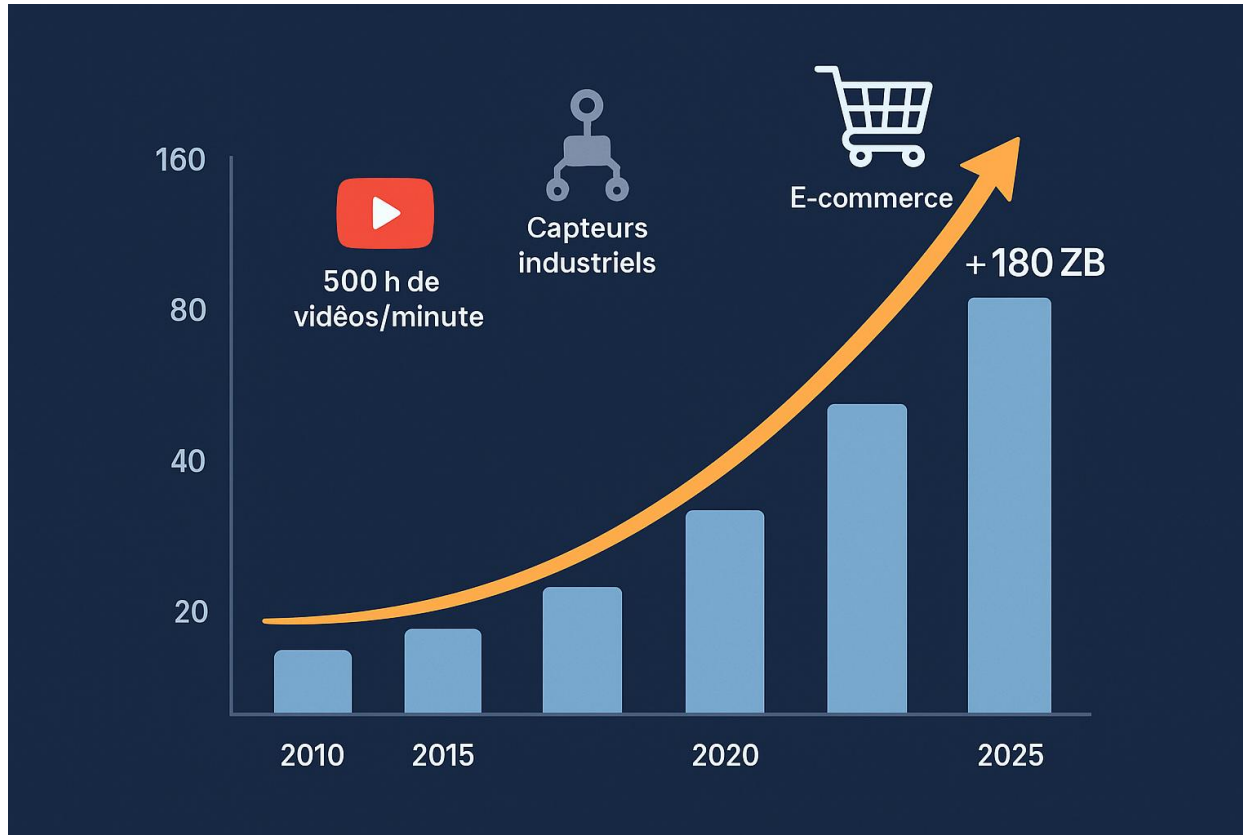
- Le Big Data désigne l'ensemble des technologies, méthodes et pratiques permettant de **collecter, stocker, traiter et analyser des volumes massifs de données**, souvent non structurées, à **grande vitesse**, pour en **extraire de la valeur**.



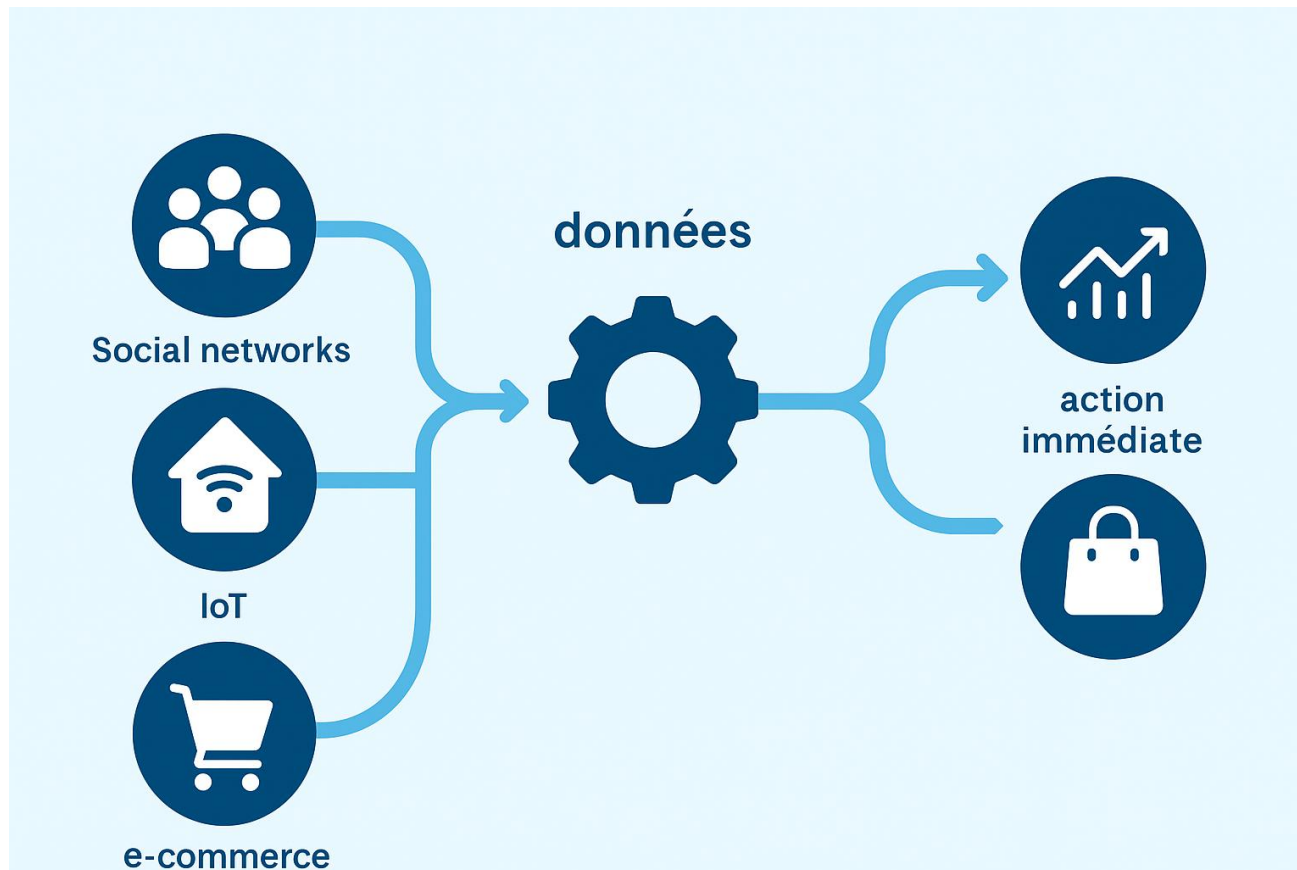
Le premier V : Volume

- Le Big Data, c'est avant tout une question d'échelle.
- Les données proviennent de milliards de sources : clics web, vidéos, capteurs IoT, transactions, etc.
- On parle aujourd'hui de **zettaoctets** (10^{21} octets).

Le premier V : **Volume**

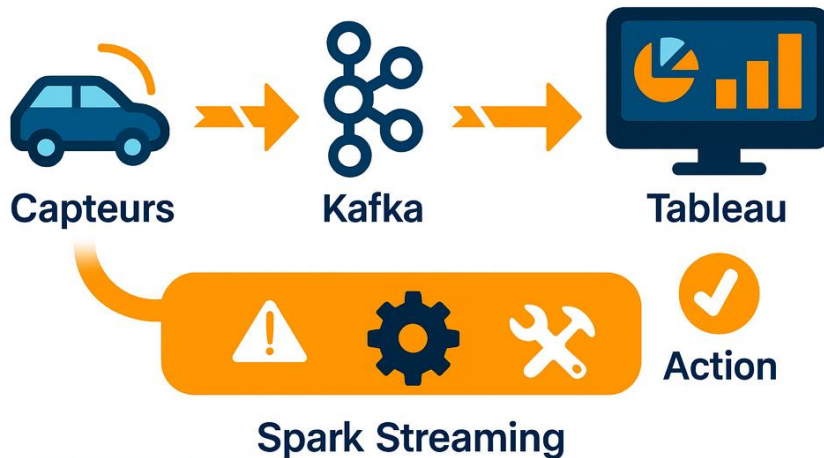


Le deuxième V : **V**élocité



Vélocité : la donnée en temps réel

De la donnée brute
à l'action instantanée



Variété

STRUCTURÉ	SEMI-STRUCTURÉ	NON STRUCTURÉ
		
Base de données	Document JSON	Image, vidéo, texte



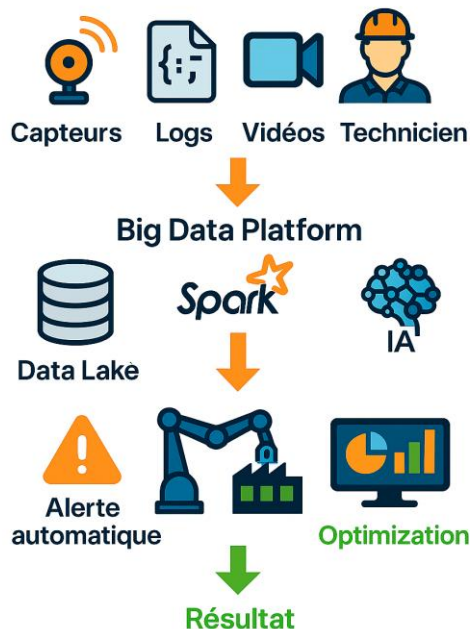
GARBAGE IN



GARBAGE IN
GARBAGE OUT

Variété : use case : la donnée au service de la maintenance prédictive

Maintenance prédictive – L'usine connectée

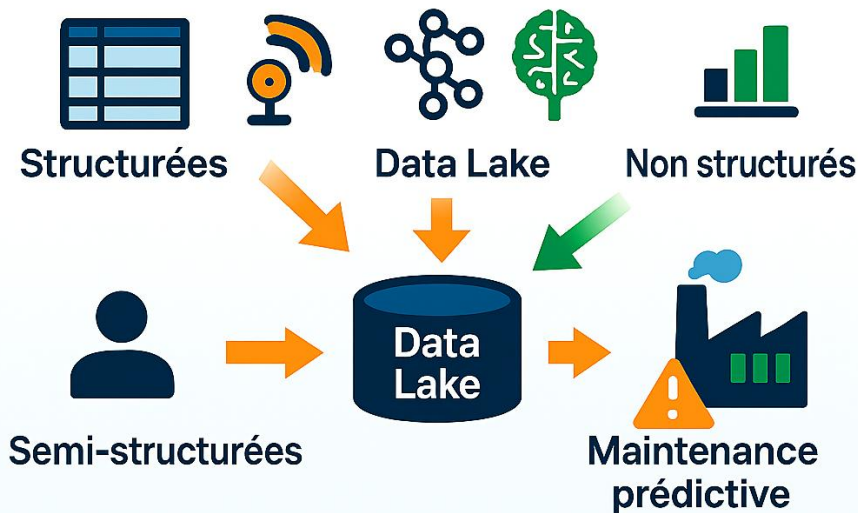


Le groupe aéronautique **Safran** cherche à **réduire les arrêts non planifiés** et à **optimiser la maintenance** de ses moteurs d'avion et équipements aéronautiques.

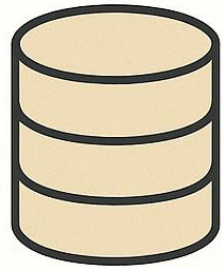
Chaque atelier de production et chaque moteur d'essai intègre des **centaines de capteurs connectés** générant des volumes massifs de **données hétérogènes**.

Variété : use case : la donnée au service de la maintenance prédictive

Les données variées au cœur de l'usine connectée



Véracité



Donnée
brute



Information



Connaissance



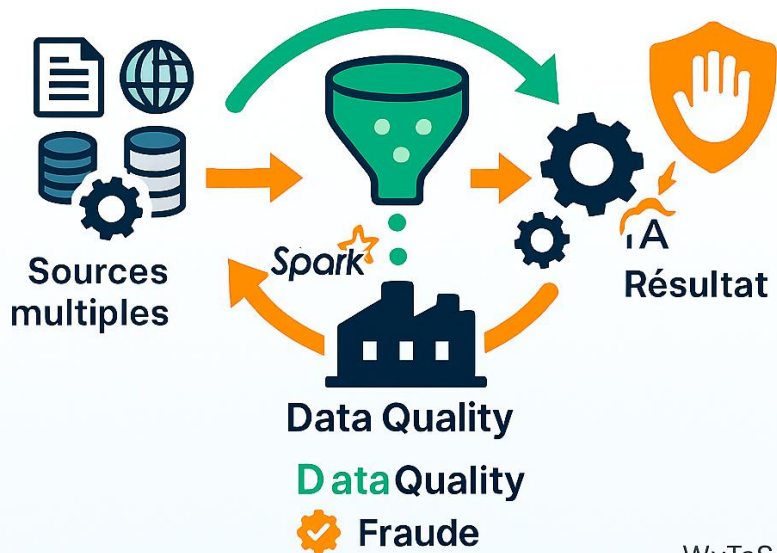
Valeur



GARBAGE OUT

Véracité

La véracité des données au service de la confiance bancaire



WyTaSoft

Une grande banque comme **BNP Paribas** ou **Société Générale** gère chaque jour **des milliards de transactions financières**.

Ces données sont critiques : une erreur, un doublon ou une fraude peuvent avoir un **impact majeur** sur la conformité réglementaire et la confiance client.

Véracité



Fiabilité accrue des transactions analysées
(+98 %)



Détection de fraude en moins d'une seconde



Réduction des faux positifs de 30 %



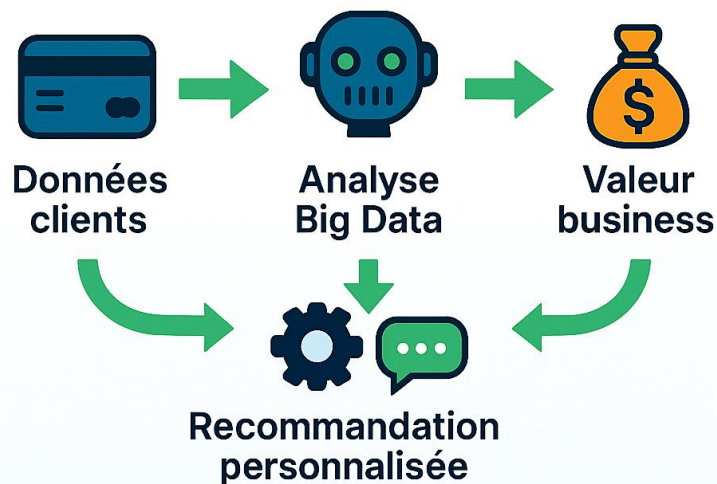
Amélioration de la conformité RGPD et des audits internes

Le cinquième V : Valeur

- La **valeur** est le but ultime du Big Data.
- Transformer la donnée brute en **information utile** pour créer un avantage compétitif.
- Exemples :
 - Recommandations Amazon / Netflix
 - Maintenance predictive
 - Optimisation logistique
 - Personnalisation marketing

Le cinquième V : Valeur

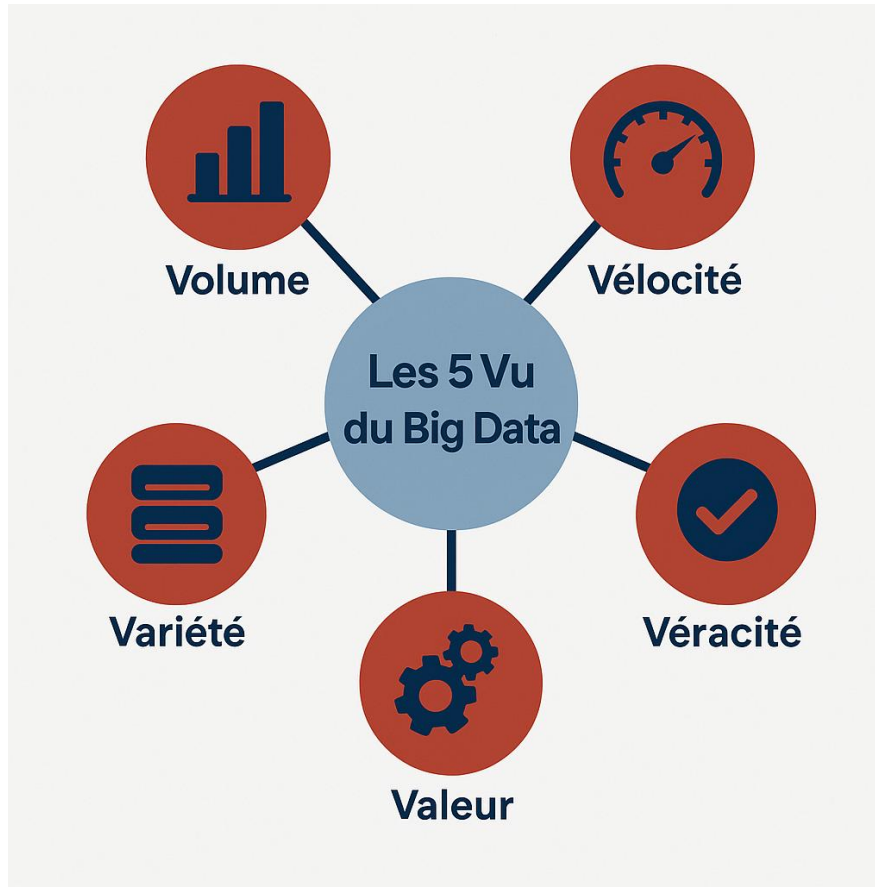
De la donnée à à valeur client



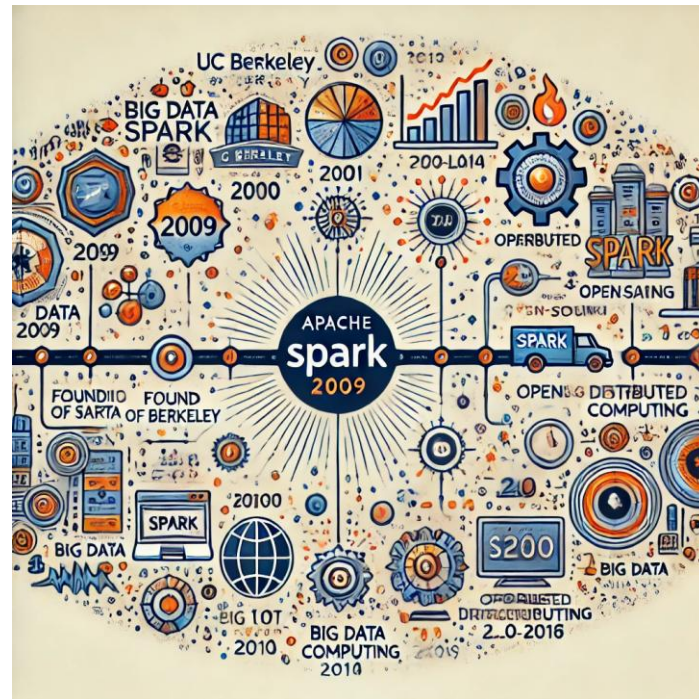
WyTaSoft

Une grande banque comme **Crédit Agricole** ou **BNP Paribas** veut améliorer la **relation client** et **augmenter le taux de conversion** de ses produits (crédit, assurance, épargne).

Les 5V du Big Data



Les origines du Big Data ?



Les origines du Big Data

Avant le Big Data :
la donnée traditionnelle



Avant le Big Data : la donnée traditionnelle

3. 2001 – la naissance
du concept « Big Data »
Diag. Larry Green (chez Meta/Google)
formalise les 3V du Big Data :



2001 – la naissance du
concept « Big Data »

L'explosion numérique



L'explosion numérique



Les années 2010 : l'ère du
Cloud et des technologies
distribuées



Aujourd'hui : le Big Data à
l'ère de l'IA

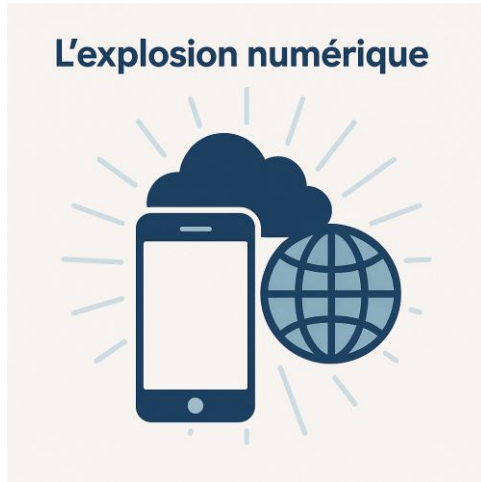
Avant le Big Data : la donnée traditionnelle

Avant le Big Data : la donnée traditionnelle



- Avant les années 2000, les entreprises stockaient principalement des données **structurées** dans des bases relationnelles (SQL, ERP, CRM).
- Le volume restait **limité et maîtrisé**, car les sources étaient essentiellement internes.
- Les systèmes étaient **centralisés** et souvent coûteux à faire évoluer.

L'explosion numérique

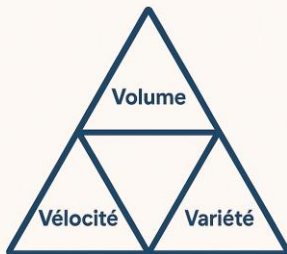


- Début des années 2000 : arrivée du **Web 2.0**, des **réseaux sociaux**, des **smartphones** et du **cloud computing**.
- Multiplication des **sources de données** : clics, vidéos, géolocalisation, capteurs, objets connectés (IoT).
- Résultat : un volume de données **impossible à traiter avec les outils classiques**.

2001 – la naissance du concept « **Big Data** »

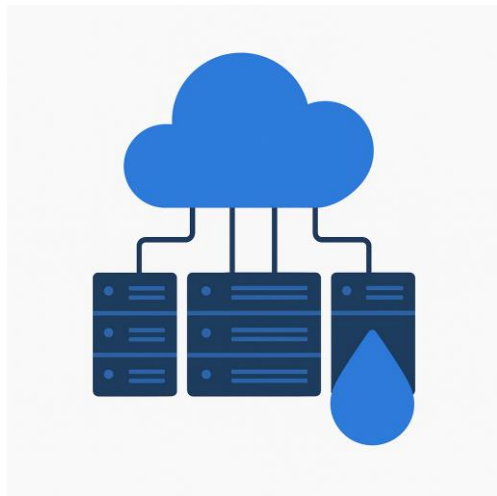
3. 2001 – la naissance du concept «**Big Data**»

Doug Laney (analyste chez Meta/Gartner) formalise les 3V du Big Data :



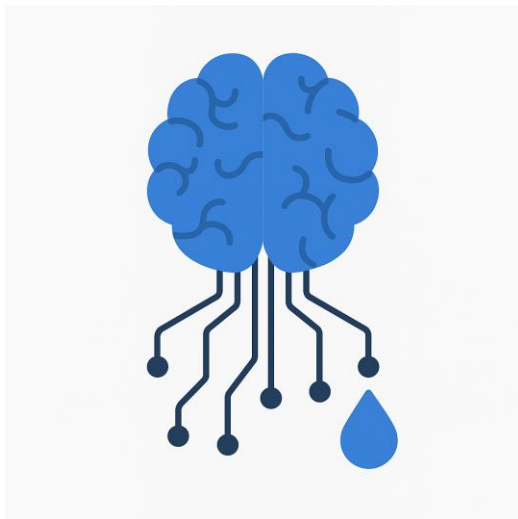
- Doug Laney (analyste chez Meta/Gartner) formalise les **3V du Big Data** :
 - **Volume** : explosion de la quantité de données
 - **Vélocité** : flux en temps réel,
 - **Variété** : diversité des formats.
- Ces “3V” deviendront la **base conceptuelle du Big Data moderne**.

Les années 2010 : l'ère du Cloud et des technologies distribuées



- Montée d'**Hadoop** (Yahoo, 2005), puis **Spark**, permettant de **distribuer le stockage et le calcul** sur des clusters.
- Le **Cloud** (AWS, GCP, Azure) démocratise ces technologies : on peut maintenant stocker et traiter des pétaoctets de données à la demande.
- Émergence du concept de **Data Lake**.

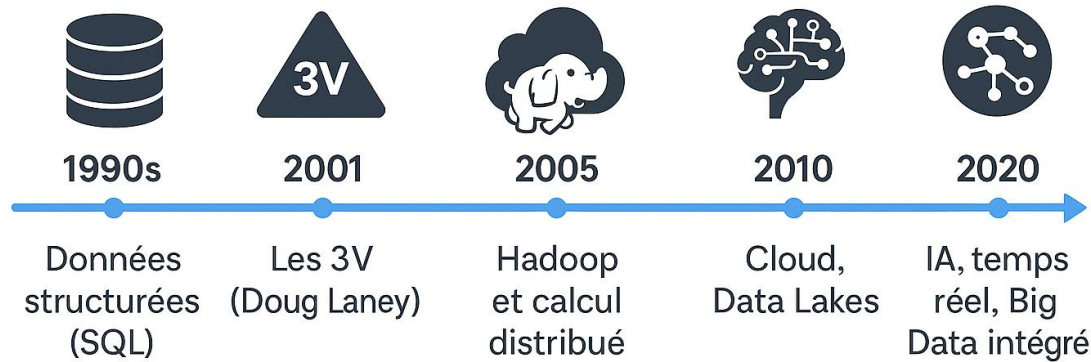
Aujourd'hui : le **Big Data** à l'ère de l'**IA**



- Les technologies Big Data sont désormais la **colonne vertébrale de l'Intelligence Artificielle**.
- Les modèles d'IA et d'apprentissage automatique reposent sur des volumes massifs de données pour s'entraîner.
- On parle désormais d'**écosystèmes data modernes** (Data Lakehouse, temps réel, IA générative).

Les origines du Big Data

Les origines du Big Data



Chiffres clés du Big Data



Un marché mondial en pleine explosion



L'adoption massive de la data



Le marché français du Big Data



la donnée, un actif stratégique

Un marché mondial en pleine explosion



\$390

milliards

Marché mondial



+13 %

Croissance
annuelle moyennée

Le marché mondial du Big Data et de l'analytics est estimé à environ 390 milliards de dollars en 2025, selon IDC

Le marché français du Big Data

En France, le marché du Big Data est évalué à

4,9 Md€

En 2025 (IDC France)

Les principaux secteurs utilisateurs:



Banque / Assurance



Santé et services publics



Retail / e-commerce

La France connaît une forte demande en data engineers, analysts, et data scientists.

Source : IDC France 2024 - "Big Data Market in Western Europe"

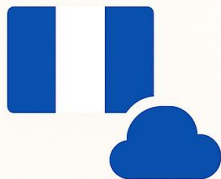
L'adoption massive de la data



En résumé : la donnée, un **actif stratégique**



Marché mondial:
~ **400 Mds \$**
croissance
+13%/an



France:
- 5 Mds €
forte adoption
sectorielle

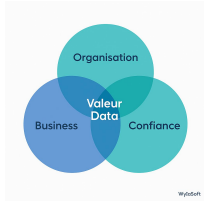


97 %
des entreprises
investissent
dans la donnée

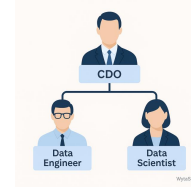


Les entreprises
data-driven
surperforment
leurs concurrentes

Les enjeux **stratégiques** du Big Data



Les 3 grands piliers
stratégiques du Big Data



Organisation &
nouveaux métiers

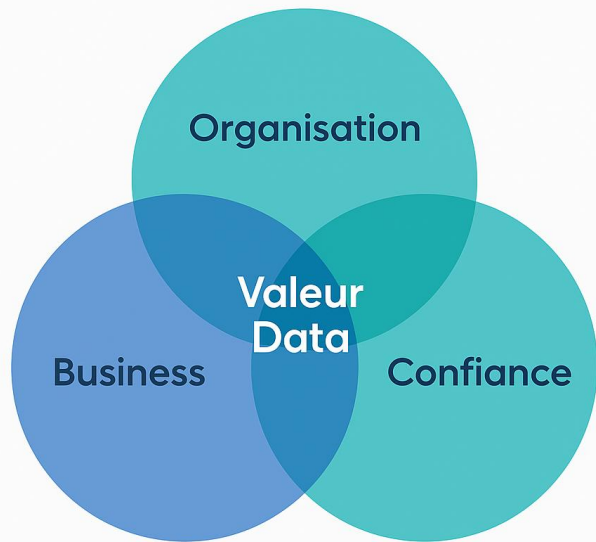


ROI & Performance



Confiance & conformité

Les 3 grands piliers stratégiques du Big Data



WyiaSoft

- Le Big Data n'est pas qu'un enjeu technique. C'est d'abord un **levier stratégique** qui touche à trois dimensions clés :
- Le **business**, avec le retour sur investissement,
- L'**organisation**, avec de nouveaux métiers et une culture data-driven,
- Et la **confiance**, avec la confidentialité et la conformité des données.

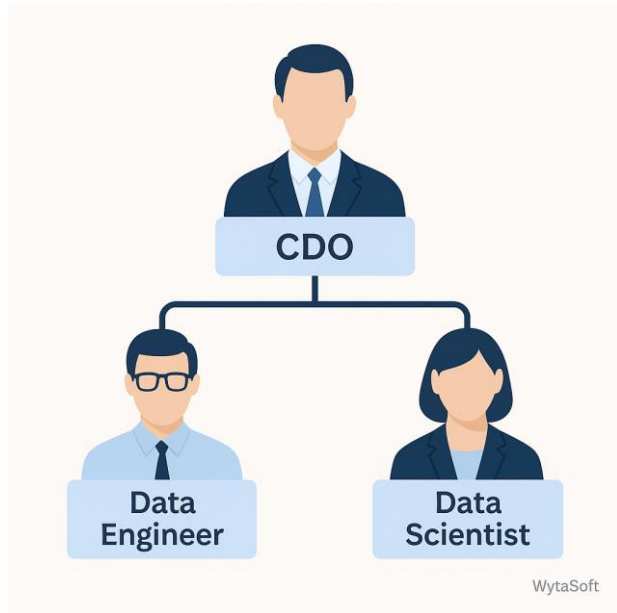
ROI & Performance



- Objectif : **maximiser la valeur économique** des données.
- Leviers principaux :
 - **Optimisation des processus** (ex : maintenance prédictive, logistique)
 - **Réduction des coûts** (automatisation, efficacité opérationnelle)
 - **Personnalisation client** (recommandations, segmentation fine, expérience utilisateur).

Les entreprises data-driven génèrent **jusqu'à 20 % de marge supplémentaire** (source : McKinsey).

Organisation & nouveaux métiers



- L'adoption du Big Data s'accompagne d'une **transformation culturelle et organisationnelle**.
- Nouveaux rôles clés :
 - **Data Engineer** : construit et maintient les pipelines de données
 - **Data Scientist** : modélise et analyse pour générer des insights
 - **Chief Data Officer (CDO)** : pilote la stratégie et la gouvernance des données

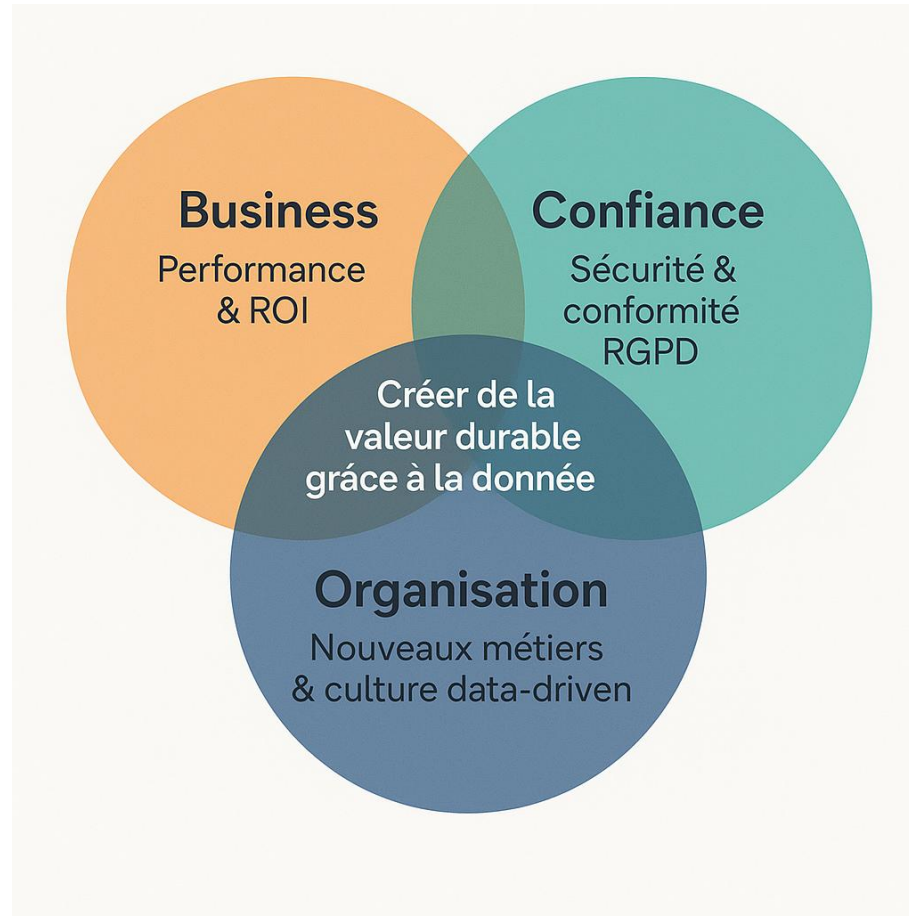
L'entreprise devient **data-driven** : la donnée guide les décisions.

Confiance & conformité



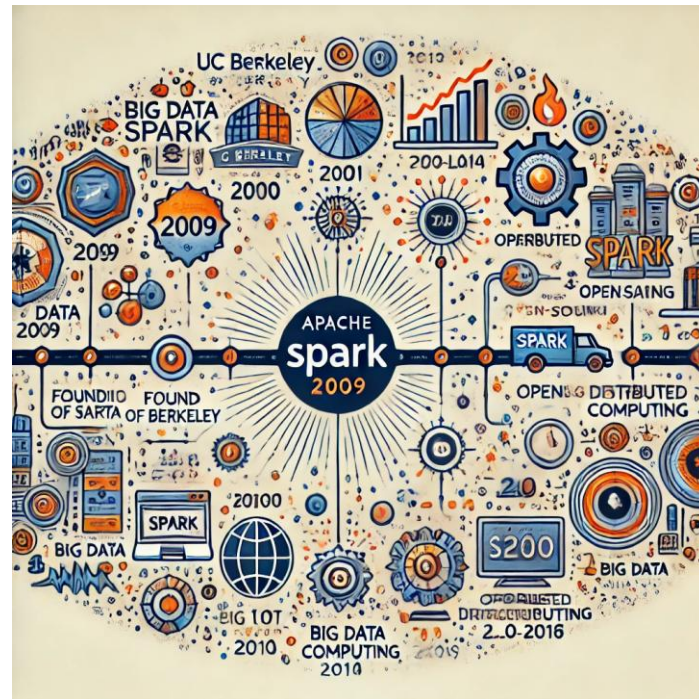
- Le Big Data implique une **responsabilité forte** dans la gestion des données.
- Enjeux principaux :
 - **Confidentialité & sécurité** : protection contre les fuites ou cyberattaques
 - **Conformité RGPD** : consentement, droit à l'oubli, minimisation
 - **Gouvernance & anonymisation** : gestion des accès, traçabilité, auditabilité
- Sans confiance, aucune stratégie data n'est durable.

un sujet **technologique**, ou un système stratégique



Architecture

Big Data ?



Architecture Big Data



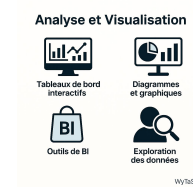
Collecte des données



Traitement des données



Ingestion des données



Analyse et visualisation

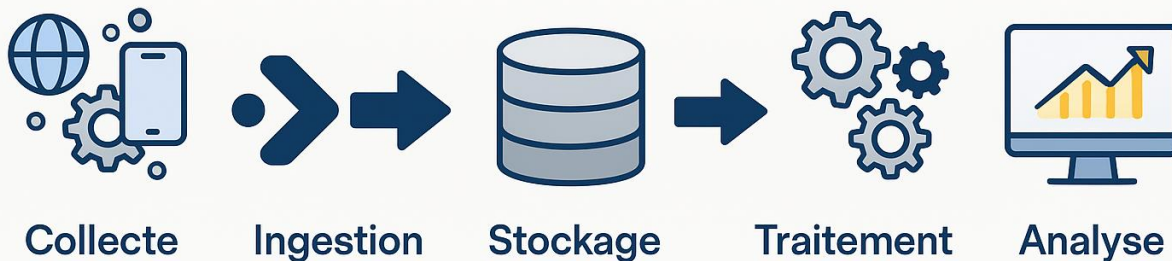


Stockage des données



le pipeline complet

Comprendre une architecture **Big Data**



Collecte des données

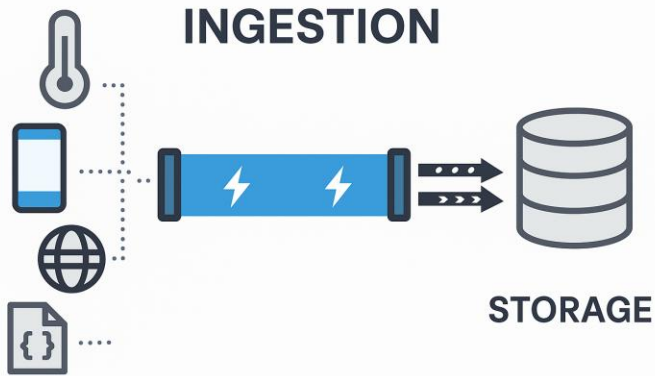


WyTaSoft

- Les sources sont multiples :
 - 🌐 **API** (sites web, apps)
 - ⚙️ **Capteurs IoT**
 - 💻 **Logs systèmes / applications**
 - 📱 **Réseaux sociaux, plateformes clients**

Les données arrivent souvent en flux continu (*streaming*).

Ingestion des données



WyTaSoft

- Objectif : acheminer et centraliser les données.
- Outils courants :
 - **Kafka, Dataflow, NiFi**
- Deux modes :
 - **Batch** → chargements périodiques
 - **Streaming** → flux en temps réel

Stockage des données



Unified
storage



Batch and
streaming



Lakehouse



Data
management

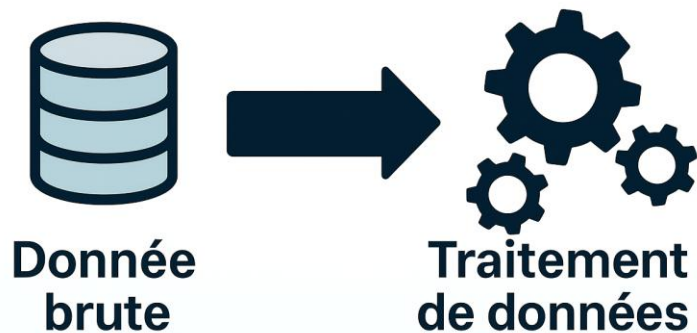


Transactional
support

WyTaSoft

- Deux grandes approches :
 - **Data Lake** : stockage brut (S3, GCS, Azure Data Lake)
 - **Data Warehouse** : stockage structuré (BigQuery, Snowflake, Redshift)
- Le **Data Lakehouse** combine les deux (modern data architecture).

Traitement des données



WyTaSoft

- Objectif : transformer la donnée brute en donnée exploitable:
- Outils principaux :
 - Spark, Dataproc, Flink
- Types de traitements :
 - Batch (traitement périodique)
 - Streaming (temps réel)

Analyse et visualisation

Analyse et Visualisation



Tableaux de bord
interactifs



Diagrammes
et graphiques



Outils de BI

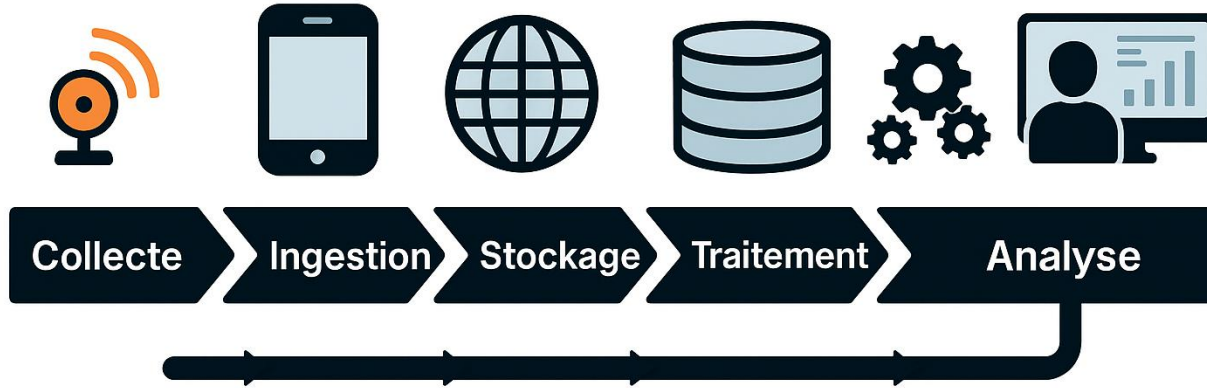


Exploration
des données

WyTaSoft

- Dernière étape : rendre la donnée **lisible et utile**.
- Outils courants : **Looker, Power BI, Tableau, Data Studio**.
- Objectif : permettre aux métiers de prendre des décisions data-driven.

Le **pipeline** complet



Comprendre les concepts et les enjeux du Big Data

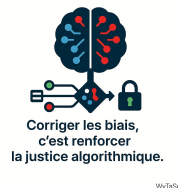
Les enjeux éthiques et juridiques du Big Data



Les enjeux éthiques et juridiques du Big Data



La dimension éthique et réglementaire



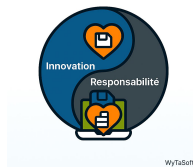
Les biais algorithmiques



Le respect du RGPD

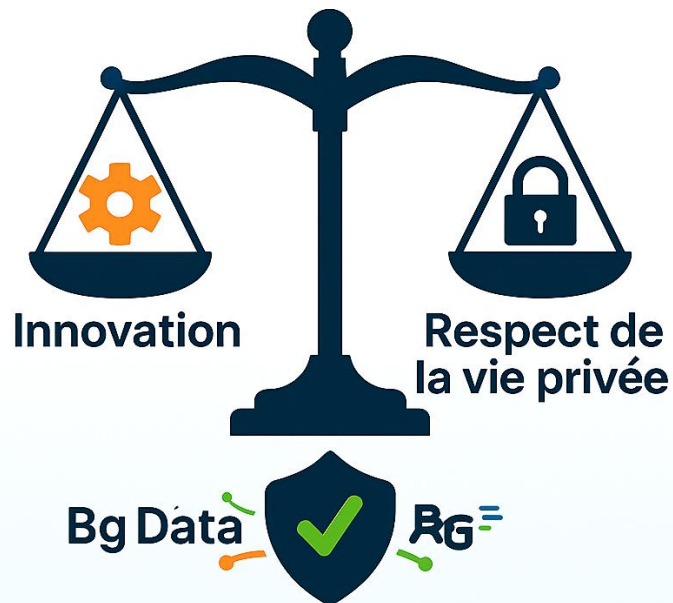


Vers une gouvernance responsable



Synthèse : Innovation & Responsabilité

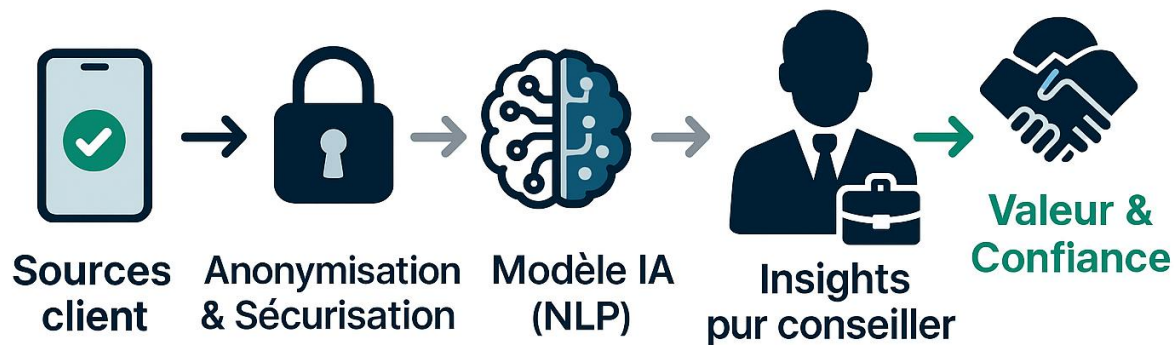
La dimension éthique et réglementaire



- Le Big Data ne se limite pas à la technologie : il soulève aussi des **questions morales, sociales et juridiques**.
- Quand on collecte, analyse ou automatise avec des données, il faut garantir le **respect des droits fondamentaux**, la **transparence** et la **responsabilité**.

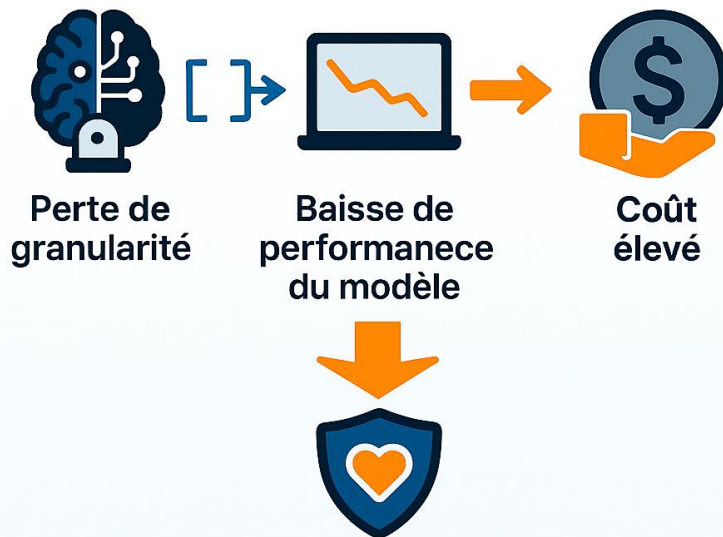
Cas réel : Crédit Agricole – Project « **Advisor Report Analysis** »

Innovation responsable au service du client



Cas réel : Crédit Agricole – Project « **Advisor Report Analysis** »

Anonymisation et pseudonymisation



Le respect du RGPD



- Le RGPD (Règlement Général sur la Protection des Données) encadre la collecte et l'utilisation des données personnelles.
- Principes clés :
 - **Droit à l'oubli** : un individu peut demander la suppression de ses données.
 - **Portabilité** : il peut récupérer ses données dans un format exploitable.
 - **Consentement explicite** : aucune donnée ne peut être utilisée sans accord clair.
- L'entreprise doit aussi assurer la **sécurité** et la **traçabilité** des données.

Le respect du RGPD

Faits marquants



BNP Paribas a provisionné
125 millions d'euros
pour régler une enquête liée
à l'usage de messageries
non approuvées. Reuters

L'amende record s'inscrit dans un
mouvement plus large : les banques
américaines et internationales ont payé
plus de

! \$2,5 milliards

Construire une confiance durable

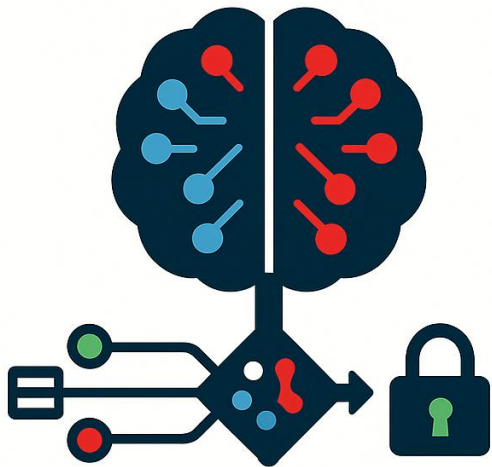
WyTaSoft

WYTA
SOFT

Le respect du RGPD



Les **biais** algorithmiques



**Corriger les biais,
c'est renforcer
la justice algorithmique.**

- Les algorithmes apprennent à partir des données... et peuvent **reproduire les biais humains** qu'ils contiennent.
- Risques : discrimination, décisions injustes, manque d'explicabilité.
- Solutions :
 - Auditer les modèles,
 - Diversifier les données d'entraînement,
 - Garantir la **transparence** et la **reproductibilité** des résultats.

les **biais** algorithmiques à l'ère du Big Data

les biais du Big Data



En 2024, la plateforme RH **Workday** a été visée par une plainte aux États-Unis pour **discrimination** dans les recrutements.

Vers une **gouvernance** responsable



- La **gouvernance des données** garantit que l'usage de la donnée respecte la loi et les valeurs de l'entreprise.
- Notions clés :
 - **Data Stewardship** : suivi, documentation, qualité des données.
 - **Contrôle des usages** : qui accède à quoi, pourquoi, comment.
 - **Charte éthique** : cadre commun pour tous les acteurs.
- Objectif : instaurer une **culture de la responsabilité**.

Synthèse : Innovation & Responsabilité



WyTaSoft

Les 3 piliers de l'éthique du Big Data :

1. **Protection** (RGPD et sécurité)
2. **Équité** (biais algorithmiques)
3. **Responsabilité** (gouvernance)

L'objectif : un équilibre durable entre **Innovation** et **Respect de la vie privée**.

Synthèse : Innovation & Responsabilité



Elon Musk, via ses entreprises (ex : **xAI** ou **Tesla**), illustre la tension constante entre :

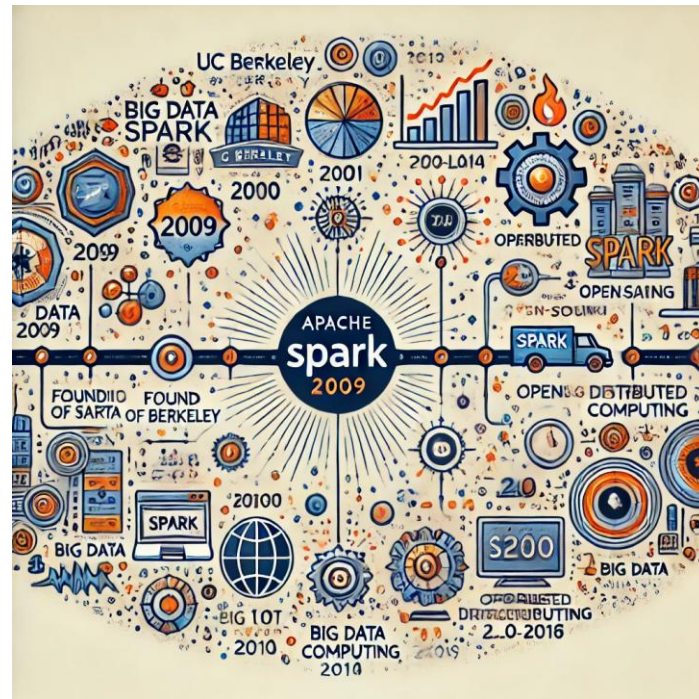
Innovation extrême :

- Véhicules autonomes collectant des milliards de kilomètres de données réelles.
- Modèles d'intelligence artificielle entraînés sur d'énormes corpus ouverts (xAI).

Responsabilité & éthique :

- Respect de la vie privée des conducteurs et passants (caméras embarquées).
- Questions sur la transparence des données d'entraînement et la sécurité algorithmique.

Sécurité des données



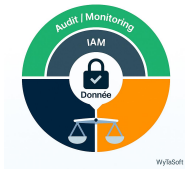
Sécurité des données



Pourquoi la sécurité des données est cruciale



Les outils du Big Data Security



Les couches de défense



Sécurité et conformité



Construire une confiance durable

Pourquoi la **sécurité** des données est cruciale

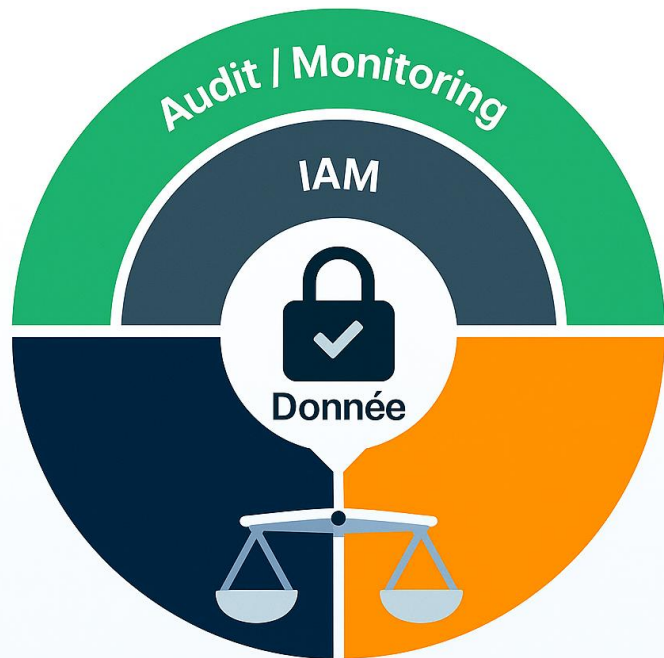
Avec la multiplication des flux de données (IoT, cloud, IA), les risques augmentent :

- **Cyberattaques** : ransomwares, injections, exploitation de failles.
- **Fuites de données** : erreurs humaines, mauvais paramétrage cloud.
- **Accès non autorisé** : absence de contrôle ou d'authentification forte.

Une seule faille peut compromettre la confiance et la conformité réglementaire.



Les couches de **défense**



WyTaSoft

La protection des données repose sur plusieurs **niveaux de sécurité complémentaires** :

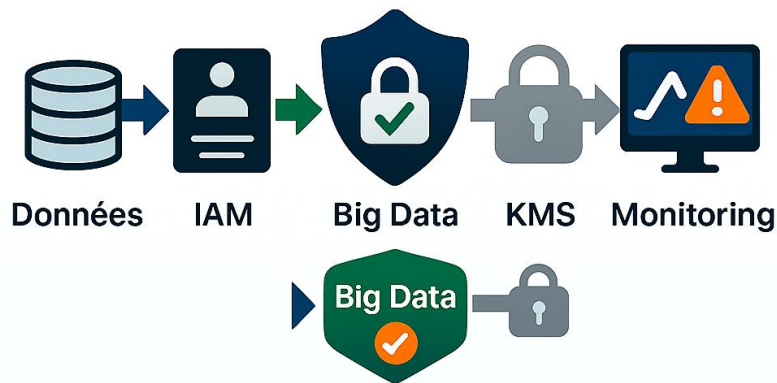
- **Chiffrement** : protéger les données au repos et en transit (AES, TLS).
- **Gouvernance des accès (IAM)** : qui peut accéder à quoi, quand, et pourquoi.
- **Audit et traçabilité** : journalisation et contrôle continu des actions.

L'objectif : détecter, prévenir et réagir avant l'incident.

Les outils du Big Data Security

Les entreprises s'appuient sur des outils spécialisés pour automatiser la sécurité :

- **IAM (Identity & Access Management)** : gestion des droits utilisateurs.
- **DLP (Data Loss Prevention)** : détection et prévention des fuites.
- **KMS (Key Management Service)** : gestion centralisée des clés de chiffrement.
- **SIEM / SOC** : supervision et détection des anomalies en temps réel.



WyTaSoft

Sécurité et conformité

La sécurité des données s'inscrit dans un cadre légal strict :

- **RGPD** : sécurité, consentement, droit à l'oubli.
- **ISO 27001** : norme internationale de gestion de la sécurité.
- **Audits et certifications** : preuve de conformité.

Protéger la donnée, c'est aussi protéger la réputation et la responsabilité juridique.



WyTaSoft

Construire une confiance **durable**



La sécurité des données, c'est plus qu'une exigence technique — c'est un **levier de confiance** :

- Sécuriser
- Surveiller
- Responsabiliser

Une entreprise sécurisée inspire confiance à ses clients, partenaires et collaborateurs.

1.2



Les technologies du Big Data

Les technologies du Big Data



1. L'architecture Hadoop : le socle du Big Data

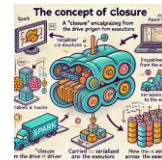
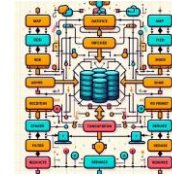


2. Les modes de stockage (NoSQL, HDFS)



3. Architectures de calcul distribuées

Wide Transformations (shuffles)
1 to N



4. Les Plateformes Big Data

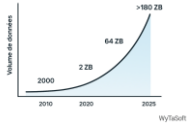
5. Les technologies du Data Scientist

6. De l'ingestion à la visualisation : Talend, Qlik, Tableau & co

L'architecture Hadoop : le socle du Big Data



L'architecture **Hadoop** : le socle du Big Data



Hadoop : un écosystème complet



Le moteur de calcul : MapReduce



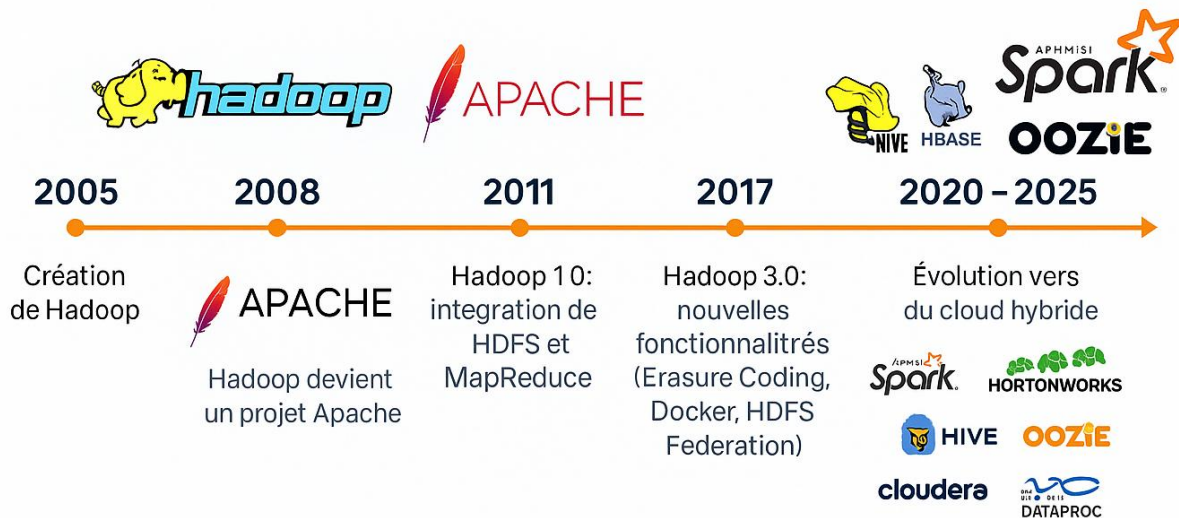
Le stockage distribué : HDFS



YARN : l'orchestrateur de ressources

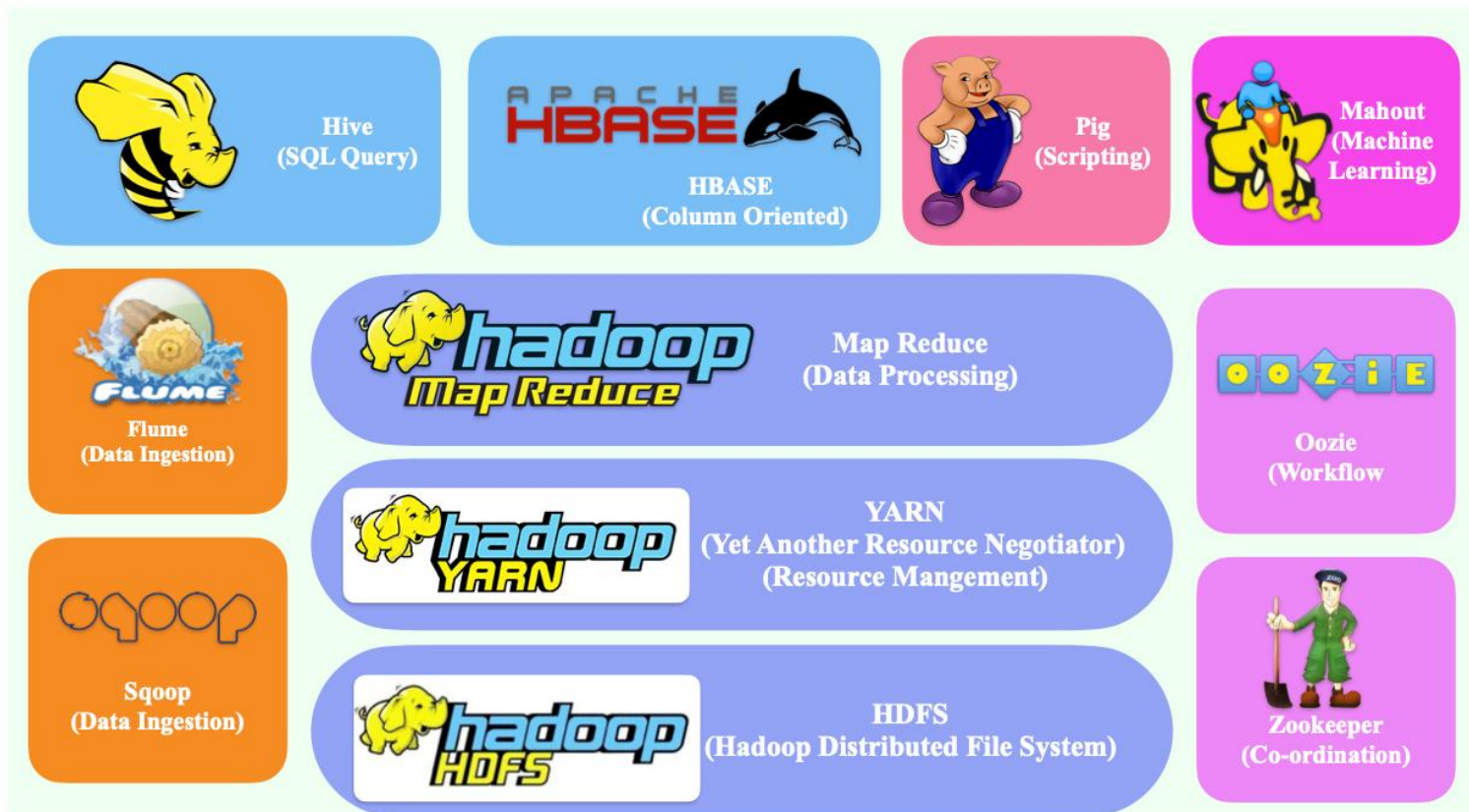
Hadoop : un écosystème complet

Hadoop – 20 ans d'évolution,
du cluster open source à l'écosystème cloud intelligent.



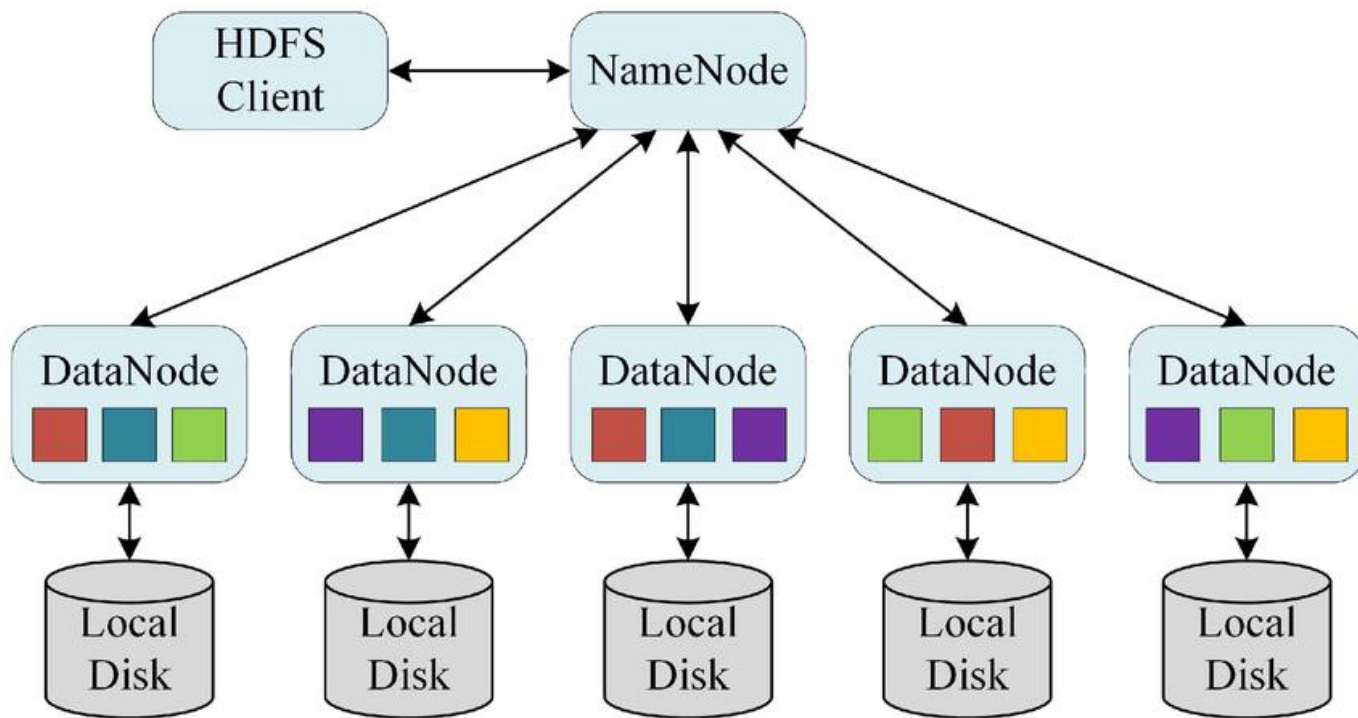
**Hadoop – 20 ans d'évolution, du cluster open source
à l'écosystème cloud intelligent.**

Hadoop : un écosystème complet



Le stockage distribué : HDFS

HDFS (Hadoop Distributed File System) divise les fichiers en blocs (128 Mo par défaut).



Le stockage distribué : HDFS

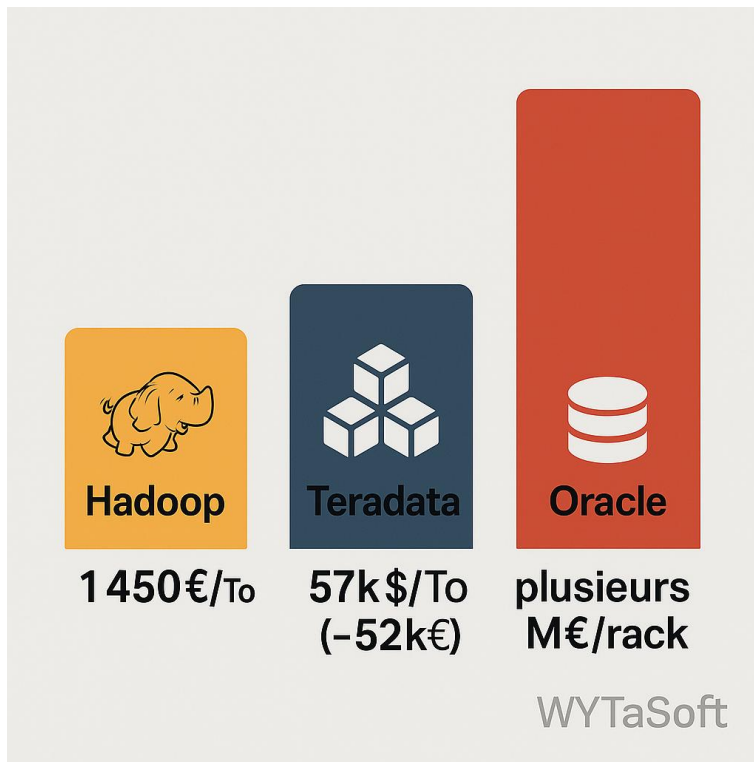
- **HDFS (Hadoop Distributed File System)** divise les fichiers en blocs (128 Mo par défaut).
 - Chaque bloc est **répliqué sur plusieurs serveurs (DataNodes)** pour assurer la tolérance aux pannes.
 - Le **NameNode** gère la table des fichiers et la répartition des blocs.
- ➔ **Avantage** : stockage massif et fiable sur des machines ordinaires.

Le stockage distribué : HDFS

- **HDFS (Hadoop Distributed File System)** divise les fichiers en blocs (128 Mo par défaut).
 - Chaque bloc est **répliqué sur plusieurs serveurs (DataNodes)** pour assurer la tolérance aux pannes.
 - Le **NameNode** gère la table des fichiers et la répartition des blocs.
- ➔ **Avantage** : stockage massif et fiable sur des machines ordinaires.

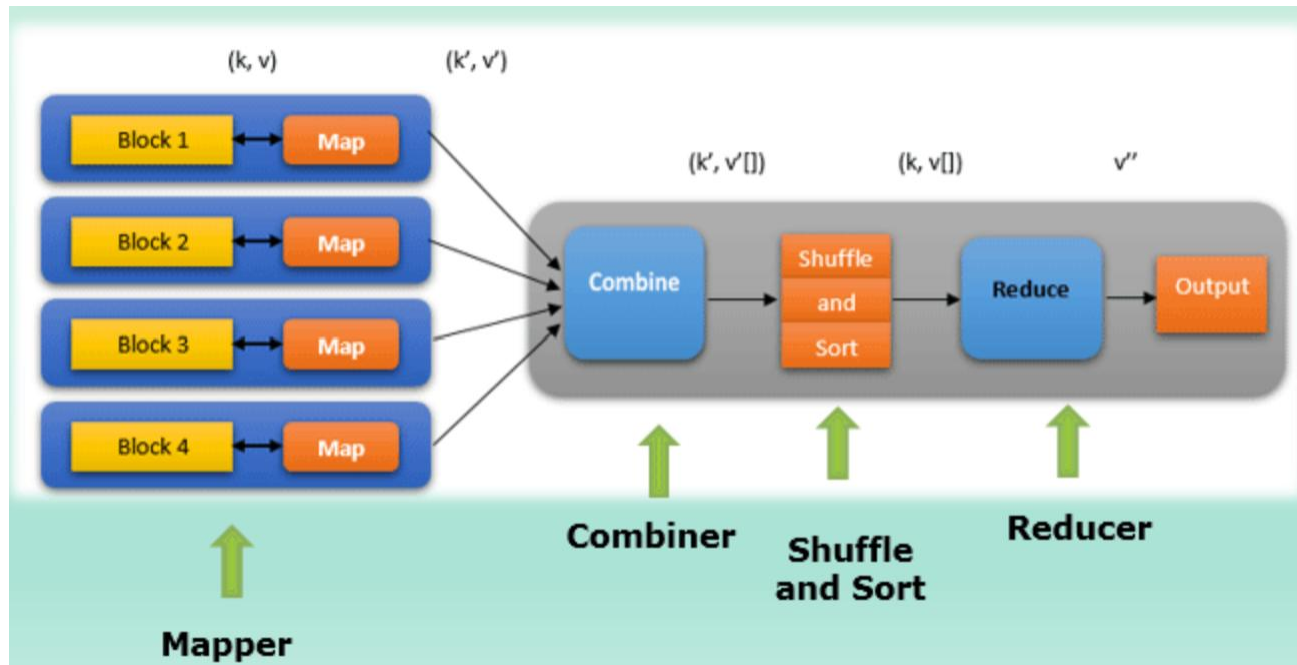
Le coût du stockage

HDFS (Hadoop Distributed File System) divise les fichiers en blocs (128 Mo par défaut).



Le moteur de calcul : MapReduce

L'ancêtre des traitements distribués modernes



Les outils de gestion et d'analyse

Les briques complémentaires de l'écosystème Hadoop

stockage, traitement, orchestration et coordination



Hive

Langage SQL
pour interroger
les données
dans Hadoop



Pig

Langage de
script pour la
transformation
de données



Oozie

Outil de
gestion et
d'orchestration
des workflows



HBase

Base NoSQL
distribuée et
temps réel

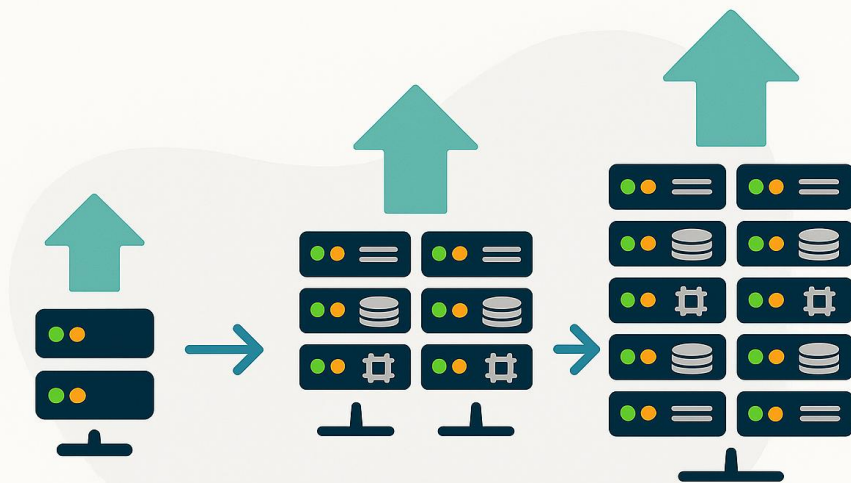


Zookeeper

Coordination
et synchronisation
du cluster

WyTaSoft

La scalabilité horizontale



La scalabilité horizontale
– le moteur du Big Data



Au lieu d'acheter un serveur plus puissant, on **ajoute simplement des machines** au cluster.

Chaque nouveau nœud augmente la **capacité de stockage et de calcul**.

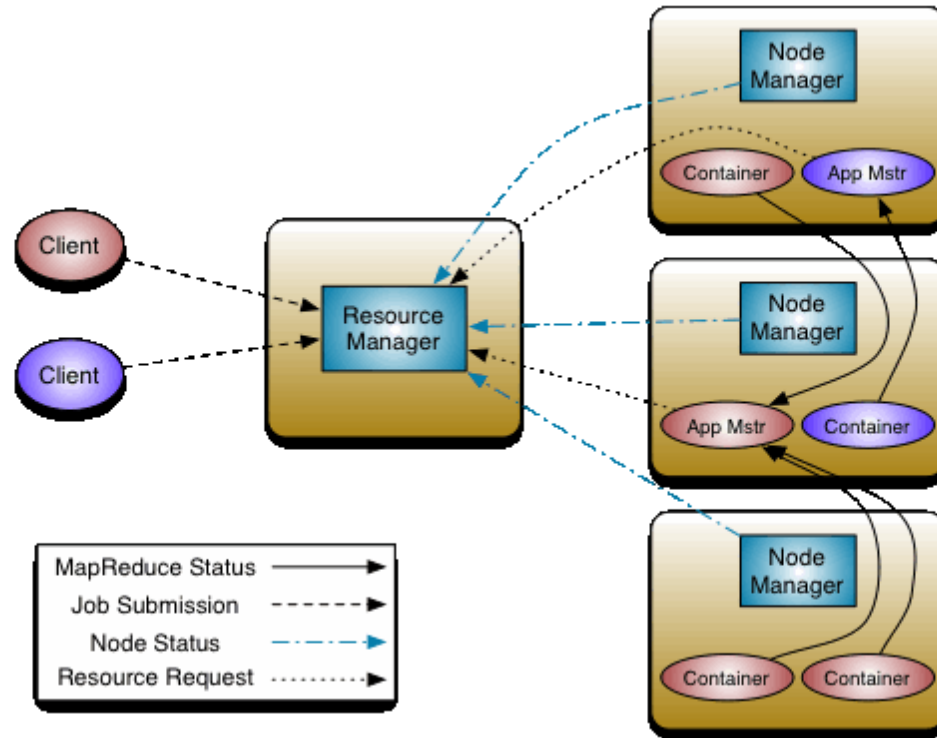
C'est le modèle économique du Big Data : **scalabilité horizontale**.

YARN : l'orchestrateur de ressources

- YARN (Yet Another Resource Negotiator) gère les ressources du cluster.
- Il répartit les tâches entre les serveurs et optimise l'utilisation du CPU et de la mémoire.
- Permet de faire tourner **plusieurs frameworks** (Spark, Flink, Tez...) sur le même cluster Hadoop.



Le Rôle YARN



YARN est le gestionnaire de ressources dans l'écosystème Hadoop

1. Gestion des ressources
2. Planification des tâches
3. Isolation et sécurité
4. Suivi et surveillance

Les Queues & les Applications YARN

Concept de Queues dans YARN : Les queues dans YARN sont des **structures logiques** utilisées pour organiser, isoler et gérer les ressources pour différentes applications ou groupes d'utilisateurs

Types de Planificateurs dans YARN :

- 1.FIFO Scheduler**
- 2.Capacity Scheduler**
- 3.Fair Scheduler**

Focus sur la Queue d'exécution de YARN

- **Qu'est-ce qu'une Queue YARN ?**
 - Système de gestion des ressources dans Hadoop pour organiser et prioriser les tâches.
 - Permet de distribuer les ressources entre différentes applications selon les priorités définies.
- **Types de Queues :**
 - **Queues hiérarchiques :** Organisation en arborescence pour séparer les ressources par département, équipe ou type d'application.
 - **Priorités des Queues :** Contrôle de la priorité d'exécution pour des tâches critiques vs. non critiques.
- **Importance de la Queue dans l'optimisation des ressources :**
 - Amélioration de la répartition de la charge de travail, évitant les conflits de ressources.
 - Assure une allocation équitable et optimale des ressources CPU et mémoire selon les besoins des applications.
- **Monitoring et Ajustement :**
 - Suivi en temps réel des files d'attente pour adapter les ressources si nécessaire.
 - Utilisation des logs et métriques pour ajuster dynamiquement les priorités et les quotas des queues.

Configuration des **Queues** pour la Priorisation

Définition des Capacités : Chaque queue se voit attribuer une capacité minimale (et maximale) en pourcentage des ressources totales du cluster.

- **Priorités au sein des Queues**
- **Politiques de Prémption**
- **Accès Contrôlé**

Vue d'ensemble des Applications

YARN

- ID de l'application, nom et utilisateur.
- Queue d'exécution.
- État de l'application : (RUNNING, FINISHED, etc.)
- Progression de l'exécution.
- Type d'application : SPARK, MAPREDUCE, etc.
- Temps de démarrage et de fin.
- Utilisation de la mémoire et du CPU.

État de l'Application dans YARN

- Les différents états d'une application :
 - **ACCEPTED** : Application en attente d'assignation de ressources.
 - **RUNNING** : Application en cours d'exécution.
 - **FINISHED** : Application terminée avec succès.
 - **FAILED** : Application ayant échoué durant l'exécution.
 - **KILLED** : Application arrêtée manuellement ou en raison d'un manque de ressources.

Logs d'Application YARN

- **Accès aux logs** : YARN offre un accès centralisé aux logs d'application, **Structure des logs** :
 - *Logs de l'ApplicationMaster*
 - *Logs des Exécuteurs (Executors)*
 - *Logs de l'erreur*
- **Récupération des logs via l'interface YARN**
- **Conservation et archivage des logs**

Récupération des Logs d'Application **YARN**

- Commandes de base pour accéder aux logs :



```
yarn logs -applicationId <application_id>
```

- Obtenir les logs pour un conteneur spécifique :



```
yarn logs -applicationId <application_id> -containerId <container_id>
```

Yarn Cheat Sheet

`yarn application -list`
List running applications

`yarn application -status <appid>`
Status of an application

`yarn queue -list`
List all queues

`yarn top`
Track Resource Usage in Real Time

`yarn application -kill <appid>`
Kill an application by ID

`yarn logs -applicationId <appid>`
Retrieve application logs

`yarn queue -status <queueName>`
Status of a queue

`yarn applicationattempt -list <Application_ID>`
Application Resource Summary

Fetch logs for the most recently submitted application
with the specified name

```
yarn logs -applicationId  
$(yarn application -list | grep  
<application_name> | sort -  
rk5 | head -n 1 | awk '{print  
$1}')
```

`yarn logs -user <username>`
Get Logs by User

`yarn container -list <appid>`
List containers of an application

`yarn container -status <containerid>`
Status of a container

`yarn scheduler`
Cluster resource allocation

`yarn application -kill <Application_ID>`

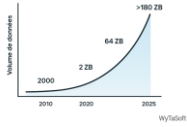
`yarn container -list <Application_ID>`
`yarn container -kill <Container_ID>`

```
yarn application -list | awk '{if($5 >  
86400000) print $1}' | xargs -l {}  
yarn application -kill {}  
Kill Applications Running Over One  
Day
```


Les modes de stockage (NoSQL, HDFS)



Les modes de stockage (NoSQL, HDFS)



NoSQL : Les modes de stockage (NoSQL, HDFS).



Types de NoSQL : Les bases orientées colonnes



Types de NoSQL : Les bases documentaires



Types de NoSQL : Les bases orientées Clé-Valeur



Types de NoSQL : Les bases orientées graphes

NoSQL : Les modes de stockage (NoSQL, HDFS).



Documentaires



MongoDB

Colonnes



cassandra



Clé-Valeur



redis

Graphes



neo4j



Types de NoSQL :

- **Documentaires** (MongoDB)
- **Colonnes** (Cassandra)
- **Clé-Valeur** (Redis)
- **Graphes** (Neo4j)

Adaptées à la **variété** et à la **vitesse** des données.

Modèles sans schéma → ajout facile de champs sans migration.

Types de **NoSQL** : Les bases documentaires



Souplesse du schéma :
on peut ajouter des champs
sans migration



Performances élevées pour
les lectures et écritures rapides



Scalabilité horizontale :
facile à répartir sur
plusieurs serveurs



Indexation puissante pour la
recherche de champs
spécifiques



Principe :

- Les données sont stockées sous forme de **documents JSON ou BSON**.
- Chaque document peut avoir sa propre structure, contrairement aux tables SQL rigides.

Structure typique :

```
{  
  "client_id": 123,  
  "nom": "Dupont",  
  "commandes": [  
    {"produit": "Laptop", "prix": 1299},  
    {"produit": "Souris", "prix": 39}  
  ]  
}
```

Types de **NoSQL** : Les bases orientées colonnes

Les données sont stockées **par colonne** et non par ligne.

Permet des **lectures ultra rapides** sur des ensembles massifs de données.

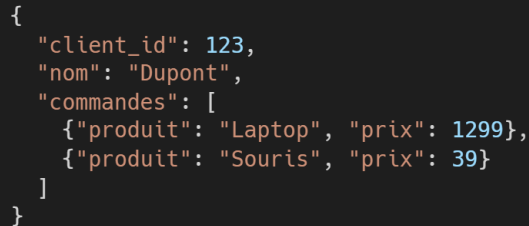
Structure adaptée à la **distribution horizontale** sur de nombreux nœuds.

Parfait pour les cas où la **vitesse d'accès** et la **scalabilité** sont essentielles.



ID Client	**Nom**	**Pays**	**Dernier Achat (€)**
001	Dupont	France	340
002	Silva	Espagne	590

→ Chaque colonne est stockée séparément sur des serveurs différents pour des accès ciblés rapides.



```
{
  "client_id": 123,
  "nom": "Dupont",
  "commandes": [
    {"produit": "Laptop", "prix": 1299},
    {"produit": "Souris", "prix": 39}
  ]
}
```

Types de NoSQL : Les bases orientées colonnes

Bases orientées colonnes



Écriture et lecture ultra-rapides
sur grands volumes



Scalabilité horizontale
automatique sur plusieurs
datacenters



Tolérance aux pannes
(réplication multi-nœuds)



Idéal pour les séries
temporelles et données
analytiques

Types de NoSQL : Les bases orientées Clé-Valeur



**Stockage en mémoire =
vitesse extrême**

Une clé, une valeur, un accès instantané.

WyTaSoft

Stockage sous forme de **paires clé → valeur**.

Accès direct à la donnée via une **clé unique**, sans schéma ni relations.

Fonctionne intégralement **en mémoire (RAM)** pour maximiser la rapidité.

Exemple :

```
clé : "user:123"  
valeur : {"nom": "Dupont", "solde": 230.75, "statut": "actif"}
```

Types de NoSQL : Les bases orientées Clé-Valeur



redis



Temps de réponse
ultra-rapide (millisecondes)



Stockage en mémoire vive
(RAM) → idéal pour le cache
et les sessions



Structure simple et efficace,
facile à intégrer
à toute application



Support de structures
avancées : listes, ensembles,
files d'attente

Haute disponibilité avec réplication
et persistance optionnelle sur disque

WyTaSoft

Types de NoSQL : Cas d'usage



Cache applicatif

(ex : sessions
utilisateur, tokens,
d'authentification)



Classements en temps réel

(jeux, e-commerce,
réseaux sociaux)

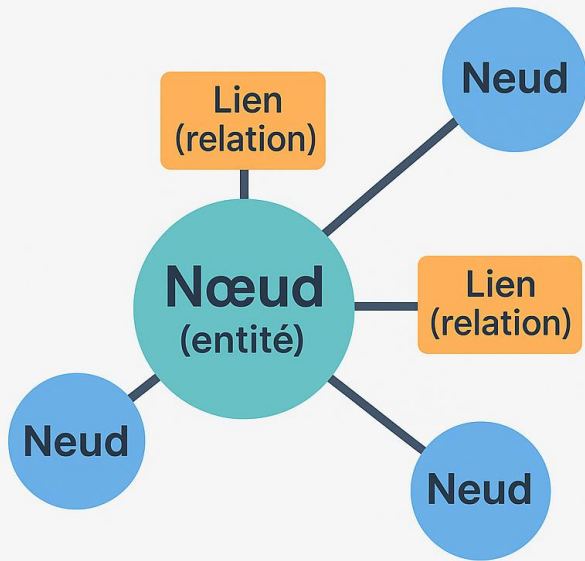


Files de messages / systèmes de notification



Comptage rapide/ limitations de taux (rate limiting)

Types de NoSQL : Les bases orientées graphes



Les données sont stockées sous forme de nœuds (entités) et de liens (relations).

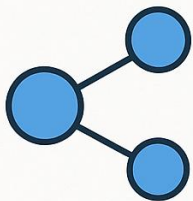
Cela permet de naviguer rapidement dans des réseaux complexes.

- Les données sont stockées sous forme de **nœuds (entités)** et de **liens (relations)**.
- Chaque relation est **directement enregistrée**, sans jointures comme dans SQL.
- Cela permet de **naviguer rapidement** dans des réseaux complexes.
- Idéal pour détecter **des connexions**, **des influences** ou **des anomalies**.

```
(Alice) -[amis_avec]-> (Bob)
      |
      └-[travaille_chez]-> (IBM)
```

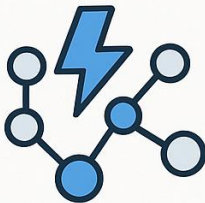
Types de NoSQL : Les bases orientées graphes

AVANTAGES DES BASES ORIENTÉES GRAPHES



RELATIONS DIRECTES

Pas de jointures complexes, navigation instantanée



REQUÊTES RAPIDES

Sur de grands graphes (ex : requêtes de voisinage)



ANALYSE DE RÉSEAUX

Social, logistique, frauduleux, etc.



VISUALISATION INTUITIVE

Des relations entre entités

Types de NoSQL : Cas d'usage

Cas d'usage



Détection de fraude

Corrélation entre
comptes, transactions
IP



Réseaux sociaux

Connexions,
recommandations
d'amis



Chaîne logistique

Relations
fournisseurs →
transporteurs →
clients

Les technologies du Big Data

Architectures de calcul distribuées



Architectures de calcul distribuées :

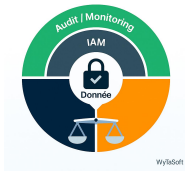
MapReduce / Spark / Storm



Objectifs des moteurs



Apache Spark : la promesse



Acteurs & positionnement



Apache Spark & BigData

Objectifs des moteurs

Objectifs



Latence cible

Batch ($\geq$ minutes),
Micro-batch (s)
Temps réel (ms).



Échelle

de $T_o \rightarrow P_o$,
 $k \rightarrow M$ events/s



SLA

disponibilité $\geq 99,9\%$,
reprise automatique
sur panne



Coût

optimiser compute +
storage (scaling
horizontal, spot, cache)

Latence cible — La latence dépend du besoin métier : *reporting* (minutes), *personnalisation/BI rapide* (secondes), *fraude/alerting* (millisecondes).

Batch ($\geq$ minutes) — On privilégie la robustesse et le coût : gros volumes, fenêtres temporelles, tolérance au retard.

Micro-batch (s) — Compromis réactivité/simplicité : mêmes API que le batch, petites fenêtres (1–30 s).

Temps réel (ms) — Traitement événement-par-événement, back-pressure, état externalisé pour tenir la latence.

Échelle ($T_o \rightarrow P_o$, $k \rightarrow M$ events/s) — On dimensionne le **parallelisme** et les **partitions** ; on évite les **skews** et les **small files**.

Débit élevé — Préférer formats colonnes (Parquet/Delta), **compression** et **data skipping** pour limiter I/O.

SLA ($\geq 99,9\%$) — Redondance zones, **auto-restart**, **checkpointing/replay**, tests de panne (chaos).

Objectifs de reprise — Définir **RPO/RTO** ; garantir **idempotence** et *exactly/at-least-once* selon le cas.

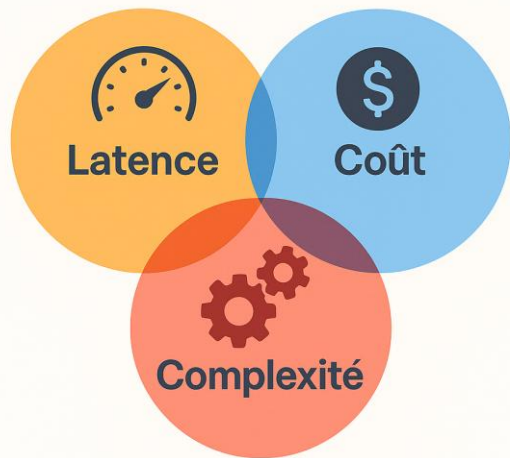
Coût — Séparer **compute/storage**, éteindre les clusters à vide, **autoscaling**.

Optimisation — **Instances spot/préemptibles** (avec tolérance à l'éviction), **cache** (broadcast joins), **compaction** régulière.

Gouvernance/ops — Observabilité par **métriques, logs, traces** ; SLO de latence surveillés en continu.

Contraintes à arbitrer

Contraintes à arbitrer



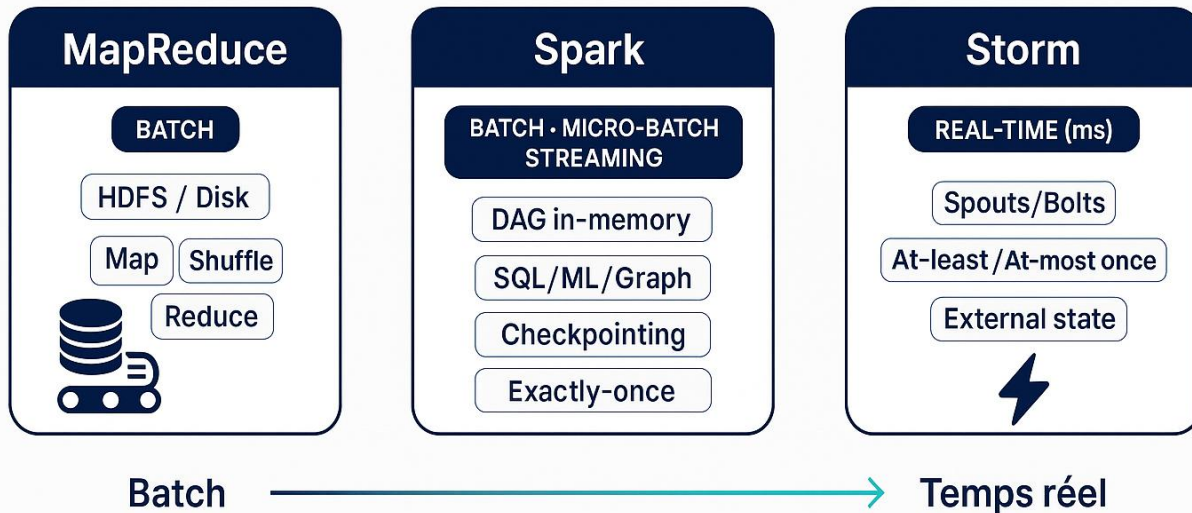
- **Latence vs Coût vs Complexité** (ops & dev).
- **Tolérance aux pannes** : replays, exactly/at-least-once.
- **Simplicité d'opération** : monitoring, autoscaling, déploiement.

- Tolérance aux pannes : replays, exactly/at-least-once
- Simplicité d'opération : monitoring, autoscaling, déploiement
- Ideal puortes structures avancades

Acteurs & positionnement

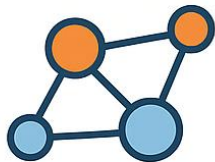
Acteurs & positionnement

Même données, trois tempos.



Apache Spark : la promesse

Un moteur distribué in-memory avec API unifiée pour Batch, Streaming, SQL, ML, Graph



DAG optimisé

Transformations lazy → exécution optimisée



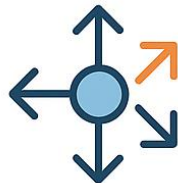
API unifiée

Même API pour batch & streaming (DataFrame)



Écosystème complet

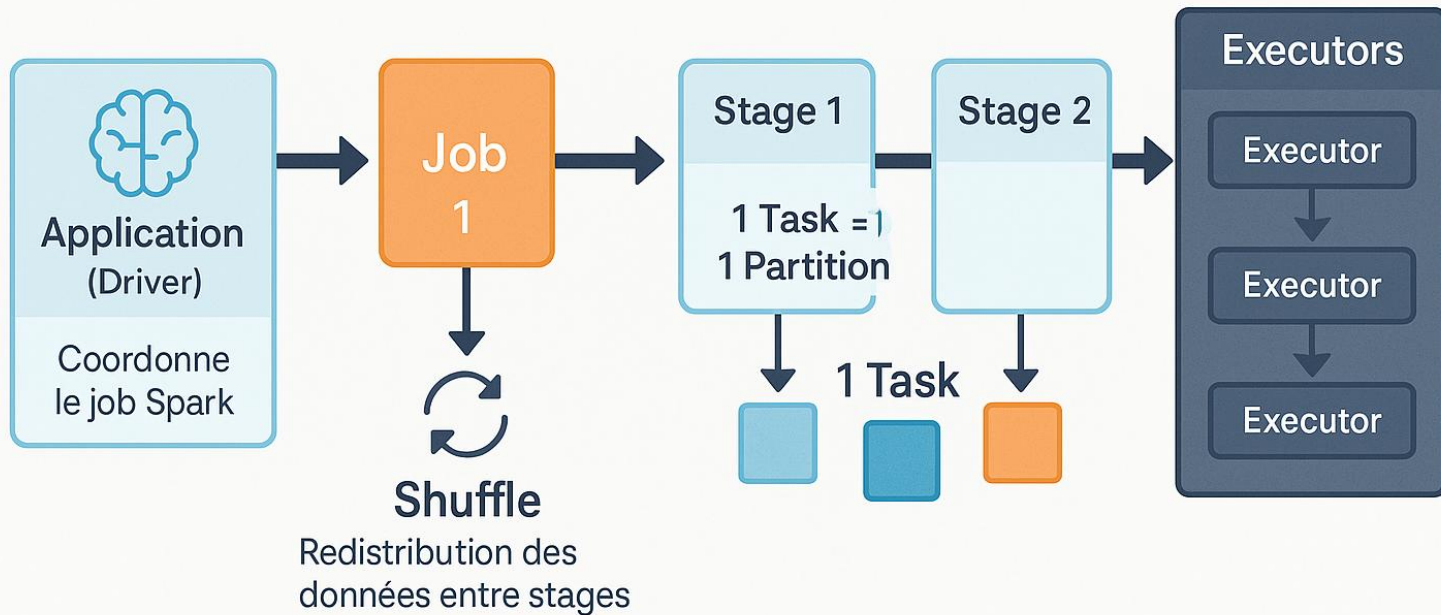
SQL, MLlib / GraphX
Structured Streaming



Scalabilité

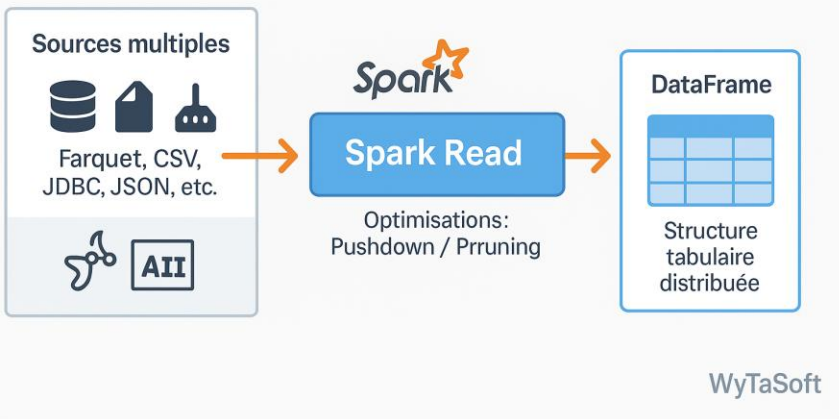
Partitions & autoscaling

Exécution d'un Job Spark



Extraction des données (**Read** avec Spark)

Lecture de données avec Spark



Connecteurs natifs : fichiers (Parquet/ORC/CSV/JSON), Delta, Kafka, DB (JDBC), cloud (S3/GCS/ADLS).

Predicate pushdown & project pushdown : moins d'I/O, colonnes/rows filtrées à la source.

Partition pruning : lit uniquement les partitions utiles (ex. dt=2025-11-07).

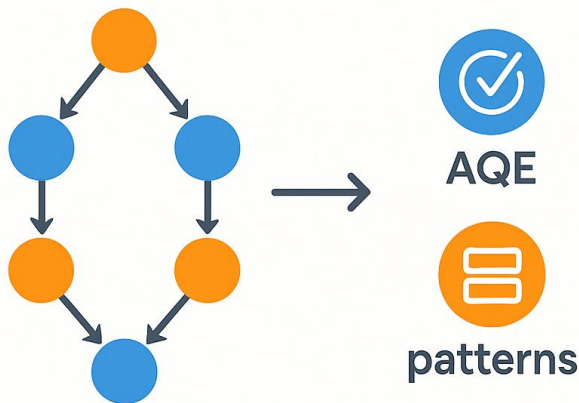
Formats colonne (Parquet/ORC/Delta) : **compression, encodage, data skipping**.

Parallelisme automatique : lecture par **splits**/fichiers en parallèle.

Schéma : **inférence + évolution** (schema evolution) selon formats.

Transformation des données (Compute avec Spark)

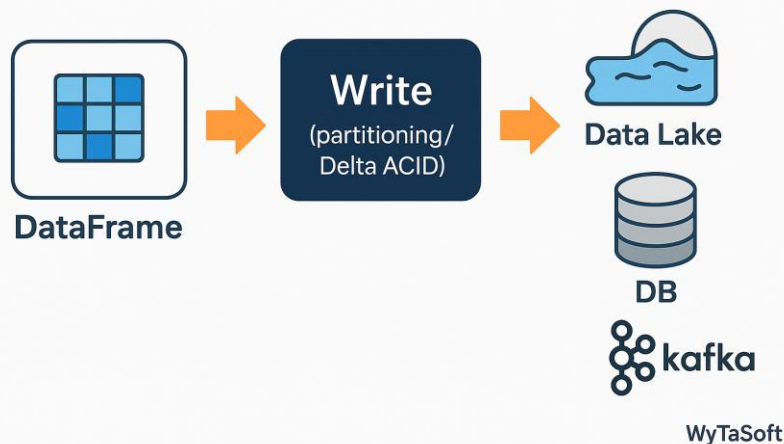
Les transformations sont paresseuses et optimisées



Avec AQE et les bons patterns (broadcast, partitions), on gagne en latence et stabilité.

- **API DataFrame/SQL** : déclaratif, lisible, optimisable.
- **Catalyst** (plan logique → physique) & **Tungsten** (mémoire/CPU).
- **Joins efficaces** : **broadcast**, sort-merge, shuffle-hash (selon taille/cardinalité).
- **Fenêtrage & agrégations** : window, groupBy, rollup, cube.
- **Adaptive Query Execution (AQE)** : ajuste partitions et joins à l'exécution.
- **Cache/Persist ciblé** : accélère itératif (ML/feature eng).
- **UDF vs fonctions natives** : privilégier fonctions Spark pour vectorisation/optis.

Load des données (Write avec Spark)



- **Sinks** : Parquet/ORC/Delta, JDBC (DW/DB), topics (Kafka), objets (S3/GCS/ADLS).

Partitionnement/Bucketing : meilleure lecture future, jointures plus rapides.

Delta Lake : ACID, time travel, schema evolution, MERGE/UPSERT.

Optimisations : compaction (OPTIMIZE), Z-Order/data skipping (selon techno).

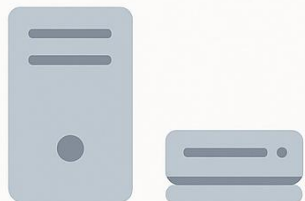
Idempotence & exactly-once (Structured Streaming) avec checkpointing + sinks compatibles.

Gestion des petits fichiers : coalesce/repartition avant write, jobs de compactage.

- **Contrôles qualité** : expectations (tests), stats, lineage/catalogue.

Changement d'échelle : des Go aux To (et plus) avec Spark

Avant



Serveur unique
I/O disque
heures/jours pour >100 Go



Maintenant



Cluster Spark
N executors, P partitions
To en minutes/heures

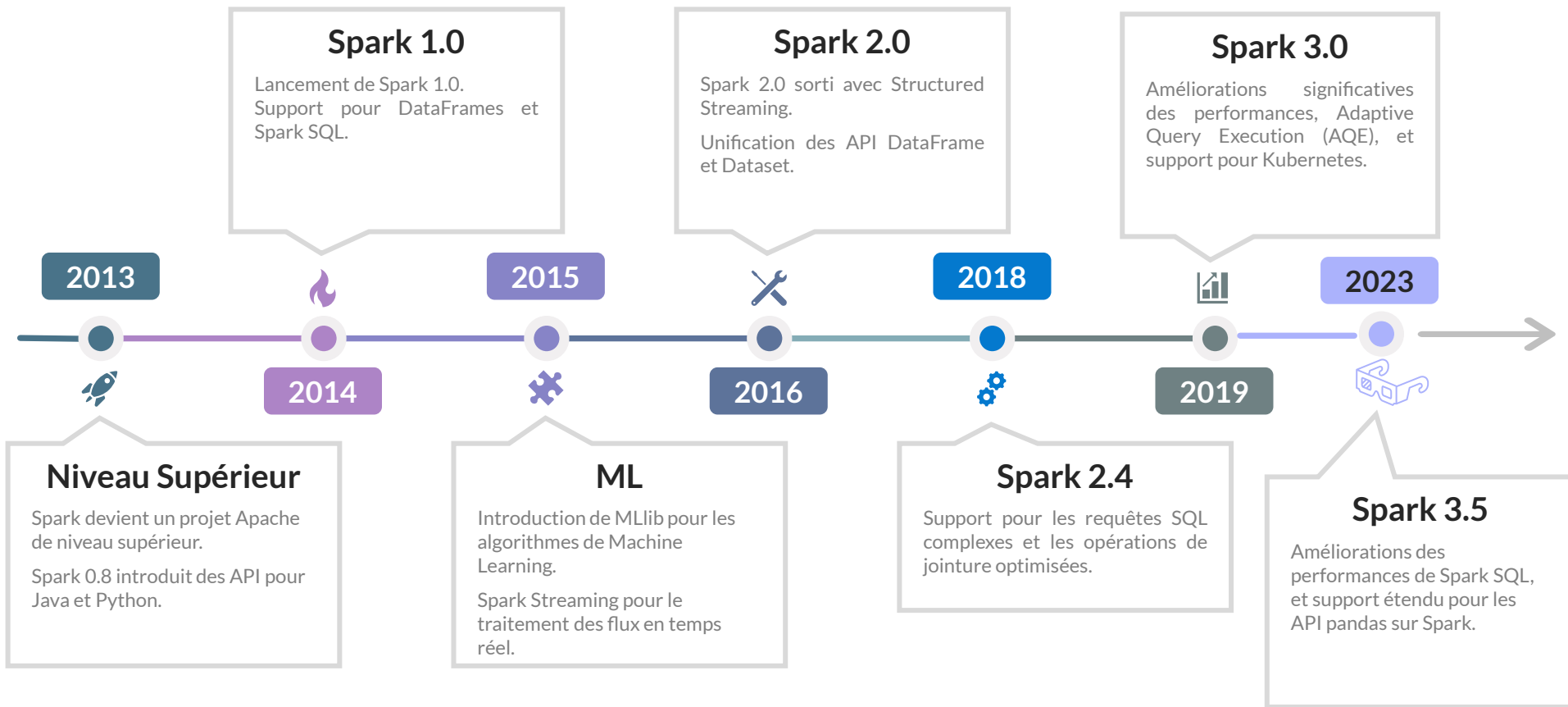


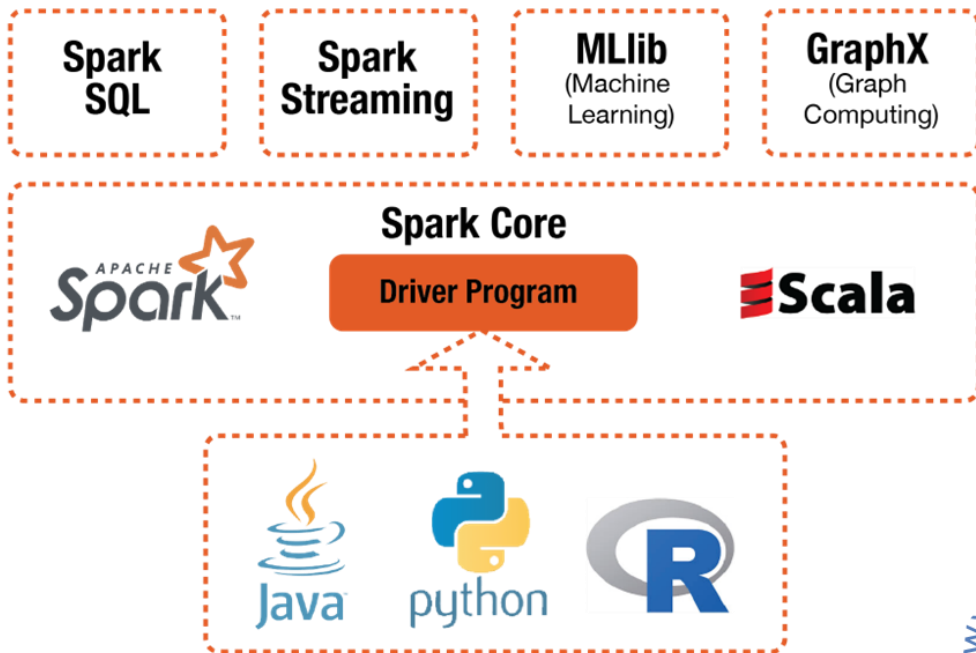
1 To lu/transformaté en < 15–30 min sur un cluster ~50–80 vCPU, si I/O soutenu.

10–50 To en quelques heures avec autoscaling et partitions bien réglées.

Débit agrégé typique > 5–20 Go/s sur clusters moyens (stockage & réseau aidants).

Historique de Apache Spark





Spark **Eco-System**

WYTA
SOFT

Responsabilités Clés de Spark Core



Gestion de la Mémoire



Scheduling



Récupération en Cas de Panne
(Fault Tolerance)



Distribution et
Surveillance des Jobs



Interaction avec les
Systèmes de Stockage

Composants Clés de Spark SQL



DataFrames



Datasets



Catalyst Optimizer



SparkSession

Composants Clés de Spark MLlib



Algorithmes de Machine Learning



Évaluation et Validation des
Modèles



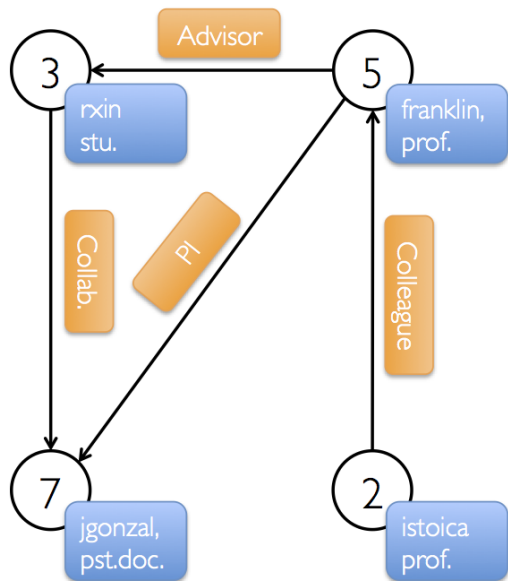
Pipelines de Machine Learning

Composants Clés de Spark Streaming



Composants Clés de Spark GraphX

Property Graph



Vertex Table

Id	Property (V)
3	(rxin, student)
7	(jgonzal, postdoc)
5	(franklin, professor)
2	(istoica, professor)

Edge Table

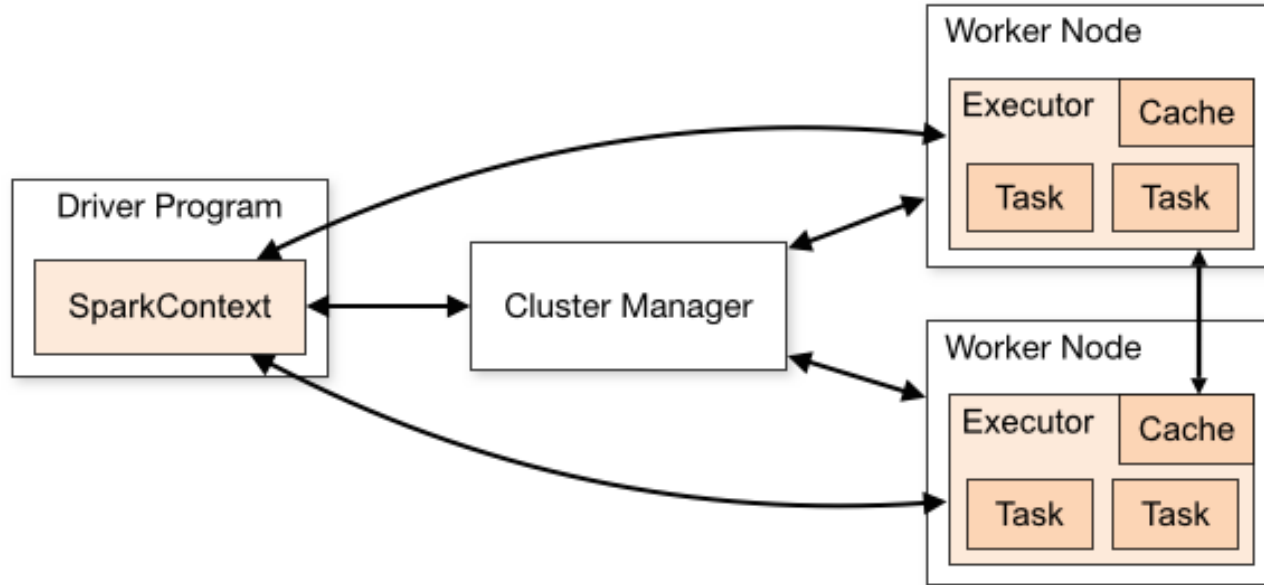
SrcId	DstId	Property (E)
3	7	Collaborator
5	3	Advisor
2	5	Colleague
5	7	PI

Graphes Dirigés et Non Dirigés

Property Graph

Triplets

Vue d'Ensemble de l'Architecture Apache Spark



Rôle du Driver

Le Rôle du Driver

Définition : Le Driver est le processus central qui exécute le programme principal de votre application Spark.

Fonctions :

- Il transforme le programme Spark en tâches.
- Il maintient des informations sur le contexte Spark et les sessions utilisateur.

Driver & SparkUI



3.4.0-amzn-0

Jobs

Stages

Storage

Environment

Executors

SQL / DataFrame

Spark shell application UI

Executors

[Show Additional Metrics](#)

Summary

	RDD Blocks	Storage Memory	On Heap Storage Memory	Off Heap Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Excluded
Active(7)	2556	156.6 GiB / 169.8 GiB	156.6 GiB / 169.8 GiB	0.0 B / 0.0 B	459.9 GiB	96	0	0	3104	3104	11.4 h (46 min)	127.6 GiB	62.4 KiB	62.4 KiB	0
Dead(0)	0	0.0 B / 0.0 B	0.0 B / 0.0 B	0.0 B / 0.0 B	0.0 B	0	0	0	0	0	0.0 ms (0.0 ms)	0.0 B	0.0 B	0.0 B	0
Total(7)	2556	156.6 GiB / 169.8 GiB	156.6 GiB / 169.8 GiB	0.0 B / 0.0 B	459.9 GiB	96	0	0	3104	3104	11.4 h (46 min)	127.6 GiB	62.4 KiB	62.4 KiB	0

Executors

Show entries

Search:

Executor ID	Address	Status	RDD Blocks	Storage Memory	On Heap Storage Memory	Off Heap Storage Memory	Peak JVM Memory OnHeap / OffHeap	Peak Execution Memory OnHeap / OffHeap	Peak Storage Memory OnHeap / OffHeap	Peak Pool Memory Direct / Mapped	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Logs	Thread Dump
driver	ip-172-31-3-68.eu-west-1.compute.internal:41281	Active	0	66.9 KiB / 10.5 GiB	66.9 KiB / 10.5 GiB	0.0 B / 0.0 B	920.2 MiB / 274.8 MiB	0.0 B / 0.0 B	1012.2 KiB / 0.0 B	6.5 MiB / 0.0 B	0.0 B	0	0	0	0	0	18 min (0.8 s)	0.0 B	0.0 B	0.0 B		Thread Dump
1	ip-172-31-3-170.eu-west-1.compute.internal:46697	Active	473	25.8 GiB / 26.6 GiB	25.8 GiB / 26.6 GiB	0.0 B / 0.0 B	43.8 GiB / 142.3 MiB	0.0 B / 0.0 B	26.6 GiB / 0.0 B	2.6 GiB / 1.7 GiB	91.4 GiB	16	0	0	679	679	2.1 h (7.6 min)	22.5 GiB	0.0 B	10.6 KiB	stdout stderr	Thread Dump
2	ip-172-31-3-222.eu-west-1.compute.internal:44907	Active	466	26.2 GiB / 26.6 GiB	26.2 GiB / 26.6 GiB	0.0 B / 0.0 B	40.6 GiB / 141.5 MiB	0.0 B / 0.0 B	26.6 GiB / 0.0 B	2.3 GiB / 1.8 GiB	89.1 GiB	16	0	0	701	701	2.1 h (7.3 min)	23.9 GiB	0.0 B	11.4 KiB	stdout stderr	Thread Dump
3	ip-172-31-3-196.eu-west-1.compute.internal:39167	Active	473	26.2 GiB / 26.6 GiB	26.2 GiB / 26.6 GiB	0.0 B / 0.0 B	41.6 GiB / 141.5 MiB	0.0 B / 0.0 B	26.5 GiB / 0.0 B	2.2 GiB / 1.1 GiB	91.4 GiB	16	0	0	680	680	2.1 h (8.0 min)	22.7 GiB	0.0 B	10.9 KiB	stdout stderr	Thread Dump
4	ip-172-31-3-36.eu-west-1.compute.internal:46023	Active	482	26.1 GiB / 26.6 GiB	26.1 GiB / 26.6 GiB	0.0 B / 0.0 B	42.6 GiB / 144.3 MiB	0.0 B / 0.0 B	26.6 GiB / 0.0 B	2.2 GiB / 1.7 GiB	94.3 GiB	16	0	0	674	674	2.1 h (9.2 min)	23.9 GiB	62.4 KiB	11.4 KiB	stdout stderr	Thread Dump
5	ip-172-31-3-244.eu-west-1.compute.internal:43799	Active	317	26.3 GiB / 26.6 GiB	26.3 GiB / 26.6 GiB	0.0 B / 0.0 B	41.8 GiB / 131.7 MiB	0.0 B / 0.0 B	26.6 GiB / 0.0 B	1.7 GiB / 754 MiB	44.3 GiB	16	0	0	178	178	1.4 h (7.7 min)	16.8 GiB	0.0 B	8.7 KiB	stdout stderr	Thread Dump
6	ip-172-31-3-39.eu-west-1.compute.internal:43111	Active	345	26 GiB / 26.6 GiB	26 GiB / 26.6 GiB	0.0 B / 0.0 B	43.6 GiB / 130.3 MiB	0.0 B / 0.0 B	26.5 GiB / 0.0 B	1.9 GiB / 1.1 GiB	49.5 GiB	16	0	0	192	192	1.4 h (6.2 min)	17.8 GiB	0.0 B	9.4 KiB	stdout stderr	Thread Dump

Showing 1 to 7 of 7 entries

Previous **1** Next

Rôle du Executors

Les Executors

Définition : Les Executors sont des processus distribués responsables de l'exécution des tâches assignées par le Driver.

Fonctions :

- Exécutent les calculs et stockent les données pour les applications Spark.
- Chaque Executor est un processus JVM distinct.

Show Additional Metrics

- ☐ Select All
- ☐ On Heap Memory
- ☐ Off Heap Memory

Summary

	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Blacklisted
Active(3)	0	5.9 KiB / 1.1 GiB	0.0 B	2	0	0	5	5	4 s (0.2 s)	0.0 B	0.0 B	0.0 B	0
Total(3)	0	5.9 KiB / 1.1 GiB	0.0 B	2	0	0	5	5	4 s (0.2 s)	0.0 B	0.0 B	0.0 B	0
Dead(0)	0	0.0 B / 0.0 B	0.0 B	0	0	0	0	0	0.0 ms (0.0 ms)	0.0 B	0.0 B	0.0 B	0

Executors

Show20entries

Search:

Executor ID	Address	Status	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Logs	Thread Dump
1	10.12.221.27:55834	Active	0	2 KiB / 366.3 MiB	0.0 B	1	0	0	3	3	2 s (0.1 s)	0.0 B	0.0 B	0.0 B	stdout stderr	Thread Dump
0	10.12.221.27:55835	Active	0	2 KiB / 366.3 MiB	0.0 B	1	0	0	2	2	2 s (94.0 ms)	0.0 B	0.0 B	0.0 B	stdout stderr	Thread Dump
driver	10.12.221.27:55827	Active	0	2 KiB / 366.3 MiB	0.0 B	0	0	0	0	0	0.0 ms (0.0 ms)	0.0 B	0.0 B	0.0 B		Thread Dump

Rôle du DAG (Directed Acyclic Graph)

Le DAG (Directed Acyclic Graph)

Définition : Un « DAG » est un graphe des opérations à effectuer, où les nœuds représentent les opérations, et les arêtes représentent les dépendances entre ces opérations.

Catalyst Optimizer

Fonctionnement :

- Spark crée un « DAG » pour chaque job, permettant une optimisation et une planification efficace des « Tasks ».
- Permet de récupérer partiellement en cas de défaillance.

Details for Job 0

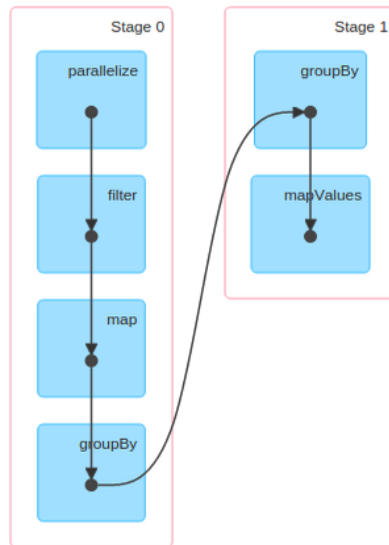
Status: RUNNING

Active Stages: 1

Pending Stages: 1

▶ Event Timeline

▼ DAG Visualization



Stage

Active Stages (1)

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read
0	groupBy at DAGTest.java:53 (kill) +details	2016/08/24 12:45:38	Unknown	0/2			

DAG
&
SparkUI

Actions

- Les **actions** en Apache Spark déclenchent l'exécution des transformations précédentes et renvoient un résultat au driver.
 - **count()** : Compte le nombre d'éléments dans le DataFrame/RDD.
 - **show()** : Affiche les premières lignes du DataFrame.
 - **saveAsTextFile(path)** : Sauvegarde les données du RDD en tant que fichier texte à l'emplacement spécifié.
 - **write** : Écrit les données du DataFrame dans un format spécifié (parquet, csv, json, etc.) à un emplacement donné.

Transformation

Toute opération qui **transforme** un DataFrame/RDD en un autre. Étant donné que les objets DataFrame/RDD sont immuables, ces opérations aboutissent à un **nouveau** DataFrame.

Exemples de Transformations

1. select(cols)

- **Description** : Sélectionne un ensemble de colonnes dans un DataFrame.
- **Exemple** :

```
val selectedDf = df.select("name", "age")
```

2. join(df2, joinExprs)

- **Description** : Joint deux DataFrames sur des colonnes spécifiées.
- **Exemple** :

```
val joinedDf = df1.join(df2, df1("id") === df2("id"))
```

Types Transformations

Il faut savoir qu'ils existent deux types de transformations :

- ***Narrow Transformations (étroites)***
- ***Wide Transformations (Larges)***

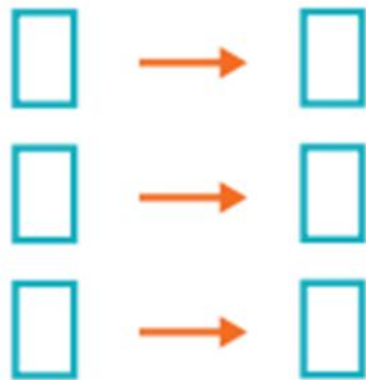
Narrow Transformations

Sont des transformations où chaque **partition** des données de sortie dépend d'une **seule** partition des données d'entrée.

map(func) : Applique une fonction à chaque élément du RDD/DataFrame.

```
val rdd1 = sc.parallelize(Seq(1, 2, 3, 4))  
val rdd2 = rdd1.map(x => x * 2)
```

Narrow Transformations
1 to 1



Wide Transformations

sont des transformations où chaque **partition** des données de sortie peut dépendre de **plusieurs** partitions des données d'entrée.

agg(agggregationExprs) : Applique des fonctions d'agrégation à un DataFrame, ce qui peut impliquer un shuffle pour regrouper les données en fonction des clés d'agrégation.

```
val aggregatedDf = df.groupBy("category").agg(sum("value"))
```

Wide Transformations (shuffles)
1 to N



PySpark Demo



Banking Stress Test with PySpark (Read → Transform → Write)

- Read CSVs into Spark DataFrames
- Create stressed risk metrics (PD/LGD/EL) per scenario
- Aggregate by **segment** and **scenario**
- Write results to **Parquet** and **CSV**

Banking Stress Test with PySpark (Read → Transform → Write)

Datasets

- `exposures.csv` — loan-level portfolio
- `scenarios.csv` — stress multipliers

Banking Stress Test with PySpark (Read → Transform → Write)

Setup (2–3 min)

- Open VS Code → new Jupyter notebook.
- Start/verify Spark:

Banking Stress Test with PySpark (Read → Transform → Write)

Part A – Read

- **Task A1.** Read both CSVs (with headers & inferred schema). Update the base path if you downloaded elsewhere.
- **Task A2.** Inspect a few rows:
- **Checkpoint:** Do you see columns ead, pd, lgd as numeric? If any came in as string, cast them.

Banking Stress Test with PySpark (Read → Transform → Write)

Part B — Transform

We'll compute stressed metrics **for every scenario** for each loan.

Task B1. Join exposures with every scenario (cross join):

Task B2. Create stressed metrics and **expected loss (EL)**:

```
stressed_pd = pd * pd_multiplier
```

```
stressed_lgd = min(1.0, lgd + lgd_addon)
```

```
stressed_rate = interest_rate + rate_shock_bps/10000
```

```
expected_loss = ead * stressed_pd * stressed_lgd
```

Task B3. Preview: **Checkpoint:** Are stressed_lgd values ≤ 1.0 ? (Ceiling applied.)

Banking Stress Test with PySpark (Read → Transform → Write)

Part C — Aggregate

- Task C1. Aggregate by (scenario, segment):
- **Checkpoint:** Do you see one row per scenario/segment with sensible EL ratios?

Banking Stress Test with PySpark (Read → Transform → Write)

Part D – Write

- Task D1. Write **loan-level** results to **Parquet**.
- Task D2. Write **aggregated** results to **CSV**.
- **Checkpoint:** Re-read your outputs to verify:

Banking Stress Test with PySpark (Read → Transform → Write)

Compute **stressed Expected Loss (EL)** by joining a loan portfolio with stress scenarios and aggregating results by **segment** and **scenario**.

Inputs

- exposures.csv — one row per loan: loan_id, segment, country, ead, pd, lgd, interest_rate
- scenarios.csv — stress parameters per scenario: scenario, pd_multiplier, lgd_addon, rate_shock_bps

Outputs

- **Loan-level stressed metrics** (Parquet)
- **Aggregated EL by segment & scenario** (CSV or Parquet)

Banking Stress Test with PySpark (Read → Transform → Write)

Formulas (per loan, per scenario)

- $\text{stressed_pd} = \text{pd} * \text{pd_multiplier}$
- $\text{stressed_lgd} = \min(1.0, \text{lgd} + \text{lgd_addon})$
- $\text{stressed_rate} = \text{interest_rate} + \text{rate_shock_bps} / 10000$ (*optional, for P&L/use later*)
- $\text{expected_loss} = \text{ead} * \text{stressed_pd} * \text{stressed_lgd}$

We'll then **aggregate** by (scenario, segment):

- $\text{total_ead}, \text{total_expected_loss}, \text{and EL_over_EAD} = \text{total_expected_loss} / \text{total_ead}.$

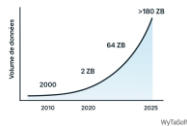
Les technologies du Big Data

Les Plateformes Big Data



Les Plateformes Big Data

ToDo



Distributions & managés



Guide de choix (cloud-native)



Plateformes analytiques & streaming



Types de NoSQL : Les bases orientées Clé-Valeur



Types de NoSQL : Les bases orientées graphes

Distributions & managés (Hadoop/Spark)

CLUDERA
CDP

Hybride · SDX



Hewlett | Packard
Enterprise

Data Fabric · multi-protocoles

HPEEzmeral

HPE Ezmeral
Data Fabric



Hadoop/Spark managé
autoscaling



Google
Cloud

Dataproc

Hadoop/Spark managé



Cloud Pak for Data

IA · intégration

Plateformes analytiques & streaming

Plateformes analytiques & streaming



Databricks

(multi-cloud, incl. Azure Databricks)

Lakehouse unifié
Delta / MLflow / Unity Catalog



Azure Synapse

(Spark)
SQL + Spark intégrés
Pipelines



GCP Dataflow

(Beam)
Streaming / Batch



GCP Dataproc

Serverless (Spark)
Serverless
Jobs-as-Service



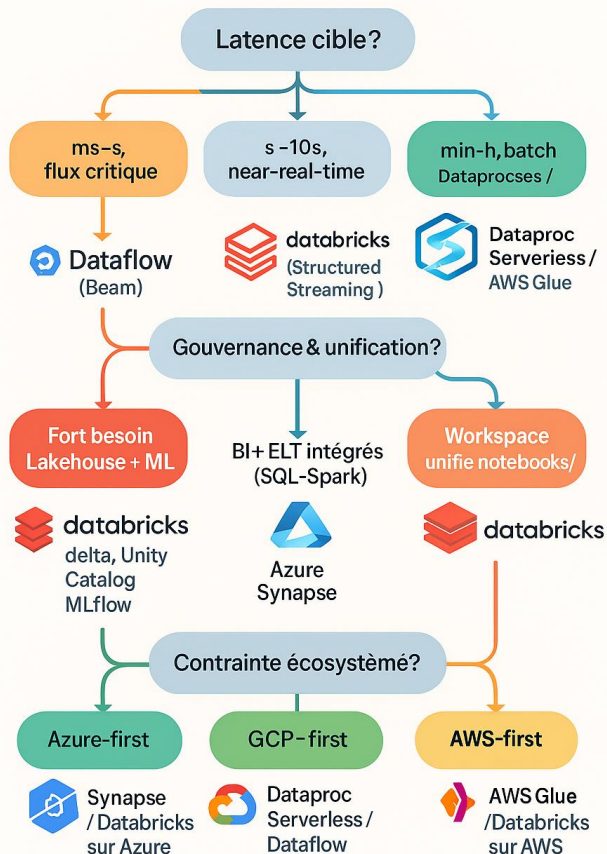
AWS Glue / Spark

Serverless ETL
Intégration AWS

WyTaSoft

- **Databricks** → *ELT/BI/ML* sur **Delta Lake**, gouvernance **Unity Catalog**, **DLT** pour l'orchestration de tables.
- **Synapse (Spark)** → **SQL Serverless + Spark** dans un studio unique, **Pipelines** natifs ; bon pour *BI+ELT* intégrés.
- **Dataproc Serverless** → **Spark sans cluster** pour batch éphémère, facturation au job.
- **Dataflow (Beam)** → **streaming** critique (watermarks, fenêtres), **exactly-once** possible, **autoscaling** avancé.
- **AWS Glue/Spark** → **ETL serverless** avec catalogage, crawlers et intégration **S3/Lake Formation**.

Choisir sa plateforme analytique & streaming



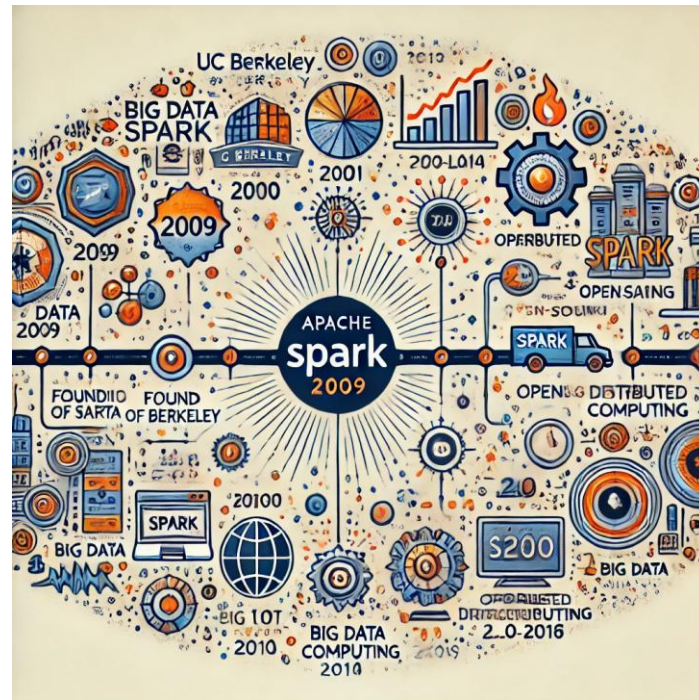
WyTaSoft

Guide de choix (cloud-native)

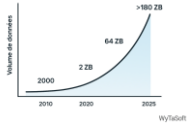
- **Perf** : Delta/Parquet, partitionnement/compaction, broadcast joins.
- **Coût** : autoscaling, jobs éphémères, surveiller shuffle/state/durée.
- **Gouvernance** : catalogue/lignage, RBAC/masking, quality checks.

Les technologies du Big Data

Les technologies du Data Scientist



Les modes de stockage (NoSQL, HDFS)



Panorama & cycle de vie



Préparation (cycle ML)



Collecte (cycle ML)



Modélisation (cycle ML)

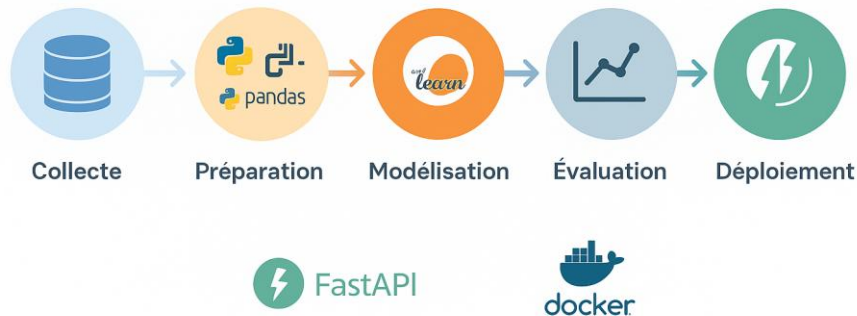


Types de NoSQL : Les bases orientées graphes

Panorama & cycle de vie

Panorama & cycle de vie

Du data prep au déploiement, un outillage cohérent.



WyTaSoft

1. Collecte → Sources, Ingestion, Catalogage

- **Livrables** : données brutes (Bronze), schéma
- **Outils** : **Python**, **SQL**, connecteurs, **Docker** (envs)

2. Préparation → Nettoyage, Feature Eng, Qualité

- **Livrables** : tables nettoyées (Silver), features
- **Outils** : **pandas** / **polars**, **Great Expectations**, **Python**

3. Modélisation → Training, Tuning, Sélection

- **Livrables** : modèle candidat + métriques
- **Outils** : **scikit-learn**, **XGBoost**, **Python**

4. Évaluation → Validation, Explainability, Fairness

- **Livrables** : rapport d'éval (metrics, SHAP)
- **Outils** : **scikit-learn metrics**, **SHAP**, **MLflow** (tracking)

5. Déploiement → Packaging, API/Batch, Tests

- **Livrables** : artefact modèle, **API**/job prêt prod
- **Outils** : **FastAPI**, **Docker**, **CI/CD**

6. MLOps → Monitoring, Registry, Retrain

- **Livrables** : modèle **versionné**, dashboards, alertes
- **Outils** : **MLflow** (registry), observabilité, jobs planifiés

Collecte (cycle ML)

Données : fondation du Machine Learning



Collecter les bonnes données, au bon moment, avec les bons droits – pour éviter biais, fuites et dette technique.



Couverture des cas d'usage



Classes, segments, edge cases.



Qualité & traçabilité



Données et labels fiables, auditable.



Conformité & éthique



Consentement, PII, droits d'usage.

WyTaSoft

Collecter les bonnes données, au bon moment, avec les bons droits – pour éviter biais, fuites et dette technique.

Objectifs (ML-first)

Couverture des cas d'usage (classes, segments, edge cases).

Qualité/traçabilité des données et des labels.

Conformité (consentement, PII, droits d'usage).

Collecte - Sources & stratégies (cycle ML)

Collecte – Sources & stratégies



Online

Événements (logs, clics, IOT, Kafka)



Batch

Bases, data lake, exports métier



Labels

Annotation interne/externe
weak supervision, règles



Synthétique

Génération/augmentation
pour classes rares

Online : événements (logs, clics, IoT, Kafka).

Batch : bases, data lake, exports métier.

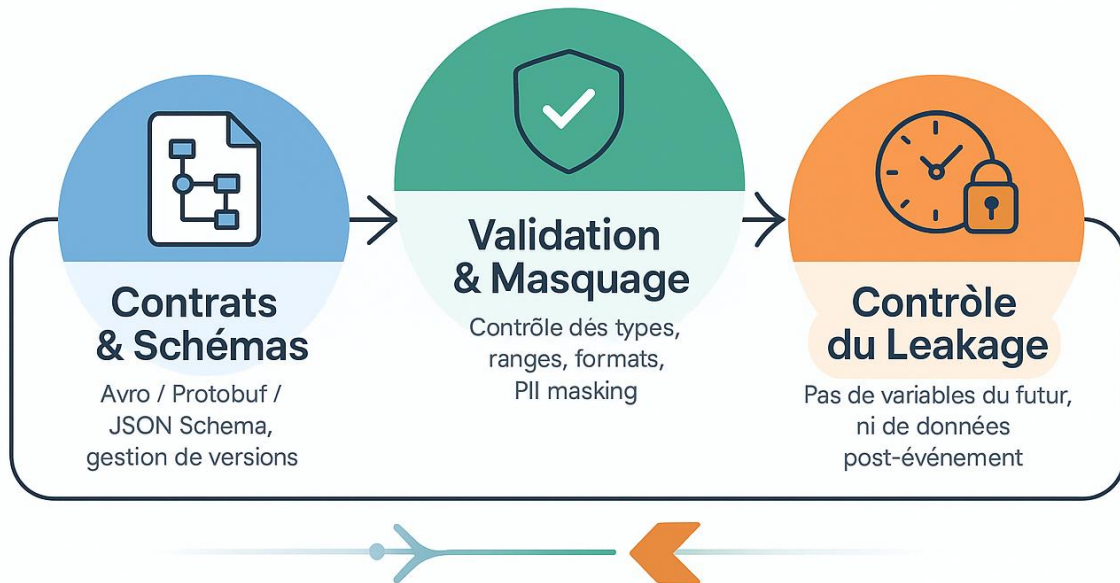
Labels : annotation interne/externe, **weak supervision**, règles.

Synthétique : génération/augmentation pour classes rares.

Collecte - Data contracts & schéma (cycle ML)

Data Contracts & Schémas

Assurer qualité, sécurité et intégrité des données
tout au long du pipeline ML



Collecte - Biais & représentativité (cycle ML)

Biais & Représentativité

Garantir l'équité et la couverture des données pour un apprentissage fiable et éthique.



Stratification intelligente

Par classe, segment, region



Équilibre des classes

Undersampling / oversampling guidés



Audit des biais

Groupes sensibles, fairness check représentativité



Détection d'anomalies

Données manquantes, outliers, distributions atypiques

Une donnée représentative, c'est un modèle plus juste et plus robuste.

Une donnée représentative, c'est un modèle plus juste

WyTaSoft

WYTA
SOFT

Collecte - Datasets & versioning (cycle ML)

De la donnée brute au jeu prêt pour le modèle –
qualité, traçabilité et reproductibilité.

BRONZE



Logs, IoT, exports,
collectes initiales

SILVER



Nettoyage, filtrage,
validation, align-
ement de schémas

GOLD



Jeux enrichis,
features finales.
coherence et stabilité



VERSIONNEMENT

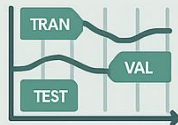
DVC / Git-LFS,
hash immuable,
tracabilite totale



DATA CARD

Documentation
du dataset :
origine, taille,
biais, usages

**DÉCOUPAGE
TRAIN / VAL / TEST**



Split cohérent
(chronologique si
séries / produit)

Bronze/Silver/Gold pour cru → propre → prêt
modèle.

Versionner jeux (DVC/Git-LFS), **hash** immuable, *data card* (fiche).

Split propre : train/val/test **chronologique** si
séries/produit.

Collecte - Observabilité (cycle ML)

Surveiller la fraîcheur, la qualité et la fiabilité des données à la source.



Qualité des flux

Freshness (P95),
volumétrie, taux d'échec,
dérive (PSI / KL)



Contrôle des rejets

DLQ, logs d'annotation
(qui, quand, qualité)



Suivi des labels

IAA, bruit estimé, qualité des labels

Freshness (P95), volumétrie, taux d'échec, drift (PSI/KL).

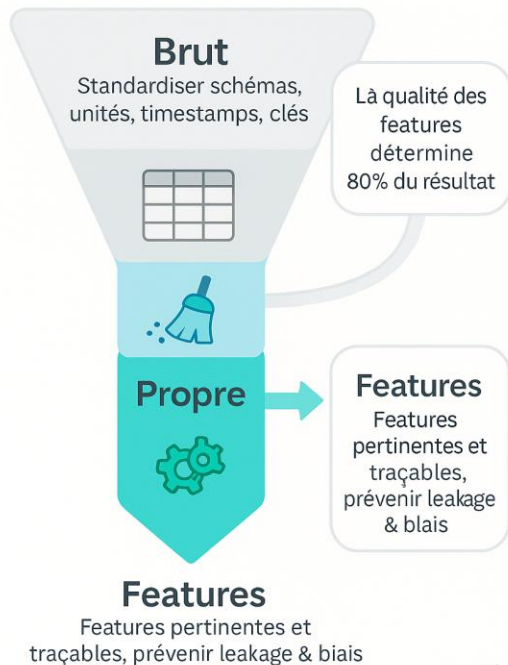
DLQ (rejets), logs d'annotation (qui, quand, qualité).

KPIs labels : IAA (accord annotateurs), bruit estimé.

Préparation : objectifs & périmètre (cycle ML)

Préparation : objectifs & périmètre

Nettoyer, structurer, fiabiliser et enrichir les données avant le modèle.



Nettoyer, structurer, **fiabiliser** et **enrichir** les données avant le modèle.

- Standardiser schémas, unités, timestamps, clés.
- Gérer **valeurs manquantes/outliers**.
- **Features** pertinentes et traçables.
- Prévenir **leakage** & biais.

Préparation : Hygiène de données (Clean & Standardize) (cycle ML)

Hygiène de données (Clean & Standardize)

Un socle cohérent et reproductible.

	Avant	Après
 Types & unités	12,5 kg / 27,56 lb 01-02-2023 (US)	 5.67 kg 2023-01 (UTC)
 Ded - blonnage taliseles harmonisation	Client_ID: 12345 / client id: 12345 / CLIENT-12345	client_id = 12345 client.id = 1345
	Age: 999 Price: NULL 	Age: 45 (median) Price: 120.5 (winsorized)

La qualité des données détermine la qualité du modèle.



Types & unités
Cast, TZ, locales, encodage.



Missing
Drop / impute (mean, median, knn, indicator).



Typés & unités
Cast, TZ, locales, encodage.



Dédoublonnage
Cles stables, harmonisation libelles.



Outliers
Winsorize, robustscaling, regles metier.

Modélisation : objectifs & cadre (cycle ML)

Modélisation : objectifs & cadre

Produire un modèle robuste, explicable et mesurable, optimisé pour l'objectif métier.

Produire un **modèle robuste, explicable et mesurable**, optimisé pour l'objectif métier.



Cadrer la **variable cible**, la granularité et la fenêtre temporelle



Fixer les **contrats d'entrée** (features gelées), choix de modèle, protocole de validation



Définir les **métriques métier** : coût FP/FN, précision, rappel, AUC, etc.

- Cadrer **variable cible**, granularité, fenêtre temporelle
- Définir **métriques métier** (ex. coût FP/FN)
- Fixer **contrats d'entrée** (features gelées)

Un bon modèle, c'est d'abord un bon cadrage.

Modélisation : Baseline & métriques (cycle ML)

Toujours battre une baseline simple et la mesurer proprement.

Baseline



AUC	0.50
PR-AUC	0.20
F1	0.40
MAE	12 +
RMSE	18 +
MAPE	40 %

Modèle #1



AUC	0.72
PR-AUC	0.45
F1	0.55
MAE	7 ✓
RMSE	9 ✓
MAPE	20 %

Modèle #2



AUC	0.86
PR-AUC	0.68
F1	0.70
MAE	5 ✓
RMSE	6 ✓



PR-AUC

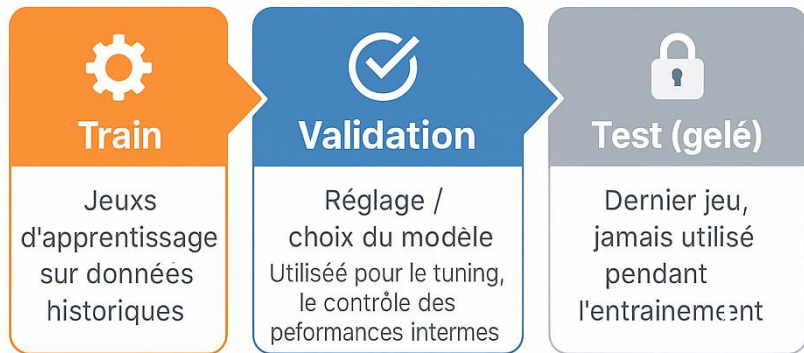
Toujours battre une **baseline simple** et la mesurer proprement.

- Baselines : moyenne/médiane, **logistic**, **Naive**
- Métriques : **AUC/PR-AUC**, **F1**, **MAE/RMSE**, **MAPE**
- Courbes : PR pour classes rares

Un bon modèle se prouve,
il ne se devine pas.

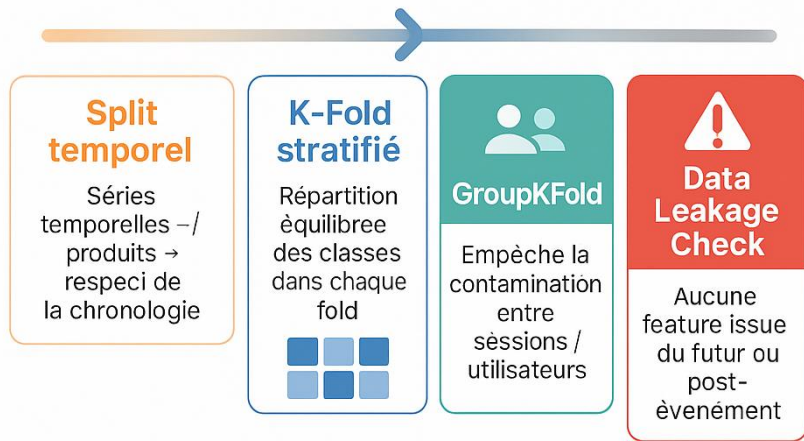
Modélisation : Stratégie d'entraînement/validation (cycle ML)

Une validation représentative de la production.



Une validation représentative de la prod.

- **Split temporel** si séries/produit ; sinon **k-fold stratifié**
- **GroupKFold** si fuites par utilisateur/session
- **Data leakage checks** (features futures)

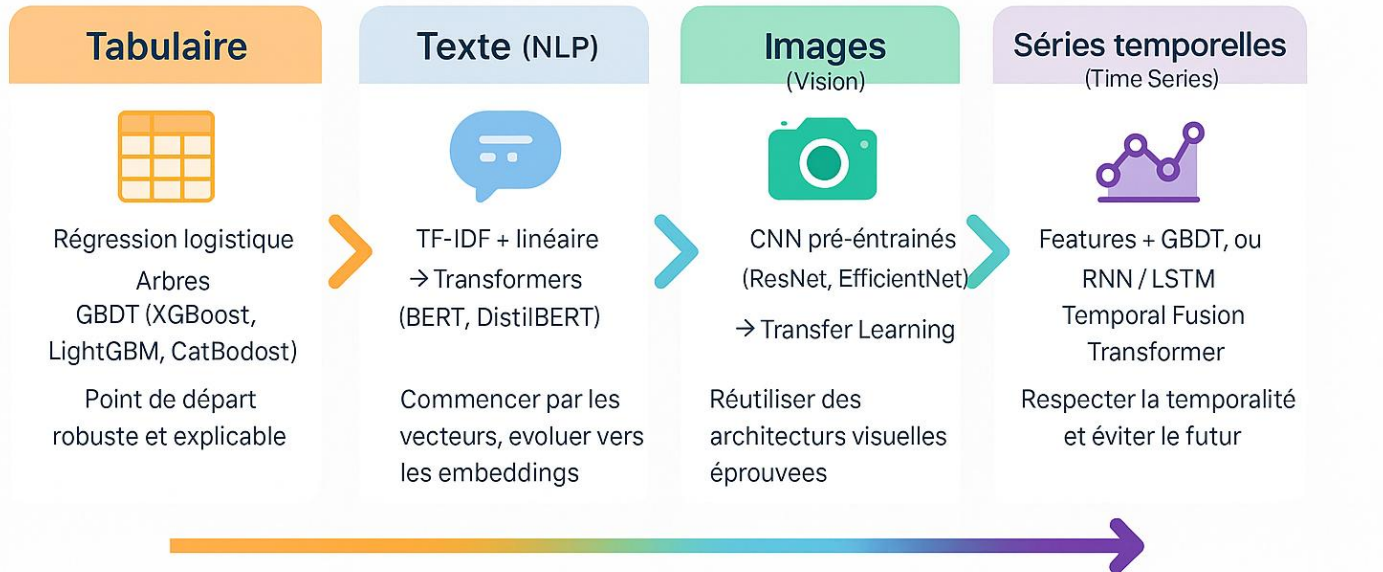


Modélisation : Sélection d'algorithmes (cycle ML)



Choisir simple d'abord, spécifique ensuite

Sélection d'algorithmes

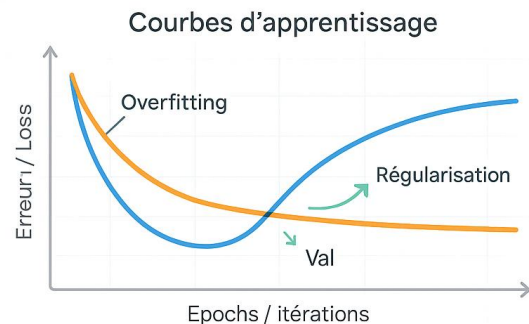


La simplicité bien maîtrisée est le meilleur point de départ.

Overfitting & régularisation



Généraliser > mémoriser.



Régularisation
linéaire



Ridge, Lasso,
ElasticNet
(pénalisation
du modèle)

Régularisation
neuronale



Dropout, Data
Augmentation,
Label Smoothing

Validation &
calibration



Learning curves,
calibration
des scores

Modélisation : Overfitting & régularisation

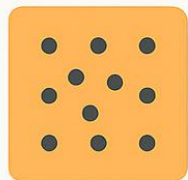
- Ridge/Lasso/ElasticNet, early stopping
- Dropout/data aug (DL), label smoothing
- Validation learning curves, calibration des scores

*"Un modèle qui apprend trop bien finit
par oublier pourquoi il apprend."*

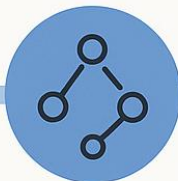
Hyper-paramétrage (Tuning)



Automatiser, contrôler, journaliser.



Espace de recherche



Essais (Trials)



Meilleure configuration



Search

Random / Bayes (Optuna),
grids courtes



Budget

Arrêts précoces,
pruning.
ressources maîtrisées



Tracking

MLflow : params,
métriques,
artefacts

Modélisation : Hyper- paramétrage (tuning)

Search : Random/Bayes (Optuna), grids courtes

Budget : arrêts précoces, **pruning**

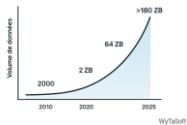
Tracking : **MLflow** (params, metrics, artefacts)

Un tuning maîtrisé vaut mieux qu'un miracle irrépérable.

De l'ingestion à la visualisation : Talend, Qlik, Tableau & co



De l'ingestion à la visualisation : Talend, Qlik, Tableau & co



Qu'est-ce qu'un ETL provider?



Connectivité & métadonnées



Architecture type Talend



Démo

Qu'est-ce qu'un ETL provider?



Une plateforme qui connecte, transforme et charge des données, de façon fiable, traçable et reproductible.

Le cycle **ETL** (Extract → Transform → Load)

- **Extract** : fichiers, APIs, DB (JDBC), SaaS.
- **Transform** : mapping, jointures, agrégations, règles métier.
- **Load** : DWH/Lake (Parquet/Delta), DB, files, message bus.

ETL / ELT (Talend & co.)

- **Technos** : Talend, Informatica, AWS Glue, Azure Data
- **Factory**, dbt (ELT SQL).
- **Forces** : connecteurs riches, jobs visuels, gouvernance.
- **Limites** : risque de lock-in, coûts, complexité d'exploitation.
- **Quand ?** : migrations, intégrations hétérogènes, data marts.

Architecture type **Talend**

- **Studio** (design visuel) → **Jobs** (pipelines) → **Talend**
- **Runtime/Workers** (exécution) → **Admin/Control** (planification, monitoring).
- **Référentiels** : Job Repository (Git), Metadata/Context, Secrets.

Démo Talend

Qlick Cloud Analytics — Vue d'ensemble?

Plateforme **SaaS** d'analytique moderne, moteur **associatif**, **auto-ML** léger, gouvernance centralisée.

À retenir : pas de serveurs à gérer, déploiement rapide, collaboration.

De la **donnée** au **tableau de bord** dans un hub cloud unifié.

Couches : Connecteurs → Data Prep (**Qlik Data Manager / scripts**) → Modèle en mémoire → Apps → Alerting/Automations.

Explorer **sans requêtes SQL** explicites : *sélectionné/possible/exclu*.

Bénéfices : découverte ad hoc, questions imprévues, vitesse.

Visuel : schéma couleurs vert/blanc/gris + liste de filtres dynamiques.

2.1



Fondamentaux du Data Engineering

Fondamentaux du Data Engineering



1. HDFS : Principes et Fonctionnement



2. HDFS : Les données vers HDFS

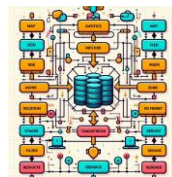


3. HIVE : database & SQL

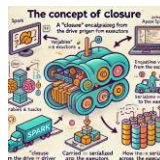
Wide Transformations (shuffles)
1 to N



4. Apache PIG



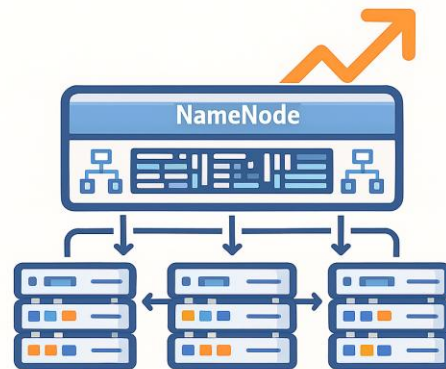
5. Introduction aux ETL



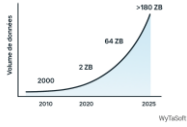
6. Spark Streaming

Fondamentaux du Data Engineering

HDFS : Principes et Fonctionnement



HDFS : Principes et Fonctionnement



Qu'est-ce que HDFS ?



NameNode

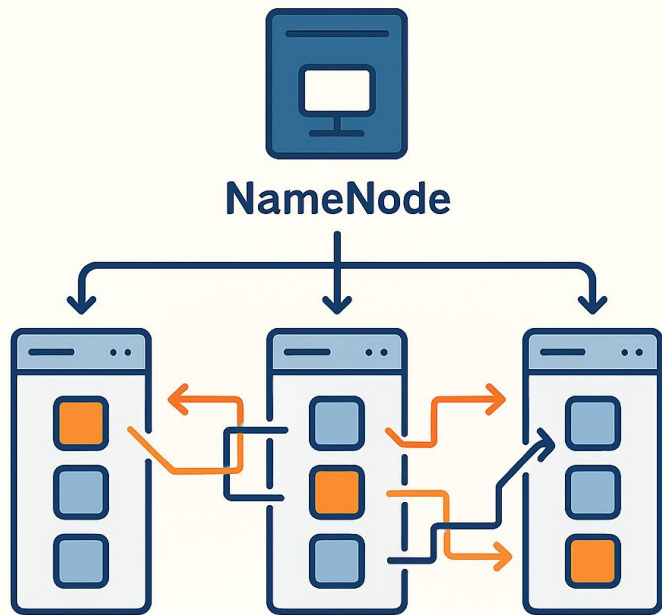


Architecture : NameNode & DataNodes



DataNodes

Qu'est-ce que HDFS ?

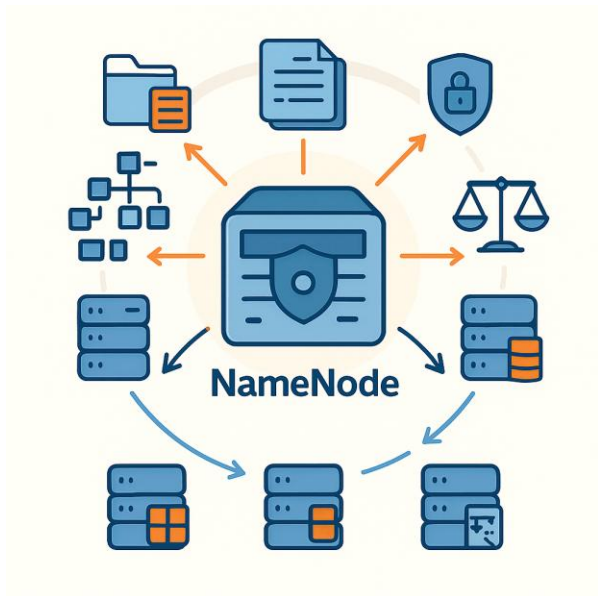


- Système de fichiers distribué utilisé par Hadoop
- Optimisé pour le stockage et le traitement de données massives
- Conçu pour fonctionner sur des clusters de serveurs “commodity”
- Haute tolérance aux pannes et forte évolutivité
- Stockage orienté **Write Once — Read Many**

Objectifs de HDFS

- Stocker des volumes massifs de données (TB → PB)
- Distribuer les données sur plusieurs machines
- Assurer la continuité même en cas de panne de nœud
- Permettre le traitement parallèle (MapReduce, Spark...)
- Idéal pour données structurées, semi-structurées et non structurées

Architecture : NameNode & DataNodes



NameNode

- Gère la *métadonnée* (arborescence, permissions, blocs)
- Point central du cluster

DataNodes

- Stockent physiquement les blocs de données
- Exécutent les opérations de lecture/écriture

Secondary NameNode

- Gère les checkpoints du NameNode (≠ backup !)

NameNode : Le Cerveau de HDFS

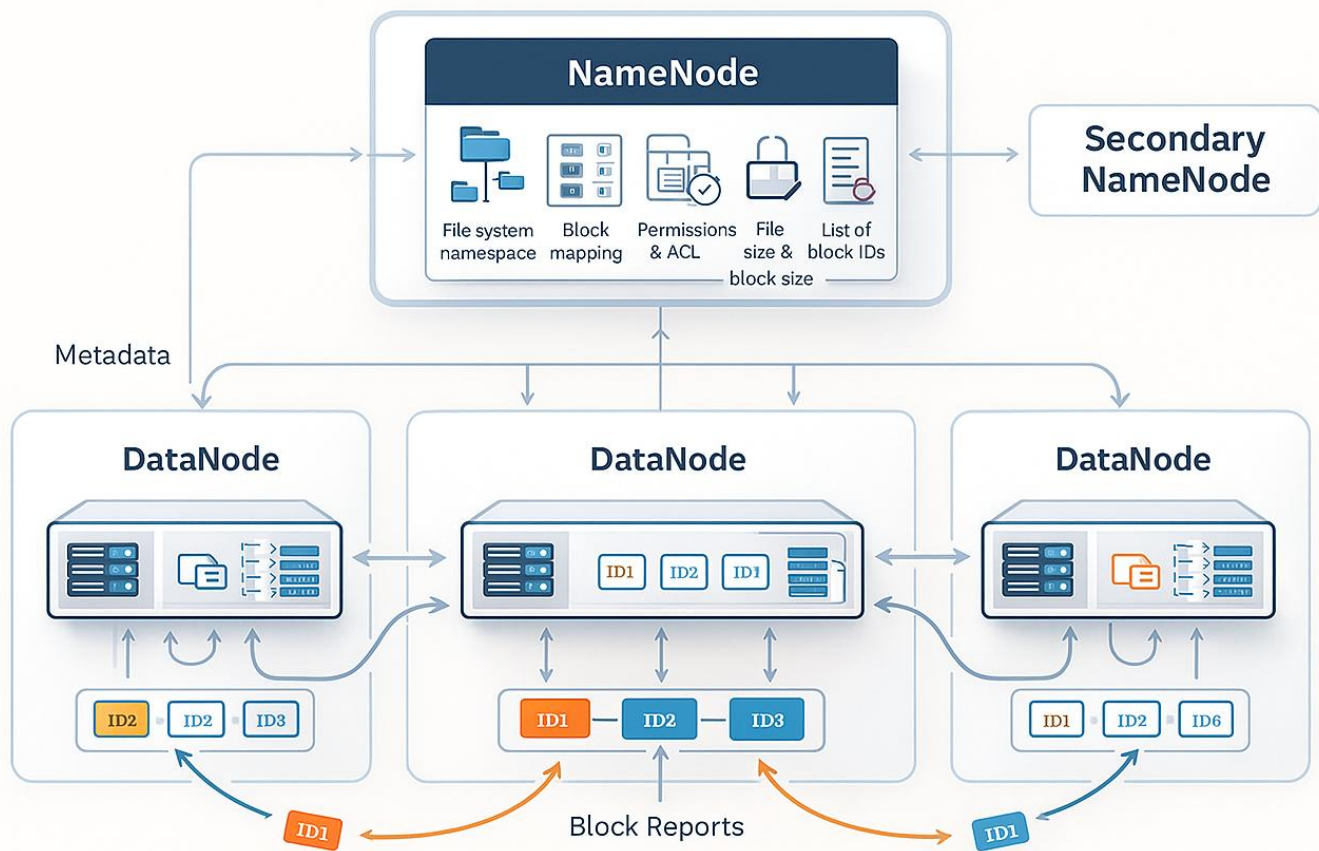
Rôle principal : Gestion de la métadonnée

Gère l'arborescence du système de fichiers HDFS

Conserve l'inventaire de tous les fichiers et répertoires

Ne stocke *pas* les données, uniquement les **métadonnées** :

- Nom des fichiers
- Permissions
- Taille
- Liste des blocs
- Emplacement des blocs sur les DataNodes



NameNode : Fonctionnement interne

Maintient deux fichiers essentiels :

- **fsimage** : snapshot complet du système de fichiers
- **edits** : journal des modifications depuis le dernier snapshot

Charge ces informations en mémoire RAM pour assurer rapidité et disponibilité

Communique avec les DataNodes à travers des messages réguliers :

- **Heartbeats** : état du nœud
- **Block Reports** : liste des blocs stockés

DataNodes : Les travailleurs de HDFS

Stockent physiquement les **blocs de données** d'HDFS

Chaque fichier → découpé en plusieurs blocs → répartis sur plusieurs DataNodes

Responsable de :

- **Lecture** des blocs demandés par les clients
- **Écriture** des blocs envoyés par les clients
- **Réplication** automatique (en coopération avec le NameNode)

DataNodes $\rightleftarrows$ NameNode : une communication permanente

Envoient régulièrement des **Heartbeats** (toutes les 3 secondes typiquement)

→ indique que le DataNode est vivant

Envoient périodiquement des **Block Reports**

→ la liste exhaustive des blocs qu'ils stockent

Reçoivent des ordres du NameNode :

- Ajouter une réplique
- Supprimer un bloc
- Rééquilibrer des blocs entre nœuds (balancing)

DataNodes : Stockage en blocs et réplication

- Les blocs sont stockés dans le filesystem local du DataNode
- Le facteur de réplication détermine combien de copies d'un bloc sont stockées
- HDFS assure que les réplications sont placées sur différents nœuds et racks
- Le DataNode participe à la réplication en envoyant des blocs à d'autres DataNodes
- Si un DataNode tombe :

→ Les autres reconstituent les répliques manquantes

DataNodes : Localité des données (Data Locality)

Spark et Hadoop essaient de **déployer les tâches sur les DataNodes qui possèdent déjà les blocs**

Avantages :

- Très faible transfert réseau
- Lecture ultra rapide via disque local
- Pipelining parfait avec MapReduce ou Spark

Résultat :

- Réduction du temps de traitement
- Réduction de la congestion réseau dans le cluster

DataNodes : Défaillances et gestion des pannes

Si un DataNode ne répond plus :

→ Le NameNode le marque comme **dead**

→ Les blocs stockés dessus sont considérés comme perdus

→ HDFS réplique automatiquement les blocs à partir des copies restantes

Le système continue de fonctionner tant que le facteur de réplication minimal est assuré

Rebalancer (HDFS Balancer) peut être utilisé pour redistribuer les blocs après une panne

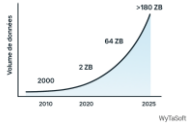
Limitations des DataNodes

- Dépendent fortement des performances du disque local
- Ne conviennent pas pour des lectures aléatoires intensives
- Peuvent devenir des bottlenecks si la réplication ou rebalancing est trop fréquente
- Fragiles en cas de mauvaise gestion du hardware (disques pleins, défaillances...)

Fondamentaux du Data Engineering

HDFS : Les données vers HDFS

HDFS : Les données vers HDFS



Introduction aux commandes HDFS



HDFS et les permissions



HDFS et les fichiers



Résumé des commandes

Introduction aux commandes HDFS

HDFS offre une commande centrale : **hdfs dfs** pour gérer fichiers et répertoires dans le cluster Hadoop.

- HDFS ≠ système de fichiers local
- Commandes inspirées de Linux
- Permettent : explorer, importer, lire, déplacer, supprimer...

Anatomie d'une commande **HDFS**



```
hdfs dfs -<action> <arguments>
```

Chaque commande suit une structure simple :

- **hdfs dfs** : client CLI
- **-action** : ce que vous faites
- **argument(s)** : chemin ou fichier

Exemples

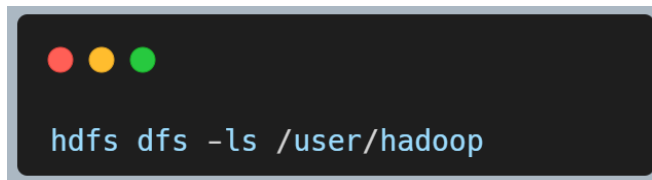
- `hdfs dfs -ls /`
- `hdfs dfs -mkdir /training`

Explorer **HDFS** (ls, mkdir, du, count...)

Naviguer dans HDFS est très proche de Linux.

Action	Commande
Lister	<code>hdfs dfs -ls /</code>
Créer répertoire	<code>hdfs dfs -mkdir /training</code>
Lister récursivement	<code>hdfs dfs -ls -R /training</code>
Taille d'un dossier	<code>hdfs dfs -du -h /training</code>
Compter fichiers/blocs	<code>hdfs dfs -count /training</code>

Exemple :

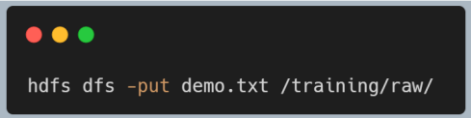
A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The command `hdfs dfs -ls /user/hadoop` is entered in a light blue monospace font.

```
hdfs dfs -ls /user/hadoop
```

Importer / Exporter des fichiers HDFS

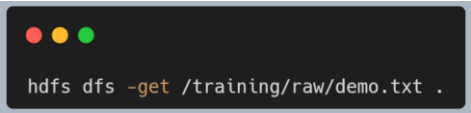
Deux commandes à retenir : **put** (local → HDFS) et **get** (HDFS → local).

- Importer (local → HDFS)



```
hdfs dfs -put demo.txt /training/raw/
```

- Exporter (HDFS → local)



```
hdfs dfs -get /training/raw/demo.txt .
```

- Copier dans HDFS



```
hdfs dfs -cp /src/file.txt /dest/
```

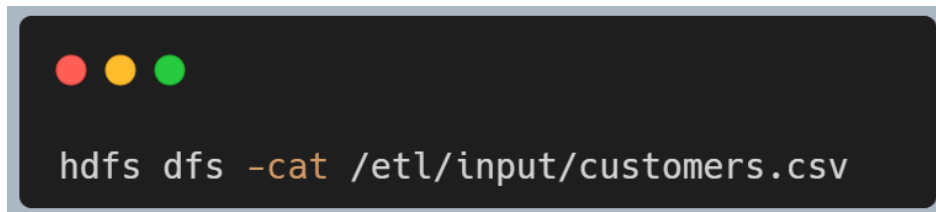

Lire ou afficher des fichiers HDFS

HDFS permet de lire sans télécharger.

Commandes

Comment lire ?	Exemple
Voir contenu	<code>hdfs dfs -cat /training/raw/demo.txt</code>
Voir 1ère ligne	<code>hdfs dfs -head /file.csv</code>
Voir dernières lignes	<code>hdfs dfs -tail /file.csv</code>

Exemple

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The command `hdfs dfs -cat /etl/input/customers.csv` is entered in a light blue monospace font.

```
hdfs dfs -cat /etl/input/customers.csv
```

Déplacer, renommer et supprimer

On retrouve les mêmes actions que sur un système local.

Déplacer



```
hdfs dfs -mv /training/raw/demo.txt /training/processed/
```

Renommer



```
hdfs dfs -mv /training/processed/demo.txt /training/processed/demo_v2.txt
```

Supprimer



```
hdfs dfs -rm /training/raw/old.txt  
hdfs dfs -rm -r /training/archive
```

Permissions & propriétaires

HDFS possède une gestion des permissions similaire à Linux.

Changer permissions



```
hdfs dfs -chmod 755 /training
```

Changer propriétaire



```
hdfs dfs -chown hadoop:hadoop /training
```

Changer groupe



```
hdfs dfs -chgrp analysts /training
```

Vérifier blocs & réplication

HDFS découpe les fichiers en blocs et les réplique.

Voir détails d'un fichier



```
hdfs dfs -stat %r /training/raw/demo.txt
```

→ facteur de réplication

Voir blocs



```
hdfs fsck /training/raw/demo.txt -files -blocks -locations
```

Modifier replication



```
hdfs dfs -setrep 1 /training/processed/demo_copy.txt
```

Résumé des commandes essentielles

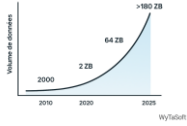
5 familles à retenir :

Catégorie	Commandes
Explorer	ls, mkdir, du, count
Import/export	put, get, cp
Lire	cat, head, tail
Manipuler	mv, rm
Administration	setrep, chmod, chown, fsck

Fondamentaux du Data Engineering

HIVE : database & SQL

HIVE : database & SQL



Introduction à Apache Hive



MANAGED vs EXTERNAL



Commandes & bases de données



Hive & ORC/PARQUET

Introduction à Apache Hive

Apache Hive est un **moteur d'entrepôt de données** (Data Warehouse) construit au-dessus d'Hadoop.

Il permet :

- d'exécuter des requêtes **SQL** sur de grands volumes de données
- de stocker les tables dans **HDFS**
- d'analyser des datasets distribués grâce à des moteurs comme **Tez**, **MapReduce** ou **Spark**
- de faciliter le travail des analystes et data engineers avec un langage familier : **HiveQL**

Pourquoi **Hive**

Rend le Big Data **accessible en SQL**

Permet de structurer, transformer et analyser de grands jeux de données

S'intègre nativement avec :

- **HDFS**
- **YARN**
- **Apache Spark**
- **Écosystèmes ETL et BI**

Supporte les formats optimisés : ORC, Parquet, Avro...

Architecture simplifiée Hive

HiveServer2 → exécute les requêtes SQL

Metastore → stocke le schéma des tables

HDFS → stocke les fichiers de données

Engine (Tez/MapReduce/Spark) → exécute physiquement les traitements

Lancer HiveServer2 & se connecter (Beeline)

Beeline est le client SQL de Hive.



```
beeline -u jdbc:hive2://localhost:10000
```

Commandes & bases de données

Afficher les bases existantes

```
SHOW DATABASES;
```

Créer une base

```
CREATE DATABASE formation_hive;
```

Utiliser une base

```
USE formation_hive;
```

Supprimer une base

```
DROP DATABASE formation_hive;
```

Tables MANAGED vs EXTERNAL

■ Table MANAGED

Hive contrôle les données, les fichiers sont supprimés avec la table.

■ Table EXTERNAL

Les fichiers sont dans un dossier choisi → Hive ne les supprime pas.

À retenir :

EXTERNAL : autonomie des fichiers —

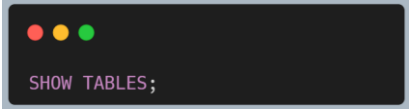
MANAGED : Hive gère tout.

```
CREATE TABLE transactions (  
  id INT,  
  amount DOUBLE,  
  date STRING  
);
```

```
CREATE EXTERNAL TABLE clients (  
  id INT,  
  name STRING,  
  country STRING,  
  age INT  
)  
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','  
LOCATION '/data/clients';
```

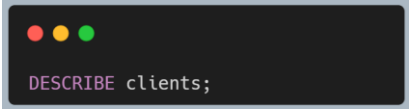
Explorer les tables

Lister les tables



```
SHOW TABLES;
```

Description rapide



```
DESCRIBE clients;
```

Description complète



```
DESCRIBE FORMATTED clients;
```

Charger des données dans tables

Charger un fichier local



```
LOAD DATA LOCAL INPATH '/path/file.csv'  
INTO TABLE transactions;
```

Charger un fichier depuis HDFS



```
LOAD DATA INPATH '/data/clients/clients.csv'  
INTO TABLE clients;
```

SQL & Hive

Sélection simple



```
SELECT * FROM clients;
```

Filtrer



```
SELECT * FROM transactions WHERE amount > 100;
```

Agréger



```
SELECT country, COUNT(*) FROM clients GROUP BY country;
```

Trier



```
SELECT * FROM transactions ORDER BY amount DESC;
```


JOINS

Hive réalise un join distribué → lecture de plusieurs fichiers en parallèle.



```
SELECT
    c.name, c.country,
    t.amount, t.category, t.date
FROM clients c
JOIN transactions t ON c.id = t.customer_id
ORDER BY t.amount DESC;
```

Tables partitionnées

Créer une table partitionnée

```
CREATE TABLE tx_part (  
    txn_id INT,  
    customer_id INT,  
    amount DOUBLE,  
    category STRING  
)  
PARTITIONED BY (year STRING, month STRING);
```

Ajouter une partition

```
ALTER TABLE tx_part ADD PARTITION (year='2024', month='02');
```

Format ORC

Créer une table ORC

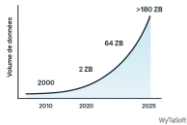
```
CREATE TABLE clients_orc (  
  id INT,  
  name STRING,  
  country STRING,  
  age INT  
)  
STORED AS ORC;
```

Fondamentaux du Data Engineering



Apache PIG

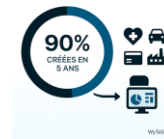
Apache PIG



Introduction à Apache Pig



Load avec Aapache Pig



Quelques CMD Apache Pig



Write avec Apache Pig

Qu'est-ce que Apache Pig ?

Apache Pig est une plateforme Big Data permettant de **manipuler, transformer et analyser de grands volumes de données** en utilisant un langage simple : **Pig Latin**.

- Créé initialement par Yahoo!, Pig facilite l'écriture de pipelines de données complexes sans avoir à coder directement en Java/MapReduce.



Pourquoi utiliser Apache Pig ?

Simplifie l'écriture des traitements distribués

Langage déclaratif simple (Pig Latin)

Très efficace pour :

- Nettoyage de données
- Préparation / transformation
- Agrégation et pré-traitement massif

Fonctionne au-dessus de Hadoop (MapReduce)

Lancer Pig & environnement

 Mode Local (pour tests rapides)

```
pig -x local
```

 Mode Hadoop / MapReduce

```
pig
```

 Exécuter un script .pig

```
pig script.pig
```

Pig Latin s'exécute en workflow → Pig compile le script → génère des jobs MapReduce.

Charger des données (LOAD)

Exemple de fichier CSV

Commande Pig

Points importants :

- PigStorage(',') → séparateur CSV
- Toujours **définir les schémas** (id:int, name:chararray...)
- Le résultat est un **bag** (table) manipulable dans Pig

```
clients = LOAD 'data/clients.csv'  
         USING PigStorage(',')  
         AS (id:int, name:chararray, age:int);
```

Explorer les données

Afficher le contenu (debug)



```
DUMP clients;
```

Voir le schéma



```
DESCRIBE clients;
```

Visualiser le plan logique



```
EXPLAIN clients;
```

Filtrer les données

- Condition = style SQL
- Donne un **nouveau bag** filtré
- Pig Latin est immuable (comme Spark / Hive)



```
adults = FILTER clients BY age > 30;
```

Sélectionner & Transformer

Projeter des colonnes



```
names = FOREACH clients GENERATE name;
```

Calculer une nouvelle colonne



```
with_category = FOREACH clients GENERATE  
    name,  
    age,  
    (age >= 30 ? 'Adult' : 'Young') AS category;
```

Pig Latin permet des expressions très flexibles.

Group By & Agrégations

Grouper les données



```
grouped = GROUP clients BY country;
```

Compter



```
counts = FOREACH grouped GENERATE  
  group AS country,  
  COUNT(clients) AS total;
```

Moyenne



```
avg_age = FOREACH grouped GENERATE  
  group AS country,  
  AVG(clients.age) AS avg_age;
```

Pig fonctionne comme SQL, mais avec une logique pipeline.

Sauvegarder les résultats

Sauvegarde au format CSV

```
STORE counts INTO 'output/counts'  
USING PigStorage(',');
```

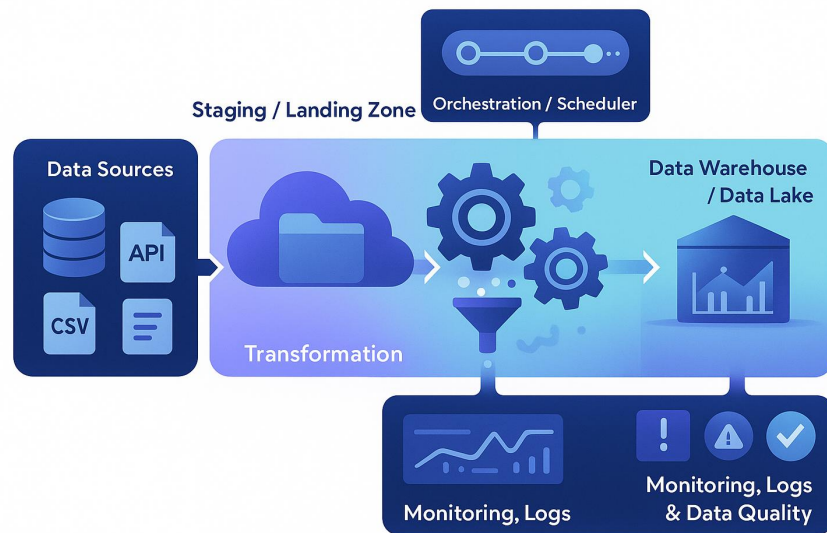
Sauvegarde binaire (optimisée)

```
STORE data INTO 'output/data'  
USING BinStorage();
```

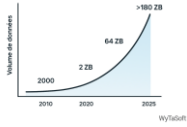
Pig écrit toujours dans un dossier, jamais un fichier unique.

Fondamentaux du Data Engineering

Introduction aux ETL



Introduction aux ETL



Qu'est-ce qu'un ETL ?



Architecture d'un pipeline ETL



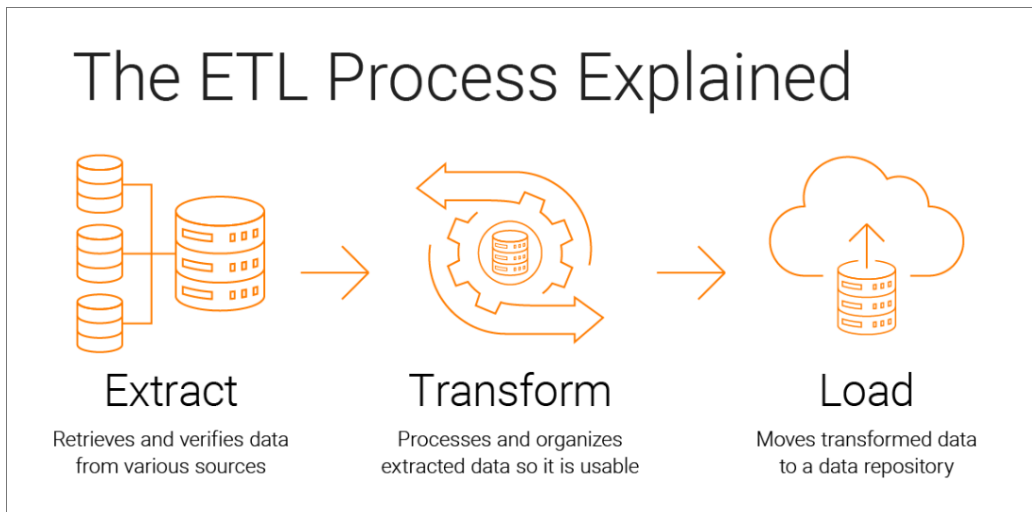
Les 3 étapes clés ETL



ETL vs ELT

Qu'est-ce qu'un ETL ?

- ETL = **Extract** – **Transform** – **Load**
- Processus central dans les architectures **Data Warehouse / Big Data**
- Objectif : **nettoyer, structurer et consolider** les données pour l'analyse
- Exemples d'outils : **Talend, Informatica, DataStage, Pentaho, AWS Glue**



Pourquoi utiliser un ETL ?

- Hétérogénéité des sources (ERP, API, fichiers, bases SQL/NoSQL...)
- Gestion des volumes importants
- Amélioration de la qualité des données
- Orchestration et planification des traitements
- Traçabilité & audit : qui a modifié quoi et quand ?

Les 3 étapes clés ETL

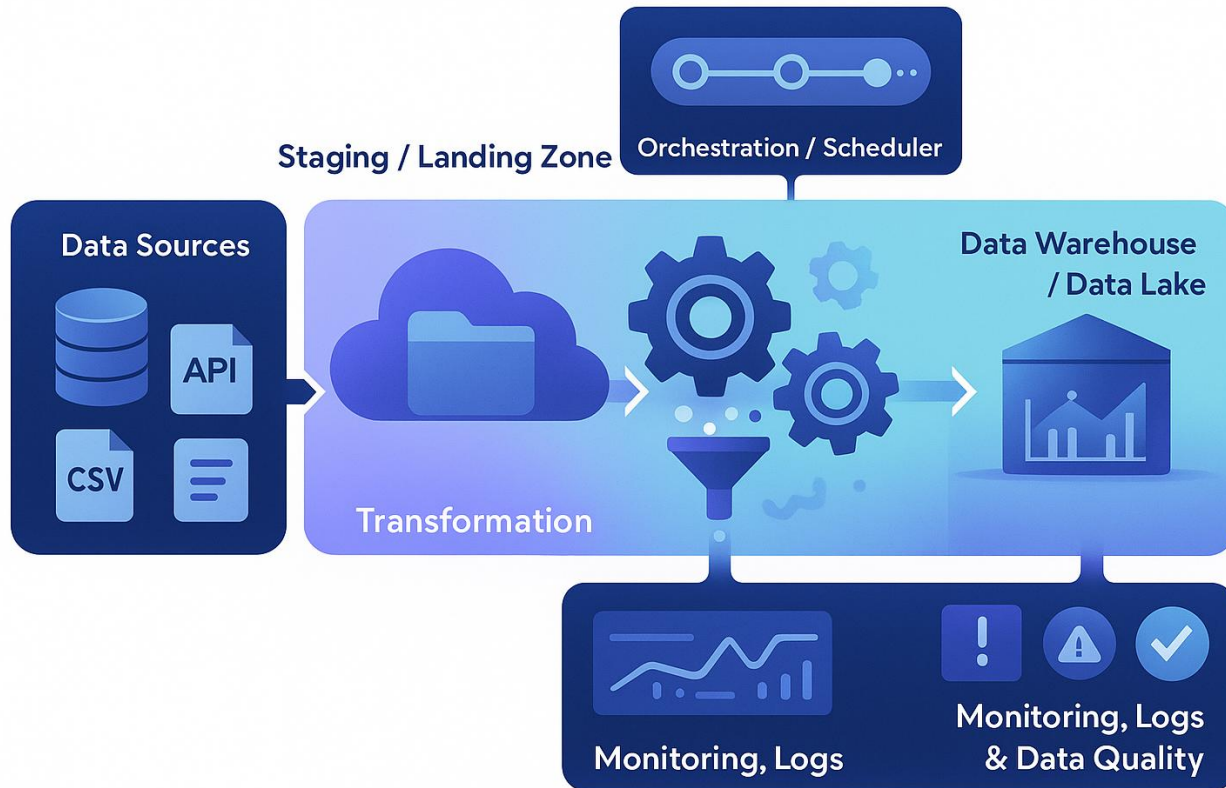
- **Extract** : lire les données (connecteurs : DB, fichiers, Web Services...)
- **Transform** : nettoyer, enrichir, normaliser, appliquer des règles métier
- **Load** : charger vers le Data Warehouse, Data Lake, ou application cible

Architecture d'un pipeline ETL

Un pipeline ETL est une chaîne cohérente et entièrement automatisée qui transforme des données brutes en données prêtes à l'analyse.

- 1 Sources de données
- 2 Zone de Staging (Landing Zone)
- 3 Zone de Transformation
- 4 Zone d'Exploitation : Data Warehouse / Data Lake
- 5 Orchestration & Planification (Scheduler)
- 6 Logs, Monitoring & Qualité

Architecture d'un pipeline ETL



ETL : Sources de données

- Bases de données (SQL/NoSQL)
- Fichiers (CSV, XML, JSON...)
- APIs, ERP, CRM
- Flux en streaming

➡ *Origine hétérogène des données à intégrer*

ETL : Zone de Staging (Landing Zone)

- Stockage temporaire des données brutes
- Aucune transformation appliquée
- Permet de **conserver l'historique brut** et de **séparer extraction et transformation**

➡ *Zone tampon pour absorber la variabilité des sources*

ETL : Zone de Transformation

- Nettoyage (qualité, formats, doublons)
- Enrichissement (jointures, règles métier)
- Normalisation (dimensions, clés techniques)

➡ *Le cœur du travail ETL où les données prennent de la valeur*

ETL : Zone d'Exploitation : Data Warehouse / Data Lake

- **Data Warehouse** → données structurées, prêtes pour la BI
- **Data Lake** → stockage massif et flexible, tous formats

➡ *Destination finale pour l'analyse, la BI et le reporting*

ETL : Orchestration & Planification (Scheduler)

- Déclenchement automatique des jobs ETL
- Gestion des dépendances
- Notification en cas d'erreur

➡ *Assure la fiabilité, la répétabilité et le respect des SLA*

ETL : Logs, Monitoring & Qualité

- Journalisation détaillée (succès, erreurs, volumes traités)
- Qualité des données (contrôles, règles, alertes)
- Supervision en temps réel

➡ *Garantit la transparence et la maîtrise du pipeline*

Exemple d'ETL : Talend

- Talend est un ETL visuel basé sur des composants.
 - Développement par **drag & drop**
 - Large catalogue de **connecteurs**
 - Jobs réutilisables et modulaires
 - Gestion des erreurs intégrée
 - Fonctionnalités d'administration (logs, monitoring, scheduling)

ETL vs ELT

ETL : Transformer avant le chargement	ELT : Transformer dans le DWH
Transformation dans l'ETL	Transformation dans le DWH (Spark, SQL)
Adapté aux systèmes traditionnels	Adapté au cloud / Big Data
Peut être plus lent	Scalabilité plus forte

Spark Streaming



Spark Streaming

Spark
Streaming



Spark Streaming

1. Introduction Spark Streaming

2. Structured Streaming

3. Streaming DataFrame

4. readStream

5. Transformation



6. window & Kafka



7. writeStream



9. foreachBatch

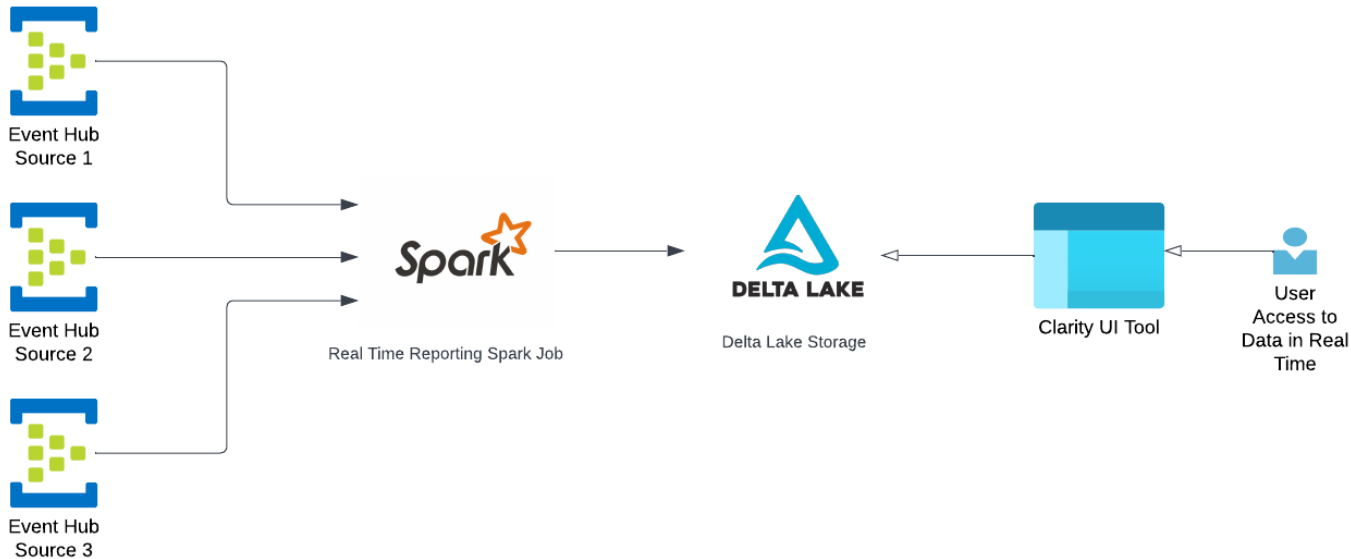


10. Intégration

Introduction au Traitement en Streaming

- Définition du streaming de données.
- Différence entre le traitement batch et le streaming.
- Présentation de **Spark Streaming** et **Structured Streaming**.

Architecture du Traitement en Streaming avec Spark



Exemple de Code pour Spark Structured Streaming

Spark read

```
# Exemple de lecture d'un flux Kafka
df = spark \
    .readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "topic_name") \
    .load()

# Exemple de transformation en streaming
df_transformed = df.selectExpr("CAST(key AS STRING)",
    "CAST(value AS STRING)") \
    .filter(df.value.contains("important"))

# Exemple d'écriture dans un stockage
df_transformed.writeStream \
    .format("parquet") \
    .option("path", "/path/to/output") \
    .start()
```

- Exemple de chargement de données depuis un flux Kafka.
- Exemple de transformation continue des données (filtrage, agrégation).
- Exemple d'écriture des résultats dans une base de données ou un stockage cloud

Use case 1 – Spark Structured Streaming

Qu'est-ce que Spark Structured Streaming ?

- API de streaming évolutive et facile à utiliser.
- Basée sur le moteur Spark SQL, unifie batch et streaming.
- Garantit la **faible latence** et le traitement **en micro-batch** ou continu.

Use case 1 – Architecture de Spark Structured Streaming

- Source de données: Kafka, fichiers, S3, etc.
- Transformations: Map, Filter, Aggregations.
- Écriture des résultats vers: HDFS, Kafka, bases de données.
- Modèle de traitement: **Micro-batch** ou **Continuous Processing**.

Use case 1 – Principales Caractéristiques

- **Traitement Faiblement Couplé:** Pas besoin de maintenir des serveurs de streaming dédiés.
- **Stateful Processing:** Gestion de l'état, comme les fenêtres temporelles.
- **Garantie d'exactitude:** Exactly-once semantics.
- **Scalabilité:** Gestion dynamique des ressources.

Use case 1 – Principales Caractéristiques

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Structured Streaming Example") \
    .getOrCreate()

# Lecture depuis une source Kafka
df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "topic") \
    .load()

# Transformation des données
transformed_df = df.selectExpr("CAST(value AS STRING) as json")

# Écriture dans une table Delta
transformed_df.writeStream \
    .format("delta") \
    .outputMode("append") \
    .option("checkpointLocation", "/wytasoft/checkpoint") \
    .start("/wytasoft/delta_table")
```

- Lecture depuis une source Kafka.
- Transformation des données.
- Écriture des résultats vers une table Delta.

Use case 1 – Avantages de Spark Structured Streaming

- Unification du batch et du streaming.
- Intégration native avec Delta Lake pour le stockage et la gestion des données.
- Hautement scalable, conçu pour fonctionner avec de grands volumes de données.
- APIs familières pour les DataFrames et SQL.

Use case 2 – Streaming DataFrame

Qu'est-ce qu'un Streaming DataFrame ?

- Extension des DataFrames pour traiter les flux de données en continu.
- Intègre des API simples et familières pour les développeurs Spark.

Use case 2 – Streaming DataFrame

- **Source de données:** Kafka, fichiers, socket, etc.
- **Transformations:** Opérations sur colonnes, agrégations, filtres.
- **Écriture:** Bases de données, Delta Lake, systèmes de fichiers.
- **Traitement:** Micro-batch ou mode continu.

Use case 2 – Streaming DataFrame

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Streaming DataFrame Example") \
    .getOrCreate()

# Lecture en continu depuis Kafka
df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "topic") \
    .load()

# Transformation et affichage
query = df.selectExpr("CAST(value AS STRING) as message") \
    .writeStream \
    .outputMode("append") \
    .format("console") \
    .start()
query.awaitTermination()
```

- Exemple de lecture en continu depuis Kafka.
- Transformation et affichage des données.

Use case 2 – Streaming DataFrame

Micro-batch:

- Traitement par lots sur des intervalles définis.
- Idéal pour les systèmes nécessitant un certain délai de latence.

Continuous Processing:

- Latence ultra-faible, traitement ligne par ligne.
- Conçu pour des cas critiques en temps réel.

Use case 2 – Meilleures Pratiques

- Toujours spécifier un checkpoint pour garantir la tolérance aux pannes.
- Optimiser les ressources pour le traitement en continu.
- Tester le pipeline avec des flux simulés avant déploiement.

Use case 3 – readStream

Qu'est-ce que readStream ?

- Une méthode Spark pour consommer des données en continu.

Sources supportées par readStream:

- **Kafka:** Consommation de messages en temps réel.
- **Sockets:** Flux de données directs depuis une adresse IP/port.
- **Fichiers:** Lecture incrémentale des données ajoutées à un dossier.
- **Delta Lake:** Lecture des mises à jour continues sur une table Delta.

Use case 3 – readStream



```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Kafka ReadStream Example") \
    .getOrCreate()

kafka_df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "topic_name") \
    .load()

kafka_df.selectExpr("CAST(key AS STRING)", "CAST(value AS STRING)") \
    .writeStream \
    .format("console") \
    .start()
```

Paramètres essentiels:

- kafka.bootstrap.servers: Serveur Kafka.
- subscribe: Topic Kafka à consommer.

Format: key, value, topic, partition, etc.

Use case 3 – readStream



```
socket_df = spark.readStream \  
  .format("socket") \  
  .option("host", "localhost") \  
  .option("port", 9999) \  
  .load()  
  
socket_df.writeStream \  
  .format("console") \  
  .start()
```

Paramètres :

- **host**: Adresse IP ou nom de domaine du serveur socket.
- **port**: Port sur lequel le flux est disponible.

Use case 3 – readStream

Configurer le checkpointing:

- Exemple: `option("checkpointLocation", "/path/to/checkpoint")`
- Importance: Garantit la tolérance aux pannes.

Gérer les volumes de données:

- Kafka: Ajuster `maxOffsetsPerTrigger` pour limiter les messages consommés.
- Sockets: Surveiller la latence pour les flux rapides.

Tester avec des données simulées:

- Exemple: Utilisation d'un serveur Kafka local ou de scripts pour envoyer des messages.

Use case 4 – Transformation de Flux

Offset Management:

- Kafka conserve les messages par partition, et chaque partition a un offset unique.
- Spark suit les offsets pour garantir que chaque message est traité une seule fois, même en cas de panne.

Spécificité:

- Contrairement aux transformations traditionnelles, les transformations sur Kafka doivent tenir compte des offsets pour assurer une continuité des flux et éviter des doublons.

```
kafka_df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "topic_name") \
    .option("startingOffsets", "earliest") \
    .load()
```

Use case 4 – Transformation de Flux

Les messages Kafka sont souvent non structurés (par exemple, JSON, Avro, ou texte brut). Cela nécessite des transformations supplémentaires pour extraire et structurer les données avant analyse.



```
from pyspark.sql.functions import from_json, col
from pyspark.sql.types import StructType, StringType

schema = StructType().add("key", StringType()).add("value", StringType())
structured_df = kafka_df.selectExpr("CAST(value AS STRING)") \
    .withColumn("data", from_json(col("value"), schema))
```

Use case 4 – Transformation de Flux

- Les opérations impliquant un shuffle, comme groupBy ou join, peuvent provoquer une latence élevée en streaming.
- Spark peut traiter chaque micro-batch en parallèle, mais les partitions doivent être équilibrées.



```
grouped_df = kafka_df.selectExpr("CAST(key AS STRING)", "CAST(value AS  
STRING)").groupBy("key").count()
```

Use case 4 – Transformation de Flux

Les opérations comme les **joins** entre deux flux en temps réel (stream-stream joins) nécessitent des fenêtres temporelles, ce qui peut augmenter la complexité.



```
from pyspark.sql.functions import window

windowed_df = kafka_df \
    .groupBy(window(col("timestamp"), "10 minutes"), col("key")) \
    .count()
```

Use case 4 – Transformation de Flux

- Kafka peut transmettre des messages en désordre (latence réseau, répliquions).
- Spark permet de gérer cela via des fenêtres avec **watermarks** pour éviter de traiter des messages trop anciens.



```
watermarked_df = kafka_df \  
    .withWatermark("timestamp", "10 minutes") \  
    .groupBy("key").count()
```

Use case 5 – window & Kafka

Objectif des fenêtres temporelles :

- Agréger des événements dans des plages de temps définies.
- Exemple : Nombre de ventes par minute.

Fonctionnalité Spark Structured Streaming :

- La fonction window permet de créer des fenêtres glissantes ou fixes.

Use case 5 – window & Kafka

Fenêtres fixes :

- Exemple : Une fenêtre de 5 minutes où les données sont regroupées uniquement par blocs de 5 minutes.

Fenêtres glissantes :

- Exemple : Une fenêtre de 5 minutes avec un décalage de 1 minute.

Fenêtres basées sur des sessions :

- Regroupement d'événements par périodes d'inactivité.

Use case 5 – window & Kafka

```
from pyspark.sql.functions import window, count

# Lecture d'un flux Kafka
kafka_stream = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "topic_name") \
    .load()

# Transformation : Application de fenêtres temporelles
windowed_count = kafka_stream \
    .selectExpr("CAST(value AS STRING)", "timestamp") \
    .groupBy(window("timestamp", "5 minutes", "1 minute")) \
    .agg(count("value").alias("count"))

# Écriture des résultats
windowed_count.writeStream \
    .format("console") \
    .outputMode("complete") \
    .start()
    .awaitTermination()
```

- Lecture d'un flux Kafka avec Spark.
- Utilisation de window pour regrouper les données dans des fenêtres temporelles.

Use case 5 – window & Kafka

- Problème des données retardataires dans les flux.
- Utilisation de with **Watermark** pour gérer les événements arrivant en retard.Exemple d'application.



```
windowed_with_watermark = kafka_stream \  
    .selectExpr("CAST(value AS STRING)", "timestamp") \  
    .withWatermark("timestamp", "10 minutes") \  
    .groupBy(window("timestamp", "5 minutes", "1 minute")) \  
    .agg(count("value").alias("count"))
```

Use case 5 – window & Kafka

- Optimisation du nombre de partitions avec **repartition**.
- Réduction des shuffles grâce au broadcast.
- Ajustement des configurations Spark :
 - `spark.sql.shuffle.partitions`.
 - Compression des shuffles (**`spark.shuffle.compress`**).

Use case 6 – writeStream

- **Objectif** : Permet l'écriture continue des résultats d'un flux dans une destination.
- **Destinations supportées** :
 - Console (pour le débogage).
 - Systèmes de fichiers (CSV, JSON, Parquet).
 - Bases de données (JDBC).
 - Kafka, Elasticsearch, etc.
- **Fonctionnalité clé** : Gère les états intermédiaires automatiquement.

Use case 6 – writeStream



```
streaming_df.writeStream \  
  .format("format") \  
  .option("key", "value") \  
  .start("wytasoft/to/destination") \  
  .awaitTermination()
```

- **format** : Destination (console, parquet, kafka).
- **outputMode** :
 - append : Écrit uniquement les nouvelles lignes.
 - update : Met à jour les lignes existantes.
 - complete : Réécrit tout le tableau.
- **checkpointLocation** : Chemin pour enregistrer l'état de traitement.

Use case 6 – writeStream

Intégration native avec Kafka.

Cas d'utilisation :

```
streaming_df.writeStream \  
  .format("kafka") \  
  .option("kafka.bootstrap.servers", "localhost:9092") \  
  .option("topic", "output_topic") \  
  .option("checkpointLocation", "/wytasoft/to/checkpoint") \  
  .start()
```

- Propagation de données transformées.
- Chaîne de traitement distribué.

Use case 6 – writeStream

- **Objectif** : Permet l'écriture continue des résultats d'un flux dans une destination.
- **Destinations supportées** :
 - Console (pour le débogage).
 - Systèmes de fichiers (CSV, JSON, Parquet).
 - Bases de données (JDBC).
 - Kafka, Elasticsearch, etc.
- **Fonctionnalité clé** : Gère les états intermédiaires automatiquement.

Use case 7 – foreachBatch

- **Qu'est-ce que foreachBatch ?**
 - Une méthode de Spark Structured Streaming.
 - Permet un traitement par lots sur les données en streaming.
- **Objectif :**
 - Appliquer une logique personnalisée sur chaque micro-batch.
 - Intégration avec des systèmes externes ou transformations avancées.

Use case 7 – foreachBatch

- **Flexibilité :**
 - Appliquer des transformations spécifiques à chaque batch.
 - Écrire simultanément vers plusieurs sinks (destinations).
- **Efficacité :**
 - Évite l'écriture directe des résultats en streaming vers des sinks lents.
- **Intégration :**
 - Compatible avec JDBC, Parquet, Delta Lake, et autres formats.

Use case 7 – foreachBatch

- **Flux de Données :**
 - Les données en streaming sont divisées en micro-batches.
 - Chaque micro-batch est traité comme un DataFrame.
 - foreachBatch applique une logique personnalisée sur chaque lot.
- **Traitement Parallèle :**
 - Exploite l'architecture distribuée de Spark pour une grande échelle.

Use case 7 – foreachBatch

```
from pyspark.sql import SparkSession

# Création de la SparkSession
spark = SparkSession.builder \
    .appName("Streaming avec foreachBatch") \
    .getOrCreate()

# Lecture depuis Kafka
streaming_df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "topic_name") \
    .load()

# Définir la fonction foreachBatch
def write_to_sink(batch_df, batch_id):
    batch_df.write \
        .format("jdbc") \
        .option("url", "jdbc:mysql://localhost:3306/database") \
        .option("dbtable", "table_name") \
        .option("user", "username") \
        .option("password", "password") \
        .mode("append") \
        .save()

# Utiliser foreachBatch
streaming_query = streaming_df.writeStream \
    .foreachBatch(write_to_sink) \
    .start()

streaming_query.awaitTermination()
```

- **Flux de Données :**
 - Les données en streaming sont divisées en micro-batches.
 - Chaque micro-batch est traité comme un DataFrame.
 - `foreachBatch` applique une logique personnalisée sur chaque lot.
- **Traitement Parallèle :**
 - Exploite l'architecture distribuée de Spark pour une grande échelle.

Use case 7 – foreachBatch

- **Optimisation des opérations sur les sinks** : Minimise la latence.
- **Gestion des erreurs** : Traitez les erreurs de manière batch-spécifique.
- **Idempotence** : Garantisiez que chaque batch est écrit une seule fois sans doublons.
- **Surveillance** : Utilisez Spark UI pour analyser les performances.

Use case 8 – Intégration avec d'autres outils

- Kafka comme plateforme centrale de gestion de flux de données.
- Pourquoi l'intégration avec HBase ou Cassandra ?
 - **HBase** : Stockage distribué de données structurées.
 - **Cassandra** : Bases de données NoSQL optimisées pour les écritures rapides.
- Avantages :
 - Traitement en temps réel.
 - Stockage et récupération des données pour des analyses ultérieures.

Use case 8 – Intégration avec d'autres outils

- **Cas d'utilisation :**
 - Archivage de flux Kafka dans HBase.
 - Accès rapide aux données historiques pour l'analyse.
- **Étapes techniques :**
 - Consommer des messages Kafka avec Spark Streaming.
 - Stocker les messages dans HBase à l'aide de connecteurs.

```
val kafkaDf = spark.readStream
  .format("kafka")
  .option("kafka.bootstrap.servers", "localhost:9092")
  .option("subscribe", "topic_name")
  .load()

kafkaDf.write
  .format("org.apache.hadoop.hbase.spark")
  .option("hbase.table", "my_table")
  .save()
```

Use case 8 – Intégration avec d'autres outils

- **Cas d'utilisation :**
 - Stockage des données Kafka dans Cassandra pour des analyses rapides.
 - Surveillance des applications et tableaux de bord en temps réel.
- **Étapes techniques :**
 - Lire les messages Kafka.
 - Écrire les données directement dans Cassandra à l'aide de Spark Cassandra Connector.

```
val kafkaDf = spark.readStream
  .format("kafka")
  .option("kafka.bootstrap.servers", "localhost:9092")
  .option("subscribe", "topic_name")
  .load()

kafkaDf.write
  .format("org.apache.spark.sql.cassandra")
  .options(Map("keyspace" -> "my_keyspace", "table" -> "my_table"))
  .save()
```

Use case 8 – Intégration avec d'autres outils

- Flux Kafka comme source de données centralisée.
- **HBase** pour l'archivage des données et l'accès aux historiques.
- **Cassandra** pour des analyses rapides et des réponses interactives.
- Workflow typique :
 - Kafka produit des données en continu.
 - Spark consomme les données pour les transformer.
 - Les données sont stockées dans HBase pour un accès historique ou dans Cassandra pour des analyses rapides.

Use case 8 – Intégration avec d'autres outils

- **Évolutivité** : Kafka gère les flux massifs, tandis que HBase et Cassandra gèrent efficacement les données en volume.
- **Flexibilité** : Architecture modulaire pouvant s'adapter à différents besoins.
- **Performance** : Traitement des données en temps réel, stockage rapide et récupération optimisée.
- **Fiabilité** : Combinaison de Kafka pour la résilience et des bases distribuées pour la tolérance aux pannes.

- Mehdi TAJMOUATI
 - mehdi.tajmouati@wytasoft.com
 - <https://www.wytasoft.com/wytasoft-group/>
 - <https://www.linkedin.com/in/mtajmouati/>
 - <https://medium.com/@mehdi.tajmouati.wytasoft>
-

